

VECTOR

"The Best Journal of APL"

- Cannon on Windows with APL*PLUS II 44
- Eastwood on Speeding up your APL 60
- Sandles on Genetic Algorithms 71
- Thomson on Integration 113
- McIntyre on J for Eigenvalues 125
- ... plus
- 16-page Educational Supplement 17



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.9 No.3 January 1993

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1992-93

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail, not needed for Europe)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 04393-385.

CONTENTS

Editorial: Commercial APL	Jonathan Barman	Page 3
APL NEWS		
Quick Reference Diary		5
APL Training Courses for 1993	Gill Smith	7
British APL Association News:		
Projects Officer's Report	George Macleod	10
News from Sustaining Members	Gill Smith	12
The Education Vector	Alan Mayer	17
REVIEWS SECTION		
APL Product Guide	Gill Smith	33
Windows Programming with APL*PLUS II	Ray Cannon	44
RECENT MEETINGS:		
Meeting at Morgan Stanley, 23 October	Jonathan Barman	56
Speeding Up Your APL	David Eastwood	60
Genetic Algorithms: their APlication	Jon Sandles	71
APL-Club Germany, Schwetzingen, November 6 1992	David Eastwood	79
GENERAL ARTICLES		
Seize the Time	Dick Bowman	86
A Little J Horner	Howard A. Peelle	89
From the Vital Execute to Fractals and 5-Fold Symmetry	Gérard A. Langlet	91
TECHNICAL SECTION		
Hackers Corner: Talking to PROGMAN	Adrian Smith	98
Technical Correspondence		100
ASWI Programming Standards; some Whys.	F.H.D. van Batenburg	101
APL*PLUS II Keyboard Configuration	Ajay Askoolum	107
Integrating with Insight	Norman Thomson	113
Some J Verbs for		
Orthogonal Factorial Experiments	Keith Smillie	117
Jacobi's Method for Eigenvalues an Illustration of J	Donald McIntyre	125
AMENDMENT: "A Change for the Better"	Donald McIntyre	134
An Example of Intransitivity in Probability	Graham Parkhouse	141
Index to Advertisers		143

APL★PLUS® II

The Comprehensive APL System for DOS and Windows™

APL★PLUS II Version 5		
Only APL★PLUS II gives you all this:		One system for ☒ DOS ☒ Windows
× Easy Object Oriented Toolkit × Call Every Windows 3.1 Function × Use GDI Graphics × Third Party Toolkits & Custom Controls × Common Dialog Boxes		Complete DDE Capability ☒ APL control of other applications ☒ APL controlled by other applications ☒ Sample functions for EXCEL & Lotus 1-2-3 ☒ Sample interface for Visual Basic
APL2-style Language Features Selective Specification Enclose/Disclose with Axis Enlist(ε) Find (⌵), Without(⌶) IIN, IOUT		
Database Interfaces ☒ The Paradox Engine ☒ Lotus .WK3 Files & dBase Files		Fully Integrated Debugger Step-through Primitive Statement Line ☒ Breakpoints Watchpoints based on: Reference Assignment Iteration Location Condition
☒ Support for ALL Popular Printers ☒ Workspace Cross-Referencing Tools		
☒ Line & Bar Charts ☒ Terminal Emulator		☒ Performance Monitoring ☒ TrueType Font Support
Optimized Assembler Fns	On-the-fly Macro HotKeys	Input Capture & Playback
User Command Processor	Numeric Data Editor	On-Line Help Facility

If you're wondering which way to go as you venture into the world of Windows, consider all your options. And then choose the product that lets you exercise them. All of them.

Whether you're coming from a PC or a mainframe, APL★PLUS II offers you everything you need to solve your problems and build integrated applications in DOS and Windows.

Get started today! In the UK, phone Cocking & Drury at 071-436-9481. Elsewhere, call your local dealer or contact Manugistics, Inc. at (301) 984-5412, Fax (301) 984-5094 for the name of a dealer near you.



Manugistics™
working as one

APL★PLUS is a registered trademark of Manugistics, Inc., 2115 E. Jefferson Street, Rockville MD 20852.
Other trademarks are the property of their respective owners.

Editorial: Commercial APL

by Jonathan Barman

I was deeply impressed with the BAA meeting at Morgan Stanley. We saw a vibrant APL community producing systems crucial for the survival of the business. Speed of development and speed of execution were both vital, and APL and its relative $\mathcal{A}$ were delivering the goods.

Morgan Stanley's in-house development of $\mathcal{A}$ was fascinating. Here is a small team developing a language which has got to be better than anything that is available commercially. New features can be included quickly, and as there is a relatively small number of users, changes to the language are not a big problem.

Commercial APL has to have features. Every vendor makes their APL special in some way. The process of including a new feature is often a lengthy procedure, requiring surveys of user requirements and analysis of discussions at user forums. There is always the problem of upwards compatibility. It is essential that existing customers can continue to run the code that may have taken considerable amounts of time and money to create. The vendor will want to select those features that will appeal to the widest possible audience and will help to sell the product.

The aims of the developers of $\mathcal{A}$ at Morgan Stanley are, therefore, rather different from the developers of commercial APLs. What is so interesting is to see the features that the Morgan Stanley developers have decided to include, and, equally interesting, the elements of APL which have been removed. How would you get on without being able to branch to a label? Phil Last and Gérard Langlet have both written articles in Vector showing that they abhor branching, so they would be delighted by such a move. But the majority of APL programmers would, I think, be in some difficulty without it. I cannot imagine any APL being sold where the branch arrow had been removed.

There are several facilities in $\mathcal{A}$ that I would love to be available in standard APL. The Name Association ideas in APL2 have been taken a stage further in $\mathcal{A}$, where they are called *Contexts*. I feel that this type of facility is one of the key things missing from APL. Dick Bowman, in his article *Seize the Time*, bemoans the fact that there are so few APL toolkits available. In my view this is a symptom of the absence of a facility to bundle up groups of functions into an independent library. In C one can buy libraries of functions distributed in object code format, which can be used but you cannot see the source code. Functions can be locked

in conventional APL, but this is not particularly secure. Libraries of machine code utilities are available, but when APL crashes there can be disputes about whether it is the interpreter or the library that caused the problem. A decent library facility could encourage the sale and distribution of toolkits.

One cannot buy *A*, it is not APL, so the interest is somewhat academic. But there are lessons to be learnt on how a language can evolve to solve real commercial problems, and therefore what features should be included in future versions of APL.

In the last issue of Vector I promised a review of the APL*PLUS II/386 Windows interface. I was looking forward to seeing how it stacked up against Dyalog APL/W as I knew it took a quite different approach. Also, I hoped to see some clever ideas which would mask the complexity and difficulties in programming Windows. Unfortunately, the whole thing is rather a disappointment. Obviously one can make it work, and every Windows function is available, but it would seem to take much more skill and effort than is normally associated with programming in APL. Ray Cannon's review was hampered because his machine did not have enough memory, at 4Mb, and he was working with Windows version 3.0 rather 3.1. The next release, APL*PLUS II/386 Version 5.0, looks as though it will be much easier to use, but until APL*PLUS becomes a full-blown Windows program the interface is bound to be rather convoluted.

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association
c/o Gill Smith
Brook House, Gilling East,
YORK YO6 4JJ**

Price in UK: £12 per complete volume (4 issues);
£14 (overseas), £20 (airmail) including Postage.

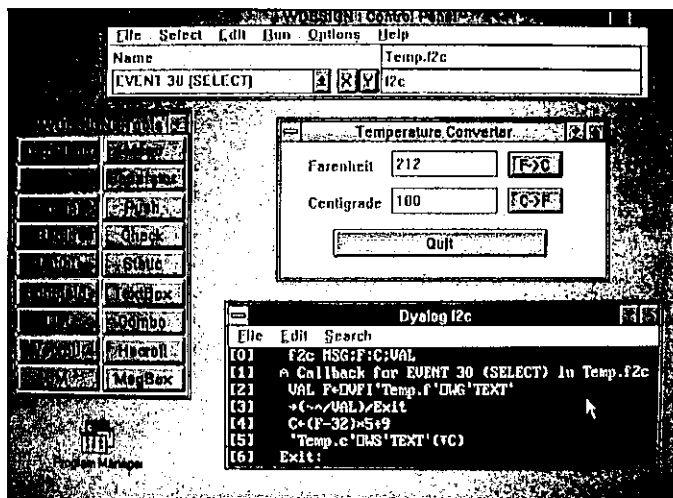
Quick Reference Diary 1993

Date	Venue	Event
12 February 1993	IEE London	APL in Forecasting & Planning
19 March	IEE London	Graphics Workshop: Speakers will include Graham Parkhouse & George MacLeod
23 April	IEE London	J Workshop (please bring a portable)
11 June	IEE London	AGM
August 15-19 1993	Toronto, Canada	APL93
17 Sept	IEE, London	Vendor Forum
26 November	TBA	APL Professional Development Workshop: This will be an all-day event (lunch is included) - a small fee will be payable

British APL Association meetings are held in the IEE, Savoy Place. Nearest tube outlets: Temple or Embankment.

Dates for Future Issues of VECTOR

	Vol.9 No.4	Vol.10 No.1	Vol.10 No.2
Copy date	5th March 93	4th June 93	3rd Sept 93
Ad booking	12th March 93	11th June 93	10th Sept 93
Ad Copy	19th March 93	18th June 93	17th Sept 93
Distribution	April 93	July 93	October 93



APL for Windows™

Dyalog APL/W is a true 32-bit Windows application and it shows! Click on the APL icon and up pops a powerful multi-window APL development system, with a host of features designed to enhance your productivity and enjoyment. But that's just the start. APL system functions provide you with simple and elegant, yet powerful tools to drive the Windows Graphical User Interface (GUI). In one statement you can create a GUI object such as a PushButton, specify its appearance and behaviour, and attach a "callback" function to be run when the button is pressed. There's even a WDESIGN workspace to help you. Windows programming has never been easier - and your applications will run unchanged under OS/2 and on Unix workstations in future releases of Dyalog APL.

and it's friendly too

Dyalog APL/W supports Dynamic Data Exchange (DDE), using standard APL Shared Variables, so it can talk to any other Windows application that supports DDE. Can you imagine the power of APL combined seamlessly with your favourite databases, spreadsheets and other tools?

DYADIC

All trademarks acknowledged.

APL Training Courses for 1993

For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	MicroAPL	1	11 Feb, 8 April, 27 May, 22 July, 16 Sept, 11 Nov
APL*PLUS/PC	MicroAPL	1	14 Jan, 11 March, 6 May, 24 June, 19 Aug, 14 Oct, 9 Dec
Introduction to APL2	MicroAPL	1	28 Jan, 25 Mar, 20 May, 15 July, 9 Sept, 28 Oct
APL*PLUS II/PC Conversion	MicroAPL	1	21 Jan, 18 March, 13 May, 8 July, 26 Aug, 21 Oct, 16 Dec
Intermediate	MicroAPL	1	18 Feb, 15 April, 10 June, 5 Aug, 23 Sept, 18 Nov
Advanced	MicroAPL	1	25 Feb, 22 April, 17 June, 12 Aug, 7 Oct, 25 Nov
Statgraphics	Mercia	1	20 Jan, 24 March, 20 April

If you would like to have your courses or seminars listed in Vector, please contact Gill Smith with the details.

No More FREEPOST!

Please note that the Freepost facility for correspondence to the Administration Assistant, Mrs Rowena Small, has been terminated, because of lack of use. Instead, please use the address shown inside the back cover of Vector, and affix the correct postage.

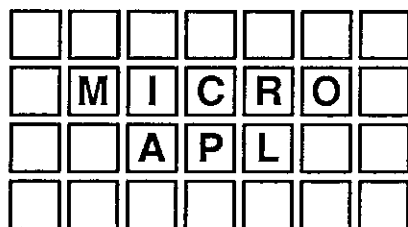
Some things are worth the wait

APL.68000 is not the only APL available for IBM's RISC System/6000. But no other APL running on the RISC System/6000 offers all these features:

- MicroAPL's acclaimed windowing interface
- Seamless integration with X-Windows
- Adherence to OSF/Motif style guidelines
- Very close compatibility with APL2/370
- Binary workspace compatibility across platforms
- High performance through MicroAPL's unique "Portable Assembler"™ technology

APL.68000 for the IBM RISC System/6000

*Fourth to announce.
First to get it right.*



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Fax: 071 928 1006

British APL Association News

Note to Members on the Data Protection Act

from Nicholas Small

The information in Vector 8.4, and on recent Membership Application forms is incorrect — for which I must take the blame, as I was in full possession of the facts, but failed to match them up.

The British APL Association is not an unincorporated members club, because it is part of the British Computer Society (BCS). Hence the exemption referred to in the earlier note does not apply. On the other hand, the BCS has to register under the Data Protection Act, because of its various activities, and this registration extends to cover the activities of the British APL Association, for which purpose it is more than adequate.

This means in particular that, subject to the conditions laid down in the Act, we are permitted to process on a computer personal data relating to our members with or without their express consent.

Outstanding Achievement Award

This is the most prestigious APL award in Britain. It is given annually to individuals who have made an outstanding contribution to either the development or application of APL.

Winners receive a cheque for £500, and may hold for one year the superb 'glass apple' trophy. The result is announced at the AGM in June.

Nominations are invited for this award, to reach Misha Jovanovic at 99 Oxford Road, Windsor, Berks SL4 5DX by the end of April 1993.

Cash Available — No interest! You don't even have to pay it back!

by George MacLeod

An Offer you Cannot Refuse?

The British APL Association has funds available to give as grants to members of the Association to fund or partly fund projects that fit in with the BAA constitution.

The constitution says that the objects of the Association are:

1. To promote the use of APL.
2. To develop awareness and competence in APL.
3. To represent the interests of Association members with other bodies.
4. To contribute to the development of the language and definition of international standards.

Recent Projects

The major recent project, which is still on-going, is the APL Statistical Library where the BAA funding included paying a programmer and paying the costs of the annual conferences.

A recent small project was the printing of the I-APL manual in Cyrillic.

Another project currently under way is being carried out by Dr Ian Clark which involves the porting of I-APL to System 7 operating system on the Macintosh. This will form the heart of Ian's development of a tool to assist in the production of educational monographs which will be accompanied by I-APL workspaces which will demonstrate the principles being taught. Ian's I-APL TrueType font is freely available via I-APL Ltd (see Product Guide); a small sample is illustrated overleaf. Most of the funding for this project has come from other sources but that funding was contingent on some funding by the BAA as a mark of its approval.

APLomb: an I-APL TrueType Font by Ian A Clark

*Here is a sample from page 20 of Education
Vector set at 13/15 point:*

```
Z+HERO FIGHTS ORC
[1] DRAW: BAR HERO[1],ORC[1]
[2] +STOP IF 0 = HERO[1] × ORC[1]
[3] SCORE ← RESULT HERO[2], ORC[2]
[4] +(WIN, DRAW, LOSE) IF SCORE = 1 0 -1
[5] WIN: ORC[1] ← ORC[1] - 1
[6] +DRAW
[7] LOSE: HERO[1] ← HERO[1] - 1
[8] +DRAW A Run as e.g. 12 3 FIGHTS 15 4
[9] STOP: Z ← HERO[1],ORC[1]
```

How Do I Get Some Money?

Contact George MacLeod, the BAA Projects Officer, at the address at the back of Vector and tell him what you have in mind. If it seems like the project is within the rules then a description of the project needs to be produced. The Projects Officer can assist with this. Once this is done the Projects Officer presents the description to the BAA Committee at its next meeting and usually approval is forthcoming.

Where's the Catch?

There really is not a catch. A condition of the grant will be a description of the project for Vector and if it is appropriate a presentation at one of the BAA meetings. Sometimes it is possible to put part of the project in the public domain for the wider use of the APL community. Part of Ian Clark's project was the production of a TrueType I-APL font which is now freely available for people who need an I-APL font for producing documents under Windows 3.1 and elsewhere.

Will this help me to my First Million?

It might, but the grants are not really intended to fund commercial ventures. However, it is possible that the BAA may act as lender of last resort, if the project is believed to be worth while, but other sources of funds are not available. Also the funds available are not enormous. A grant or a loan would be measured in hundreds or thousands of pounds, rather than tens of thousands.

News from Sustaining Members

Compiled by Gill Smith

Manugistics Inc

A lot of feverish activity has been happening at Manugistics over the past few months, mostly involved with preparing our latest version of APL*PLUS II. What started out as a small project grew rapidly into a major release as we saw the potential in our Windows support.

We started shipping Release 5 of APL*PLUS II early in December. The new release features an easy-to-use GUI Toolkit that makes building Windows applications an enjoyable task. The interface is much faster than Release 4 and your applications will run quickly and smoothly. New DDE tools include powerful sample functions to exchange data and instructions with EXCEL, Lotus 1-2-3 and Visual Basic. Functions to access the Common Windows Dialogs (like Open File Name and Choose Colour) are included.

Perhaps the most generally useful new feature is the fully integrated Debugger which is by far the most advanced APL debugging tool ever offered. It lets you step through your code primitive by primitive, literally watching your data change as it passes through the functions. The new Watchpoint feature lets you have the Debugger stop whenever a particular variable is referenced or modified, in a specific function, every so many iterations, and/or when a particular condition is met. Breakpoints can be set on lines and on statements within lines and execution can be recommenced from any stop point, even between two primitives.

For the DOS environment we have incorporated Rick Butterworth's Graflib tools which make it easy to convert your data into line and bar charts, complete with titles, legends, gridlines, etc.

Among the other features you will find our new Install Program which will get you up and running on Version 5 in no time.

Now that Version 5 is out the door, we are turning our product hats to a new version of APL*PLUS/PC. Version 11, due out early next year, will include the Screen Interface Toolkit introduced in APL*PLUS II Version 4 and a Free Runtime .EXE Builder that will let you distribute your APL*PLUS/PC applications at NO COST.

In addition to the PC release, we will be working hard on Version 5 of the UNIX product. Watch this space for more information.

One of our major goals for 1993 is to bring APL to a much wider audience. Chris Lee has just finished touring Europe and the U.S. presenting to any current APLers. Next year the emphasis will be on taking the message to the (slightly larger) group of non-APLers out there. To start with, we will be presenting APL*PLUS at the Software Developer's Conference in San Francisco in February. This will be attended by programmers who are currently using C, Pascal et al. We think that they deserve to see what array-oriented thinking can do to their productivity. We are also presenting a tutorial at the Application Developer's SIG (SIGAPP) at the ACM's annual conference in Indianapolis, Indiana. These two events will just be the start of a major campaign to broaden the market for APL.

Lastly we have just published our first Catalogue of Third Party Software. We are sending this out with all new software orders so you will get one when you get your next upgrade. If you have software you would like to see in the next catalogue, please contact us for more information.

Happy New Year!

MicroAPL Ltd

Users of our APL.68000/X product for the IBM RISC System/6000 should all have received notice of the latest upgrade by now (to version 1.2). This upgrade has been triggered by the recent release of AIX 3.2 which has been a significant upgrade to AIX and appears to have fixed a number of bugs, some of which had been showing through into APL.68000/X. The 'look and feel' of Motif programs has also been improved. Our new version of APL.68000/X also includes an improved interface to routines written in 'C', BDF-format fonts for X-terminals and a number of minor bug fixes. The somewhat thorough nature of the AIX upgrade means that older versions of APL.68000/X must be upgraded to version 1.2 if you are upgrading AIX. As is our usual practice for most of our routine upgrades, we are not charging for this upgrade.

In what is planned as the first of a series of free seminars, we recently held a half-day seminar on 'Trends in Workstation Technology'. This included an overview of the workstation market given by Richard Nabavi which looked at the various leading hardware platforms as well as the new generation of operating systems.

We contrasted the various GUIs in use and assessed the significance of the RISC System/6000 in the workstation market. Readers of Vector who would like copies of the notes for this seminar should contact David Eastwood at MicroAPL.

We shall be starting 1993 with one or more seminars devoted to the new breed of GUI development tools that are now available and offering practical advice based on recent Windows projects we have undertaken. Again, contact David Eastwood for details.

Cocking & Drury Ltd

By the time you read this we will have shipped upgrades to APL*PLUS II Version 5. This new version includes the APL Graphical User Interface Toolkit — a collection of object-oriented tools that make it easy to design and build Windows applications. There is also a complete interface to the core Windows system libraries, so you can reach beyond the APLGUI tools and use any of the hundreds of Windows library routines. The Windows interface is profoundly faster than in Version 4 thanks to a new virtual device driver. DDE utilities enable easy linking of APL*PLUS II with other Windows applications.

The DOS user is not forgotten however — the same package still allows the creation of DOS programs, and OS/2 users will now find APL*PLUS II running happily in an OS/2 DOS session. Other enhancements include the APL*PLUS II Debugger — a visual code debugger woven into the fabric of the interpreter. The GSS*CGI interface has been revised and is now supplemented by a new *)PLOT* command which provides high-quality line and bar charts in any workspace. And those of us who struggled somewhat with Version 4's installation will be delighted to hear that an automatic installation routine is included which allows for the selection of components.

Manugistics are rightly pleased with the outcome of the effort they have put into this new version and we are co-operating with them in a special offer for existing users of APL*PLUS/PC, who may purchase copies of APL*PLUS II Version 5 at the discounted price of £700 plus VAT.

Anyone who missed the talk by Chris Burke of Strand Software, Canada, on the use of APL in actuarial work can obtain a free copy of the utility disk by calling us on 071-436-9481. Cocking & Drury now have available a range of tools to assist in the migration of APL2 mainframe workspaces to APL*PLUS II 386/486 — we have seen a number of companies reaching the decision to 'down-size'

their APL use to networked PCs for greater flexibility (and considerable CPU savings!) and we have developed these tools in response to that need.

Levels of APL consultancy work and of APL training have remained fairly constant over the last 12 months. Our consultants are kept busy in other fields of endeavour and we have recently expanded into a new area of training which is proving to be very popular.

Dyadic Systems Ltd

Dyadic is pleased to announce the availability of Dyalog APL/W Version 6.3 for Microsoft Windows. This release contains a host of new features of which the following are a selection:

- A new system function, `⌈NA`, is included. This provides access to Windows Dynamic Link Library (DLL) routines. Using `⌈NA`, you can call any DLL subroutine (16 and 32-bit) using familiar APL function syntax.
- Full control over foreground and background colours in GUI objects.
- Five new "graphical" objects, namely:
 - POLY (lines, polygons, filled areas)
 - MARKER (for marking a series of points)
 - RECT (rectangles)
 - ELLIPSE (ellipses, circles, arcs, pie-charts)
 - TEXT (arbitrary graphical text)

All the graphical objects allow you to draw several things at once (and with a single name). By default, APL handles repaint for you and automatically resizes your graphics when the user resizes a FORM. Graphical objects can even be dragged and dropped! A new GRAPHS workspace containing high-level "business graphics" functions is also provided.

- New BITMAP, CURSOR and ICON objects which you can import from standard files or create from APL matrices.
- Ability to associate bitmaps with BUTTONs and MENUITEMs, so that you can provide customised "toolbars" for your users. You can specify individual bitmaps for when a BUTTON is "out", "in" and "inactive", so that the appearance of the BUTTON changes instantly when it is pressed.
- A new FILEBOX object which provides a standard Windows 3.1 File Selection dialog box.

- Direct support for the printer via a new PRINTER object which provides for the output of device-independent text and graphics.
- Clipboard support via a new CLIPBOARD object. This allows you to move data to and from the Windows Clipboard under program control. The CLIPBOARD object has a CHANGED event so that your application can react immediately when (for example) the user copies data to the Clipboard from Excel or from any other Windows application.
- A new AUTOCONF property to simplify control over resize. An object can now optionally enable/disable automatic resize independently on both itself and on its children.
- The coordinate system has been extended to provide user-definable coordinates and origin.
- Improvements to the handling of CHANGED, GOT_FOCUS and LOST_FOCUS events to simplify validation etc.
- A new general DATA property allows you to associate any APL array with an object. This reduces the need to use global variables to communicate between callback functions.
- `⌈NQ` has been extended to allow you to send a message to an object instantly, bypassing the normal queue. If there is one, the appropriate callback function is invoked at once. This extension permits an even more "object-oriented" approach to programming.

Version 6.3 completes the definition of the Dyalog APL GUI support, and will be the basis for the forthcoming OSF/Motif, OS/2 and Windows NT implementations. These versions of Dyalog APL will be 100% compatible with one another and with Dyalog APL/W for MS-Windows. This will permit true portability of APL applications between different GUIs and different hardware platforms.

THE EDUCATION VECTOR

January 1993

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.9 No.3. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Alan Mayer	18
2 Hero vs Orc — A Grade 9 Battle Sequence in APL	Walter Spunde	19
Teaching Mathematics with APL: Workshop Design	Howard Peelle	29

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel: 0792-205678 Ext.4274 Fax: 0792-295626 JANET: MAYER@UK.AC.SWAN.MS

Editorial

Happy New Year, and welcome once again to Education Vector. In this edition we have two substantial contributions from authors who have considerable experience in teaching with APL. I am grateful to them both for sharing their experiences with us.

Walter Spunde has been teaching I—APL programming to school children (I am informed that a "Grade 9er" approximately equates to Form Five in a British school), by developing computer fantasy games. His direct definition approach encourages good structure and allows initially very simple games to be developed into ever more complex strategies.

Howard Peelle uses APL to teach Mathematics, and his publications on APL will be well known to many of you. In this initial article, he describes the environment and equipment required, the design of introductory workshops and the constructivist teaching methods on which they are based. The actual teaching materials, of which he has a large variety, will be of interest to many readers, and a selection will be published in future editions of Education Vector.

I often have enquiries, especially from members of educational establishments, about low-cost APL interpreters. There are several to choose from, and each has advantages and disadvantages. I intend to publish a detailed comparison in the next edition, and will be pleased to hear from anyone who has experience (good or bad) that they would like to share with us. For now I offer the following list of products and sources:

- I—APL (free). Enquiries: 11 Auburn Rd, Redland, Bristol, BS6 6LS.
- TryAPL2 (free — specify disk size). IBM APL Development, M46/D12-278B Santa Teresa Lab, Box 49023, San Jose, CA 95161-9023 USA.
- APLIPC, APLI386 (requires 80386) and APLIWIN (requires WINDOWS 3.1). These products cost \$30 each and are available from Iverson Software Inc., 33 Major Street, Toronto, Ontario, Canada, and also from I-APL at the address above.

I am particularly interested to hear from anyone who has tried APLIWIN, which has only just been released.

Hero vs Orc — A Grade 9 Battle Sequence in APL

by Walter G. Spunde, University of Southern Queensland

Teacher-prepared software for the mathematics classroom suffers by comparison with the sophisticated games software to which many children have become accustomed. There may be little prospect of competing with the commercial vendors, but animation and full screen colour scenes are only peripherals for attracting attention and stimulating mental activity. Creative power may be equally stimulating. The following sequence of microcomputer laboratory activities (using I-APL) has been run very effectively on several occasions with Grade 9 boys at a local Grammar school. The adventure game setting may not appeal to some teachers, but it certainly does appeal to those boys who enjoy playing computer fantasy games and who can therefore see the potential use for the material developed.

The problem presented is to produce strength bars tracing a battle between an adventurer and a goblin, or an orc, or some other monster.

In any APL activity, teachers must decide which functions (or vocabulary) to provide for students and there is no suggestion that the material given here should be introduced from scratch. Depending on the level of the students, many, or even all, of the functions described could be provided by the teacher, with students working only on improvements.

Tracing the Battle

Let us assume that we have a function BAR which, given two numbers, (the combatants' strengths) draws bars with lengths proportional to them, and a function RESULT which, given two numbers (the power or skill of the combatants) returns the result of a clash between them, 1 for a win to the HERO, -1 for a loss and 0 for a draw.

There are two distinct ways of tracing through the battle. One, which encourages thinking in terms of loops, involves a multi-line function (*FIGHTS*); the other a direct definition function (*FIGHT*) encourages recursive thinking.

Looping:

```

      Z ← HERO FIGHTS ORC
[1] DRAW: BAR HERO[1], ORC[1]
[2] →STOP IF 0 = HERO[1] × ORC[1]
[3] SCORE ← RESULT HERO[2], ORC[2]
[4] →(WIN, DRAW, LOSE) IF SCORE = 1 0 -1
[5] WIN: ORC[1] ← ORC[1] - 1
[6] →DRAW
[7] LOSE: HERO[1] ← HERO[1] - 1
[8] →DRAW Run as e.g. 12 3 FIGHTS 15 4
[9] STOP: Z ← HERO[1], ORC[1]

```

Recursive:

```

      FIGHT: α FIGHT S AFTER □+BAR S : 1 ∈ (S+α CLASH ω)=0 : BAR S
or  FIGHT: α CONTINUE S : 1 ∈ (S+α CLASH ω)=0 : CONCLUDE S
with CONTINUE: α FIGHT ω AFTER □+BAR ω
      CONCLUDE: BAR ω

```

Here α are the powers and ω the strengths of the players;

$CLASH: \omega - (-1 \ 1) \in RESULT \ \alpha$

reduces the strengths as appropriate, and

$AFTER: \ \alpha$

is simply a conjunction. Run as e.g. 3 4 FIGHT 12 15.

The direct definition version requires that the vector of a player's attributes (strength, power — and perhaps food, healing potions, etc.) must be split up to run *FIGHT*. This could be easily handled by a cover function such as *ENCOUNTERS*: $(\alpha[2], \omega[2]) FIGHT \ \alpha[1], \omega[1]$. It is interesting to note that when students can see the need and significance of two, and more, numbers to describe the attributes of something, they have no trouble at all in working with higher dimension vectors.

Drawing the Bars

Leaving 4 spaces between bars, with a maximum displayed length of 36, as in

36	4	36
----	---	----

... we overwrite as many blanks on from 42 as the orc has strength with say the symbol Ψ , and as many blanks back from 37 as the hero has strength with a different symbol:

BAR: A AFTER A[38-1ω[1][36]←'4' AND A[41-1ω[2][36]←'7' AFTER A←77ρ' '

For example, $BAR \ 12 \ 15$ gives




[For general purpose functions such as *AFTER*, *AND* and *IF*, see the appendix.]

Modelling the Fight

In "fighting fantasies" the combat is simulated through the rolling of different coloured dice, i.e. producing two random numbers between 1 and 6 and comparing sizes. (Sometimes 12 or 20 sided dice are used.)

We produce 2 random numbers with *SCORES*: ? ω

and calculate their differences with $DIF: \omega[1] - \omega[2]$

Determine the sign of a number with *SIGN*: $\times \omega$

Find the result of an encounter with *RESULT: SIGN DIF SCORES* w

Thus a "compiled" *CLASH* function could be given as:

CLASH: $\omega - (-1 \ 1) \in \mathbf{x} - / ? \alpha$

A typical display
from the running
of either
110 100 FIGHT 8 9
or
8 110 FIGHTS 9 100

Grade 9ers can get to the stage of writing their own *FIGHTS* function in less than an hour in the laboratory with no previous experience of the APL notation.

Simple Extensions

A variety of minor extensions can be made to these functions. The simplest is to add a message to the end of the conflict. Using direct definition a simple solution might be

```
MESSAGE: 'Your adventure ends.' : 0<1+w : 'That''s one sorry orc.'
```

This could be executed at the *STOP*: line of *FIGHTS* as:

```
MESSAGE HERO[1]
```

or in the *CONCLUDE* function of *FIGHT* as

```
CONCLUDE: MESSAGE ω AFTER []+BAR ω
```

Students enjoy writing soul-destroying messages to the vanquished and the messages put up could be chosen at random from a matrix of message lines with *MAT*[?N;] where *N* is the number of lines in the matrix, *MAT*.

The number of refinements is limited only by enthusiasm. Each character may have a different symbol which is kept with the character information (strength, power, etc.) and then inserted in the left argument of *BAR*. The functions *FIGHT*(*S*) are symmetrical: there could be a modified one that gives an advantage to an attacker.

The hero's strength could perhaps be recovered by healing potions or food. Functions *FIGHT*(*S*) could be modified so that they return not only values for the hero's strength, but also perhaps increased skill. Perhaps superior equipment can be purchased, thereby increasing the power of a character. The vector of information stored about a character can become quite long.

The question of a player's skill opens up several avenues of investigation. A player with a high power score could still score poorly in a clash. To reflect a level of skill, a player's score might be given as 12+?7 rather than ?29, giving a distribution with the same mean but different range. The teacher can provide functions to generate random numbers from any standard statistical distribution [see Anscombe, "Computing in Statistical Science through APL", Springer-Verlag, 1981] without the student being aware of any of the intricacies involved in generating them; and of course students can try creating their own.

Delegating tasks to different student groups presents them with the need to write functions with inputs specified by others, and with outputs that another group will have to use.

Re-modelling the Battle

The nature of the battle is, despite the above refinements, still a crude process of simply determining who scored the highest — like a simple game of cards. A more sophisticated battle might allow several choices of action, the success of which may well depend on the action chosen by the opponent. A popular game to model our battle on might be the age-old rock-paper-scissors, where any one of three strategies might win depending on the other's choice. In a battle setting, the three options might be described as

- 1) to lunge with your sword
- 2) to block with your shield and parry
- 3) to cast a fireball spell.

With the same three options for the adversary, the payoff matrix for the "row player" is

0	-1	1
1	0	-1
-1	1	0

There is no reason why the two combatants should have the same set of three options available to them. All that is needed is a payoff matrix which determines the outcome for any pair of strategies. The positive entries could represent the number of strength points the HERO succeeds in deducting from his adversary for a win, and the negative entries the number he loses for a defeat. Students who have become excited by the activity have no problem in setting up and using larger payoff matrices (8×8 has been observed).

It is apparent that, in game theory terms, what we are dealing with here is not a zero-sum game. Both players can only lose strength, unless they decide not to fight at all, although they may gain in wealth and experience.

Developing a set of options and a payoff matrix is fun for children who enjoy "fighting fantasies", and may provide the opportunity to consider other types of games — zero sum games and games with saddle points. When it comes to determining optimal strategies to influence a skilful player's choice, the level of mathematics involved soon goes beyond what Grade 9ers are usually exposed to. With a little bit of help 2×2 games can be solved graphically, but beyond that we must delve into linear programming for optimal solutions to n option games. If we allow more than two combatants, or non-zero sum games, where cooperation may be beneficial (unless double-crossed) then we approach the frontiers of mathematical research. The potential for extension is boundless.

Setting up a Matrix Game

A set of functions that implement a matrix game could be used to investigate the possibility of optimal strategies through a sequence of simulations. One possible set of such functions is given below and may also demonstrate how one line of mathematics can become a heap of functions for the sake of displays — the great time-waster in using computers for mathematics.

In a matrix game, a decision in the encounter between the two combatants is obtained by simply looking up the entry in the corresponding position in the payoff matrix. If the choices of the protagonists are given in the left argument (α) and the current strengths in the right argument (ω), revised strengths are returned by

CLASH: $\omega - ((-R \leq 0), (R \geq 0)) \times R + \text{PAYOFF}[\alpha[1]; \alpha[2]]$

where *PAYOFF* is assumed to be a defined global variable.

The choices for action in any particular encounter are delivered by a function *CHOICES* which may take different forms, but for speed of operation, we collect the player's choice with a single key input and for convenience, we restrict the inputs to the numbers 1-9, thus limiting the dimensions of the *PAYOFF* matrix to $9 \times n$. The input could be collected with the function

GETNUMBER: *GETNUMBER*: $1 = (A \geq 48) \wedge (48 + 1 + p \text{PAYOFF}) \geq A + \text{INKEY} : \text{AV}[A+1]$

(Functions such as *INKEY*, *AFTER*, etc. are listed in the Appendix.) The number of options for the opponent is not limited and a random choice of action is given by $?^{-1} + p \text{PAYOFF}$. Thus

CHOICES: *GETNUMBER*, $?^{-1} + p \text{PAYOFF}$

The game is either continued or concluded, depending on whether either player's strength has been reduced to zero or below:

GAME: *CONTINUE* *S* : $1 \in \{S + \text{CHOICES CLASH } \omega\} \leq 0 : \text{CONCLUDE } S$

As before, continuation of the battle requires us to draw the strength bars and re-play the *GAME*:

CONTINUE: *CHOICES* *GAME* ω *AFTER* *DRAWING* ω

DRAWING the strength bars may be done simply as before, or we may decide to use the *PGRAPH* functions. It is when we make the decision to "improve the

presentation" that we open the door to masses of details about layout, and to generating possibly messy code, of no mathematical interest, in large quantities. It is better to allow students to by-pass the definitions of these functions by providing them with the functions to fiddle around with if they choose to do so.

One possible solution to the problem of drawing the strength bars is to draw two bars at a given spot on the screen, and black out the ends of the bars as strength points are lost:

```
SETUPBAR: 1 HBAR 300 300 +  $\omega$  AFTER  $\bar{1}$  HBAR 300 300
```

where *HBAR* draws "points" of 200×20 pixels, in white when the left argument is $\bar{1}$ and in black when it is 1:

```
HBAR: (32767 250  $\bar{1}$ ,  $\alpha$ , 200 20 0 30 0 20) FGPlot (( $\rho\omega$ ), 1) $\rho\omega$ 
```

The x-coordinates of the bottom left corner of the "points" is given by the right argument, and the y-coordinate is set at 250. This, of course, may not be suitable for all screens or tastes.

The drawing function is, assuming the HERO's strength is shown on the upper bar:

```
DRAWING: 1 HBAR [(300 300) +  $\phi\omega$ 
```

To end the game, returning the final strength of the HERO as the result of the function, we define

```
CONCLUDE: (0[1+ $\omega$ ) AND MESSAGE  $\omega$  AFTER DRAWING  $\omega$ 
```

where *MESSAGE* chooses some text to be displayed. For example, we could have

```
MESSAGE: []  $\leftarrow$  (2 8 $\rho$ 'Got him! Too bad!')[1+0 $\geq$ 1+ $\omega$ ;]
```

The game could be run by executing the function

```
PLAY: CHOICES GAME  $\omega$  AFTER SETUPBAR  $\omega$ 
```

Displaying the Game

Running this implementation of the game is a bit unsatisfactory, because, while we can see the strength bars changing in size, there is no indication of why. This sort of game demands that not only should the strength bars be displayed, but

also the actions taken by each of the combatants. Ideally this would be in terms of the movements of graphical figures, but a simpler, although not entirely satisfactory, solution is to give a text description of the action.

In order to do this there must be lists describing each of the options available to the two players. Assuming these are held in the global variables ΔHERO and ΔORC, the descriptions can be written at an appropriate spot on the screen with:

```
DISPLAY: w AND(200,255)WRITE 10+ΔHERO[w[1];]AND(350,255)WRITE ΔORC[w[2];]
```

The pixel coordinates (200,255) and (350,255) have again been chosen to suit a particular layout relative to the strength bars.

The display of the choices is handled by the function

```
SHOWCHOICES: ,10 : 10+1+A+CHOICES : DISPLAY A AFTER CLEAR
```

where

```
CLEAR: (3 55p' ') PUT 5 20
```

is used to clear the chosen writing space.

The controlling function is adjusted to

```
PLAY: SHOWCHOICES GAME w AFTER SETUPBAR w
```

and the continuation function to

```
CONTINUE: SHOWCHOICES GAME w AFTER DRAWING w
```

Another feature has been added to the game by allowing the player to Escape from the battle instead of continuing. This is done in modifying the GETNUMBER function to return a single number (10) if the Esc key is pressed. SHOWCHOICES also returns 10 in this case.

```
GETNUMBER: CHECK A : 1=(A≥48)^57≥A+INKEY : (1□AV[A+1]) ANY 1+pΔPAYOFF
CHECK: GETNUMBER : w=27 : 10
ANY: α : (α>w)∨α=0 : ?1+w
```

Provision has also been made to return a random choice if a number outside the range of options is entered, through the function ANY.

The CLASH function must be modified to take note of the escape option:

```
CLASH: w[1],-10 : 2=pα : w-((-A≤0),(A≥0))*A+ΔPAYOFF[α[1];α[2]]
```

The appearance of the game can be made a little less bare by providing more information on the screen. The function

```
SCREEN: SETUP w AND SETUPBAR a AND 'Hero' SETUPWRITE 'ORC' AND CLS
```

calls a number of display functions listed in the Appendix, and replaces *SETUPBAR* in the original *PLAY*:

```
PLAY: SHOWCHOICES GAME w AFTER A+CRS 15 0 AND w SCREEN AHERO
```

Finally, to help set up a game, a separate *SETUPGAME* function may be defined. It is simplest to do this in a multi-line function such as the one listed in the Appendix below.

APPENDIX: A Set of Functions for Matrix Games.

1. Control

```
PLAY: SHOWCHOICES GAME w AFTER A+CRS 15 0 AND w SCREEN AHERO AND PGWHEEL 1+?16
GAME: CONTINUE S : 1<(S+α CLASH w)≤0 : CONCLUDE S
CLASH: w[1], -32767 : 2=ρα : w-((-A≤0),(A≥0))*A+APAYOFF[α[1];α[2]]
```

2. Input

```
CONTINUE: SHOWCHOICES GAME w AFTER DRAWING w
SHOWCHOICES: ,10 : 10*1+A+GETNUMBER,?1+APAYOFF : DISPLAY A AFTER CLEAR
GETNUMBER: CHECK A : 1=(A≥48)^57≥A+INKEY : (±□AV[A+1]) ANY 1+APAYOFF
CHECK: GETNUMBER : w=27 : 10
ANY: α : (α>w)∨α=0 : ?w
DRAWING: 1 HBAR[(300 300)+φw
DISPLAY: w AND (200,255) WRITE 10+AHERO[w[1];] AND (350,255) WRITE AORC[w[2];]
CLEAR: (3 55p' ') PUT 5 20 AND PGWHEEL?16
```

3. Output

```
CONCLUDE: (0[1+w) AND MESSAGE w AFTER DRAWING w
MESSAGE: □+'The end of your adventure' : 0<1+w : RUNAWAY w
RUNAWAY: □+'OK, run!' : -32767*~1+w : □+'That's one sad and sorry orc.'
```

4. Screen

```

SCREEN: SETUP w AND SETUPBAR a AND 'Hero' SETUPWRITE 'ORC' AFTER CLS
SETUP: [←'Enter you choices:',[AV[14], ' (0 for random)' AND SETUPSKILLS w
SETUPSKILLS: 1 2 SHOW 'w-a-' BORDER NUMBER w
SHOW: (( ' ' ' ',w, ' ') ) PUT a
BORDER: (a[1] , (a[4], ' ',w, ' '), a[2]) , a[3]
NUMBER: ( (N,1) p: N+1+pw ), ' ', w
SETUPBAR: 1 HBAR 300 300+pw AND 1 HBAR 300 300
SETUPWRITE: ((290-16*pw),285) WRITE w AND ((290-16*pa),315) WRITE a

```

5. Setup

```

SETUPGAME: Options
11: ΔPAYOFF+ASK 'Name of PAYOFF matrix '
→Error1 IF 2*ΔNC ΔPAYOFF
→Error2 IF-NUMERIC ΔPAYOFF+ΔPAYOFF
→Error3 IF 2*ppΔPAYOFF
12: ΔOptions+ASK 'Name of OPTIONS list '
→Error4 IF 2*ΔNC ΔOptions
→Error5 IF NUMERIC ΔOptions+ΔOptions
→Error6 IF (1+ΔOptions)*+ΔPAYOFF
→Complete
Error1:+(11,0)[ESC] AND [←'Not a defined matrix.'
Error2:+(11,0)[ESC] AND [←'Not a numeric matrix.'
Error3:+(11,0)[ESC] AND [←'This is a higher dimensional array.'
Error4:+(12,0)[ESC] AND [←'Not a defined list.'
Error5:+(12,0)[ESC] AND [←'List of options for two players required.'
Error6:+(12,0)[ESC] AND [←'No. of options doesn't match PAYOFF.'
Complete: ΔHERO+ΔOptions[1+ΔPAYOFF:] AND ΔORC+((1+ΔPAYOFF),0)+ΔOptions
'-- Global variables ΔPAYOFF, ΔHERO and ΔORC have been defined.'

```

6. General

```

AND: a
AFTER: a
CRS: : *pp(T+204,2+pw,-1-1) Do ' ' : 1+T
CLS: ( ' ' Fill 0 0 25 80 ) : 2<1+pa+CRS 0 0 : CLS
HBAR: (32767 250 -1,a,200 20 0 30 0 20) PGLOT ((pw),1)pw
WRITE: (a,0 240 1 1 1) PGTEXT w
PUT: (201,w) Do a
INKEY: (T,,11519 [MC 2 1p'TT',0pT+11 0])[IO+1]
PGLOT: 101 Do w : *NC 'a' : (101,a) Do w
PGTEXT: (105,a) Do w
PGWHEEL: 103 Do w
Do: a+11519 [MC 1 2 1p'a'w'
Fill: (203,w) Do a
ASK: w : 0*PZ+(pP)+[ AND [←P+w,': ' : Z
NUMERIC: 0=0\0pw
IF: w/a
ESC: 1+27=INKEY AFTER [←(20p' '), '-- Tap any key to continue; Esc to quit.'

```

Teaching Mathematics with APL: Workshop Design

by Howard Peelle, University of Massachusetts.

Origins

The following describes the design of a series of introductory workshops using APL to teach selected topics in mathematics. Constructivist teaching methods are emphasized. This design is modelled on a 15-week teacher-training course "Teaching Mathematics with APL", offered at the University of Massachusetts' School of Education since 1987. The workshops described here may be conducted in such a series, or with fewer sessions — even a single "one-shot" workshop.

Goal and Purposes

The overall goal of the workshop(s) is to create an exemplary learning environment. Specific instructional purposes are to:

- introduce APL as an executable mathematical notation
- present a subset of APL in order to study selected topics
- reveal how APL software can be used for experimentation
- show how APL can empower students' learning and teachers' teaching
- gain insights into students' cognitive processes
- discuss mathematical content from a new perspective, and
- provide resources for continued learning.

Audience

These workshops are designed expressly for mathematics teachers in training (pre-service or in-service), although the approach is suitable for their students as well (when given ample time). Hence, both teachers in the workshop and students will be referred to as "learners" here. About 15 learners can be accommodated in a workshop, or more if additional instructors can assist.

Equipment and Software

The number of microcomputers needed is best determined by equipment available at the workshop site and/or by the number of learners. Brand of microcomputer is not critical, since the APL language is fundamentally the same on most microcomputers. Typical brands are: IBM compatibles (MS-DOS); Apple IIe; Macintosh. A free copy of I-APL (International standard APL) is provided for each learner to take home. It is public domain software, which may be copied and distributed to other students and teachers. Recipients will be asked to supply their own disks.

Role of Computer and APL

Ideally, there will be one microcomputer for everyone in the workshop, although learners are encouraged to pair up. In any case, spontaneous *dialogue* among learners is welcomed, with the computer acting as a third party. Since APL is interactive, it promotes *active* learning. Typically the interaction involves typing a short (one-line) expression, getting an immediate response, thinking about it, and then typing another expression, in succession. Having a computer available not only facilitates testing and *experimentation* but also allows attention to higher level concepts.

Familiarity with microcomputers is desirable, but not required since the software will be "up and running" at the outset, and the workshop leader will be available for help with editing programs or any other problems related to computer usage. N.B. The emphasis is on learning mathematics — not computers. Learners are not expected to be familiar with APL, nor will they be expected to learn much APL. Only a small subset of APL is needed for a given topic; and, typically, only simple programs can be developed in the short time available. N.B. The emphasis is on learning mathematics — not the programming language.

Role of Leader and Workshop Format

The workshop leader is responsible for arranging and organizing the workshop(s) in advance, as well as conducting them. Basically, conducting a workshop involves giving initial instructions to the group as a whole; then circulating around to help individual learners work with APL at computers, especially offering suggestions for experimentation; and finally, moderating a wrap-up group discussion.

The following format and time frame may be used as a guideline for tailoring a 90-minute workshop to particular classroom/lab needs — for instance, for a 45-minute period, either by dividing the allotted time in half or by alternating 1 and 2 with 3 and 4 on successive days:

1. **Introduction:** 15 minutes (first workshop only).
Ask the learners to introduce themselves (if they are not yet acquainted); state the workshop goals, format and time frame; briefly introduce APL and microcomputer facilities; identify relevant resources.
2. **Hands-on Exploration:** 30 minutes (+15 after first workshop).
Distribute the handout on a selected topic; let learners explore, analyse and experiment with APL expressions using the computer.
3. **Minds-on Construction:** 30 minutes.
Encourage learners to construct programs to represent the concept or algorithm being studied; help with editing, modifying and saving programs on disk.

4. Discussion: 15 minutes.

Bring learners back together into a group and facilitate discussion of mathematical content they discovered plus any insights into their thinking; help organize thoughts; offer overviews, summaries and connections to other topics.

Topics

Suitable topics may be drawn from and stretch somewhat beyond the traditional mathematics curriculum in elementary algebra, number theory, probability, statistics and intellectual games. Sample topics are:

Averaging	Binary Search Game
Triangular Numbers	Recursion
Pascal's Triangle	Fibonacci Sequence
Pythagorean Theorem	Tower of Hanoi Puzzle
Divisors and Primes	Coin Toss Simulation
Function Tables	Polynomials
Propositional Logic	Root-Finding
Sets	
Sorting	
Mean, Median, Mode, Standard Deviation, Correlation.	

Methods and Materials

To begin with, the learner selects a topic and is given a one-page handout containing a sequence of APL expressions. The workshop leader proffers these only as *examples* of expressions containing primitive mathematical functions, "tools" to study and try out. The learner is encouraged to try other similar expressions in order to understand more fully how they work. The implicit goal is to *re-invent* a formal definition of these new tools by generalizing from enough examples. The explicit goal is to use these tools to *construct* a program as an object for understanding the algorithm illustrated (and related concepts).

A self-teaching APL book [1] is recommended for those learners who wish to study more about the programming language per se (later). Additional APL literature can be made available for perusal. Learners are invited to bring mathematics textbooks for reference and comparison of topic treatments.

Constructivist Approach and Cognitive Processes

At a minimum, the learner can enter and execute the given expressions by computer and observe the results. Further, the learner is encouraged to form mental *hypotheses* about the behaviour of each expression and to predict results before confirming them by computer. Formulating these and additional *self-directed* expressions provokes thinking, which often elicits the learner's own "What-if" questions. Ensuing surprises may lead to realizing *misconceptions*, or reconceptualizations, or other insights.

The interplay between the learner's previous knowledge of a topic (plus how to teach it) with new knowledge of how it is represented in APL (plus how it may be taught using APL) is expected to create some *disequilibrium*: while affording the opportunity to see the same topic from a different perspective, APL stimulates questions because it is somewhat different from conventional mathematical notation, which is fraught with ambiguities and inconsistencies (see [4]). Fortunately, however, more personal *experience* with APL can lead the learner to appreciate why it is attractive as a mathematical notation: APL is precise, unambiguous and internally logical (because it has to be executed by computer); it is unified across many areas of mathematics useful from elementary school through college and professional research and applications development; and it often yields shorter, more general expressions which may be easier to understand.

This approach honours *alternatives*. Learners often find different ways to express the same thing, that is, alternative algorithms which can be compared in terms of efficiency of time and space, or simplicity, generality, etc. By testing special cases and "edge conditions" in particular, learners can challenge themselves to *generalize* their expressions and embody them as robust programs.

Learners will take *responsibility* for their thinking more readily when they build something concrete and manipulable — like a program. They can save programs for later use, especially as building blocks to create larger programs ("modular programming"). In APL, each sub-program ("module") can be defined and tested separately before being combined (see [4]). Or, when given a "glass box" program — designed for the learner to examine — one can modify it to gain ownership and use it to explore related topics (see [3]).

Additional time for group discussion is recommended, of course, as a follow-up. Since not all learners are expected to discover ('get') the same understandings, convening in a group is important to share individual views — whether problematical or insightful — and to enable the workshop leader to add structure to the experience so they can see "the big picture". In sum, by *reflecting* on what has been discovered, by sharing actual programs built, by reviewing the cognitive impact of new outlooks toward a topic (plus pedagogical approach), and by openly valuing others' ideas, each learner's workshop efforts can be validated constructively.

References

- [1] Peelle, H.A. "APL: An Introduction" (Holt, Rinehart & Winston, 1986).
- [2] McIntyre, D. "Language as an Intellectual Tool: From Hieroglyphics to APL" (IBM Systems Journal, Vol.30, No.4, 1991).
- [3] Peelle, H.A. "Computer Glass Boxes: Teaching Children Concepts with APL" (Educational Technology, Vol.14, No.4, April 1974).
- [4] Peelle, H.A. "Introduction to APL for Learning Mathematics" (Journal of Computers in Mathematics and Science Teaching, Vol.9(1), 1989).

APL Product Guide

Compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL Visual Display Units and Printers
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 04393-385

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	AWL486	1,950	486 based 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour, (inc. 1 year on site maintenance.)
APL People	IBM PCs & compatibles	poa	Includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 18" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.3Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDDM and the full line of Interprocess APL2 enhancements.
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU, Std. configuration 2Mb RAM, 18 RS232 ports, 68 Mb hard disc, 720K diskette
Optima	IBM Compatible	poa	Complete PC-based station, APL interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	DYALOG APL DGS 386	poa	Dyadic's PC 386 APL Interpreter.
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	poa 1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.

Cockling/Drury	APL*PLUS PC Rel 10	410	STSC's full featured APL for IBM's and compatibles - Version 10 includes the Quad-NA facility to interface to non-APL software, support for Microsoft Windows and mouse devices. The User-command processor has been built in to the interpreter.
			Upgrades to version 10 are available from Version 9 and earlier releases.
	APL*PLUS PC Run-Time	175 for 5	Closed version of the Interpreter for developers, prevents user exposure to APL.
	APL*PLUS PC Developer System	950	Gives rights to distribute an unlimited number of copies of Run-Time application.
	APL*PLUS II System	1200	High powered APL Interpreter for the 80386 chip. Price includes one year's maintenance and free upgrades - volume discounts VERSION 4.0 NOW AVAILABLE - THE ONLY APL THAT LETS YOU BUILD WINDOWS & DOS APPLICATIONS FROM ONE PACKAGE.
	APL*PLUS II Developer System	3200	Unlimited distribution of APL*PLUS II Run-Time applications!
	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Aitos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces-I-APL/Mac			Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8 - 11	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	11	As above
	I-APL/Macintosh	13	As above
	Iverson Software Products		I-APL is the UK agent for ALL Iverson Software products, including: Sharp APL + enhancements to the ISI-APL standard Versions of J (and the associated texts) for PC, Mac, Arc, Atari. Please call for price list and order form. Please note there is a £3 packing charge on all orders.
IBM (APL Products)	APL Version 2 Release 1	poa	Full APL2 System for IBM 370 and 390. Product No. 5688-228. See your IBM Branch Office.
	APL2 Application Env't Ver2 Rel1	poa	Run-time Environment for APL2 Packages (IBM 370 and 390). Product No. 5688-229. See your IBM Branch Office.
	APL2 for the RISC System/6000	poa	Product No. 5765-012. See your IBM Branch Office.
	APL2 for the IBM PC (US version)	\$495	Product No. 5799-PGG. Part No. 6242936. PRPQ No. RJ8411 (in HONE, RJ0411). (from IBM Direct, order by Part Number; from your IBM Branch Office, order by PRPQ Number)
	APL2 for IBM PC (European vers)	poa	Product No. 5604-260. Part No. 38F1753
	TryAPL2	free	Free APL2 for educational/demonstration use. Write to APL Products, specifying diskette size desired.
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. From all IBM dealers, including MicroAPL.

Iverson Software Inc.	APL386	\$30	Sharp APL Release 20 for PC 386, 486 with graphics, and ability to operate under Windows.
	APLIPC	\$30	For PC
MicroAPL	APL.68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL.68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL.68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL.68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL.68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Optima	APL*PLUS/PC	369	
	APL*PLUS II	950	
	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
Reuters Ltd	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL VISUAL DISPLAY UNITS AND PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Quen-data & Qume etc	500	Daisy-wheel printers
Dyalic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
Dyalic	Printers (Various)	poa	Range of APL printers available.
General Software	Mellordata	poa	
Shandell	HDS Monitors	965-1395	A range of APL monitors. Please call for details.
	X-Terminals	1000-3500	View-stations for use with all X-Windows systems

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system. Stamp & Personal accounts (Inc. Run offs)
APL-385	APL-385 FSM-385 DRAW-385 DB-385 GEN-385	50(PC), 125(mf)	Including ... Screen development Screen design Relational W.S. Miscellaneous Utilities
The APL Group	Qualedj	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions. Includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet Incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRSII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; Incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling

	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking. fns of mathematical physics, menus, and others.
IAC/Human Interfaces	IAC/Graf	15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	APL*PLUS/Xbase Interface (PC Version)		\$98 Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support.
	APL*PLUS/Xbase Interface (II/386 Version)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support.
Interprocess Systems	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
	AOC	poa	APL2 Optimizing Compiler, translates APL2 functions to FORTRAN programs.
(mainframe)	AFM	\$6500-15300	High performance component and keyed file system (VS APL and APL2)
(PC)	AFM	\$175	Single user component and keyed files for APL2/PC.
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	CALL/AP	\$4700	For calling non-APL programs (VS APL and APL2)
	WSORG	poa	Full-screen Workspace Organizer for APL2.
Mercla	STATGRAPHICS 5	635	The old favourite.
	STATGRAPHICS PLUS	895	Now running under APL*PLUS II - so you can have as much workspace as you want!
	LOGOL 90	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 5	590	
Reuters Ltd	GLCS	poa	Global risk management for banks
	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	NEWSFLASH	poa	Real-time message exchange

	VIEWPOINT	poa	4GL with interfaces to DB2, ADABAS, and MVS datasets
UNWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	poa	component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT, FILEMANAGER(EMMA) STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF12250	Complete set of hierarchical menu utilities (site license)
	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550 FF22750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
(for APL*PLUS II)	UNIASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
	UNWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)
	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
	UNITAB II 2.0	FF6950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques...	\$59.95	488pp. book, (ISBN 0-9619087-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	Consultancy	poa	PC Based APL system design, programming and implementation.
Adlee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments

APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Camecho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	250-500	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
David Crossley	Consultancy	poa	Broad experience in many APL environments
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	APL*PLUS & Windows consultancy. Porting of software written in C into APL*PLUS.
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
Lingo Allegro USA	Consultancy	poa	General APL consultancy specializing in Prototyping, Migration, Mainframe to PC Downsizing, Performance Analysis, Troubleshooting, and Graphics.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics In APL
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training. PC and/or mainframe.
Unlware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0

Wicklife Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems
-------------------	-------------	-----	---

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — In English or French.
	The BBS/APL:		301-384-3672, 300/1200/2400/9600 b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr).
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	225pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)		
		16	185pp by Thomson
	J Dictionary	16	by Ken Iverson
	Programming In J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	An Introduction to J	8	47pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	18	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
IBM (APL Products)	APL2 Keycaps	poa	Keycaps for the PC and PS/2 (USA and UK standard). Product Number SX80-0270.
	APL2 Keyboard Stickers	poa	Product Number SC33-0604.
Reuters Ltd	MVSLINK	poa	Auxiliary processor complex, providing SharpAPL (Unix & MVS) with facilities to access MVS data, and invoke non-APL MVS software on remote MVS systems.
	SSQL	poa	Auxiliary processor providing SharpAPL (Unix & MVS) with high-performance SQL interface to DB2.
Iverson Software Inc. J		\$24	Documentation & shareware for many popular machines, including PC, MAC, SUN, & ARCHIMEDES.
	J Source	\$90	Documented source code in C for porting to other machines (Shareware).
	Programming In J	\$15	76pp.
	Tangible Math	\$12	34pp.
	Arithmetic	\$18	123pp.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	USA	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL France		Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
CPC UG APL SIG (Capital PCUG)	Washington, D.C.	Capital PC Monitor	Monthly meetings, occasional classes	free
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL		ACM, 1515 Broadway, New York, NY 10035 USA Tel: (212) 626-0611
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647; Fax: 0245-496646.
Adlee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
APL-385	Adrian Smith	Brook House, Gilling East, York, Tel: 04393-385
APLBUG	Lewis H. Robinson	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: (415) 928-2058
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	IBM Germany, Wilckensstrasse 1a, D-6900 Heidelberg, Germany. Tel: (49) 6221-404-243
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 09 79 63
BACUS	Joseph de Kerf	Roolenberg 72, B-2570 Duffel, Belgium.
Anthony Camacho		11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036.
Paul Chapman		41 Lams Conduit Street, London WC1N 3NG. Tel: 071-831-3762.
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-436 0524
CPC UG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-9372.
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX. Tel: 0604-760241
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: 90-68-08-87
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256-811125 Fax: 0256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-824741

General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx, UB5 4QS. Tel: 081-864-9537
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325
HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20914, USA.
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE16 8QL Tel: 0536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0388-527190. Email: clark.i@applelink.apple.com
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760036
	J C Business Services (for pre-paid orders only)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM (APL Products)	Nancy Wheeler	APL Products (M46V/D12), IBM Santa Teresa, PO Box 49023, 555 Bailey Avenue, San Jose CA 95161-9023, USA. Tel: 1-408-463-APL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulenln Lane, St. Petersburg 191186 Russia. Tel: 812-3111611 Fax: 812-3153321
Interprocess Systems Inc.	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131, USA Tel: (404) 992-8400. Fax: (404) 998-4869
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 071-265-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: (416) 925-6096 Fax: (416) 488-7559
Lingo Allegro USA Inc.	Steven J. Halasz	203 North LaSalle St., Suite 2100, Chicago IL 60601 USA Tel: (312) 558-1342 Fax: (312) 346-9603
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Hereage Street West, Aston Science Park, Birmingham B7 4AX. Tel: 021-359-5096. Fax: 021-359-0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866 Fax: 071-928 1005
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263943
M.T.I.C.	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252-874697
Optima Systems Ltd	Paul Grosvenor	Airport House, Purley Way, Croydon, Surrey CR0 0XY Tel: 081 781-1812 Fax: 081 781-1999
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
QB On-Line Systems	Phillip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-20789. Fax: 0276-683427. Mobile: 0831-307548
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-864-3078
Reuters Ltd	Laurie Howard	Reuter Nederland BV, PO Box 8230, 1005 AE Amsterdam, Netherlands Tel: +31 20 570 8733 Fax: +31 20 570 8758
The Rochester Group Inc.	Robert Pullman	50 S. Union St., Rochester NY 14607-1828, U.S.A. Tel: 716-454-4360. Fax: 716-454-5430
Shandall Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax: 02407-3118
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-668-0131
Sydney APLUG	Rob Hodgkinson	GPO Box 1425, Sydney, NSW 2001, Australia
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel: (1) 45-27-20-61. Fax: (1) 45-27-20-71. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	53 Shenpslt St, Vernon CT 06066, USA. Tel: (203) 872-7806

APL*PLUS II/386 Interface to Windows

by Ray Cannon

This review concentrates on the Windows interface supplied with Version 4 of Manugistics' (formerly STSC) APL*PLUS II/386. The main review of the product, by David Crossley, was published in Vector 9.1, July 1992.

Over the past few months I have been converting my program which displays the Mandelbrot set to run under Windows. The majority of the program is written in Borland's C++, but assembler has to be used for the dirty bits because even C is not fast enough! Programming Windows in C is tedious, and I offered to do this review as I wanted to see if APL would make Windows programming any easier.

The APL*PLUS II/386 Windows interface is designed so that the basic Windows functions can be called at the lowest level. Any of the calls available to a C programmer working with Microsoft's Software Development Kit (SDK) can be called from APL. The comparison between APL and C programming can therefore be made directly, as a workspace of Windows utilities can be compared with an equivalent library of C programs. Unfortunately, I encountered significant problems with the APL*PLUS interface, and in this instance I think C wins.

While this review was being written, Manugistics announced APL*PLUS II/386 Version 5. By the time this review is published Version 5 should be "available in the shops". Although I have not used Version 5, I have seen a demonstration of the Beta-test Version (4.9) given by Chris Lee of Manugistics. The Windows interface in Version 5 has been completely revised, and should rectify most of the problems I encountered with Version 4.

Hardware

I installed the APL on an Elonex 386DX 20MHz with 387 maths chip and 4Mb of memory running Windows 3.0 under DOS 5.0.

Let me state from the outset, 4Mb is not enough. The installation guide says that 4Mb is the minimum required, and recommends 6Mb or more. With 4Mb it worked, but only just. Until I got the installation set-up properly tuned, Windows was taking 8 to 9 minutes just to load APL, from clicking on the APL icon to getting the APL cursor prompt. Once tuned for APL this loading time

was cut to 2 minutes 45 seconds. Cocking and Drury, who very kindly supplied me with this review copy, recommend at least 8Mb, and in general the more memory the better. So this reviewer was lumbered with insufficient resources for the task in hand.

Installation

The installation procedure was atrocious and took hours. The installation manual clearly lays out the steps that you have to carry out, but everything has to be done manually and there is very little that is automated. You need a good text editor to modify many files such as APL .PIF files, APL .INI files, APL .BAT files, Windows system .INI file, and not forgetting CONFIG.SYS. Some of the files are quite large, and I found I could not use the *EDIT* facility in APL*PLUS because the workspace was not big enough.

WARNING: Be sure to keep a copy of your old files. I had a major problem after editing my Windows SYSTEM.INI file. I found that running Windows SETUP, to change to a different video device driver, crashed part way through. I was unable to run Windows or SETUP, so I had to rebuild SYSTEM.INI by hand to fix the problem. According to the report given at the September BAA meeting by Cocking & Drury, the installation in Version 5.0 has been substantially improved.

Other Software

To use the APL Windows interface to the full there is some additional software that may be necessary. Obviously, you have to have Windows 3.0 or 3.1. If you want to create icons, bitmaps, menus and other resources then you will need Microsoft's SDK or Borland's Resource Workshop. If you want to create Windows-style help facilities in your programs then yet more software will be needed.

Bookware

The documentation supplied for the Windows interface consists of a single slim volume of about 64 pages. This manual states that you will also require a copy of *Microsoft Windows Programmer's Reference* and a Windows programming guide such as *Programming Windows* by Charles Petzold. The Version 3 of the *Microsoft Windows Programmer's Reference* costs about £36, but the *Version 3.1 Reference* is in 4+ volumes at about £30 each. Petzold's programming guide costs £27.95 for Version 3.0 and about £39.95 for Version 3.1. I can also recommend *Programmer's Introduction to WINDOWS 3.1* by Meyers and Doner (Sybex £32.95).

How APL*PLUS II/386 Works with Windows

APL*PLUS II/386 V4 is a DOS program that can run under Windows 3.0 or 3.1, and is NOT a Windows program (unlike DIALOG APL/W reviewed by Duncan Pearson in Vector Vol.9 No.2). It does however come with a Windows program, called the *APL Agent*, via which it is able to issue Windows calls and receive Windows messages. It is important to understand how this interface works, for all but the most trivial of applications.

What actually happens is that once Windows is running, APL can be started in the usual way by selecting its icon in Program Manager. Initially Windows gives APL a DOS virtual 8086 machine, but then using a DOS extender APL boots itself into full 32-bit mode via the DOS Protected-Mode Interface (DPMI) feature of Windows. In this mode the APL and Windows environments are isolated from each other. Linking together these two environments is the job of the APL Agent, which is a normal Windows program written by Manugistics, and so has full access to Windows facilities. The Agent communicates with APL via shared memory at every timer interrupt, which occurs 18.2 times per second.

The APL Agent can issue calls to Windows functions listed in an initialisation file. It is driven by commands issued from the APL workspace and also by data from the initialisation file. Each workspace requires its own initialisation file, which has the same name as the workspace but with a .INI extension. If you want to add resources (bitmaps, icons, cursors, menus, accelerators, dialogue boxes etc.) then a workspace also needs its own Agent, and becomes a .EXE file. Agents are started automatically when APL is started.

The Agent can be made to support almost any function available via the Windows dynamic link library (DDL). That is to say, from APL, one can pass a request to the Agent to run Windows Function calls. Thus this interface is at the same level as that available to a C programmer. Each call the C programmer can make can also be made from APL.

In effect the .INI file is APL*PLUS's equivalent to WINDOWS.H. In C, WINDOWS.H is the header file which defines lots of macros, constants, and function declarations. It contains about 3480 lines of C code. It is a very important source of Windows programming information, even Manugistics' slim Windows Interface Manual makes multiple references to it. Windows 3.1 requires additional header files, for which this interface provides no support or mention.

Understanding C and Windows programming is pretty important. Chapter 4 of the APL*PLUS Interface Manual, *Understanding Windows Application Structure* states:

"This chapter assumes that you can read and understand C source code, and that you understand the concepts of programming for the Windows environment."

If you don't, read no further. APL*PLUS II V4 under Windows is not for you.

Manugistics supply APLWDEF.INI as the default .INI file. It is an ASCII text file containing definitions of the most common Windows functions; 123 functions out of a total of 550 in Windows 3.0, and a lot more in 3.1. If you are going to use any of the less common Windows functions, you are going to have to add their definitions to this file, or to the .INI file for your workspace. The information needed in the .INI file can be obtained from the *Windows Programming Reference*, or from WINDOWS.H, but it will need translating in accordance with the instructions in Chapter 4 of the Interface Manual.

For those of you with Borland C, I found the easiest way of editing the .INI file was from Borland's IDE editor which has on-line help available for every single Windows function. I could cut and paste information from the on-line help text directly into the .INI file before modifying it.

So far, I have found this the biggest hurdle to using APL*PLUS under Windows. You need to check if every Windows function call you are going to make is included in the .INI file *before* you use it. Why Manugistics has not supplied a complete .INI file with every documented Windows function included I don't know. I can only guess that there must be a limit on the size of this file that the system can handle. In fact two default .INI files are really needed, one for Windows 3.0 and one for 3.1. This is a fundamental part of the APL/Windows interface, and as supplied it is incomplete.

The .INI file is bound with the workspace and should contain all the calls required by the workspace. The default file, APLWDEF.INI, is searched at run time for any calls not found in the workspace's .INI file. It is possible to edit the default .INI file during a Windows APL session, but as it is a large file you have to have plenty of Windows memory available.

As well as passing function calls from APL to Windows, the Agent also passes messages from Windows to APL. However, Windows generates hundreds of messages, most of which an APL program is not concerned with. For example, a message can be produced each time the mouse cursor is moved from one pixel to

the next, so moving the mouse from the centre of the screen to a menu box could produce as many as 300 messages, and that ignores the messages produced each time the mouse crosses a Windows boundary. But the APL Agent can only pass these messages to APL at every clock tick, at the rate of 18.2 per second. Fortunately Windows has a method of gathering messages together, so you don't lose any.

Rather than pass all messages across to APL, you can program filters in the Agent, and only pass those messages that you know you are interested in. The interface manual has 7 pages on this subject, including over a page of C source code. It's complicated stuff, to quote again the manual:

"Specifying complicated filtering logic improves application performance and documents the program structure effectively."

The following APL code is an example of this "effective documentation" copied straight from the manual:

```

WM_COMMAND'                W_FP 'WP'
                             W_FP 'BEGIN'
201      203      W_FP 'BEGIN'
              'EN_SETFOCUS' W_FP 'My_In'
              'EN_KILLFOCUS' W_FP 'My_Out'
              '*'          W_FP '*'
              W_FP 'END'
301      W_FP 'BEGIN'
              'CBS_SELECT' W_FP 'My_Sel'
              '*'          W_FP '*'
              W_FP 'END'
              W_FP 'My_WmCommand'
              W_FP 'END'
WM_PAINT'                W_FP 'My_WmPaint'
W_CreateFilter w_fp

```

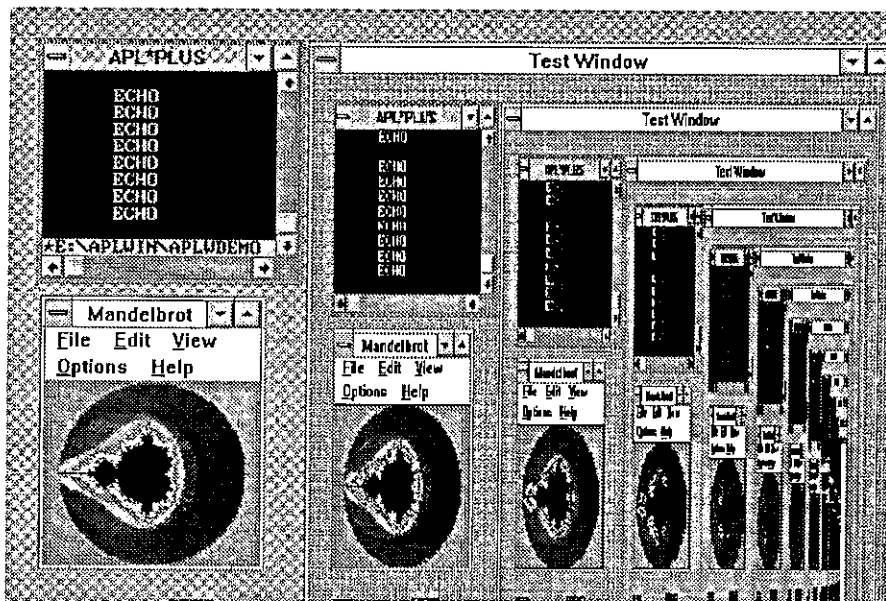
To be fair, this APL code is laid out to look like the equivalent C code it would be replacing.

Using the Interface

Once I had managed to install APL and the Windows interface and run the demo, I wanted to try out programming the interface myself.

I thought it might be easy to copy an image of the entire screen into the test Window produced by the Demo workspace, using the Windows function `StretchBlt`. If run several times, this function produces a "visual echo". The Front cover of Pink Floyd's classic LP, UMMAGUMMA, is a good example. The

following was produced by copying the screen to the Clipboard, pasting it into Paintbrush, and saving it as a bitmap:



The main Windows function used, `StretchBlt`, is (partially) defined as:

```
BOOL StretchBlt(hDestDC, X, Y, nWidth, nHeight, hSrcDC, XSrc,
YSrc, nSrcWidth, nSrcHeight, dwRop)
```

This function moves an image from a source rectangle into a destination rectangle, stretching or compressing the image if necessary to fit the dimensions of the destination rectangle. The `StretchBlt` function uses the stretching mode of the destination device context, set by the `SetStretchBltMode` function, to determine how to stretch or compress the image.

Using `StretchBlt`, the task is a simple one. *ECHO* has only 6 real lines of code:

```

V ECHO
[1]  A ECHO the Windows screen in the TEST window
[2]  A Get a device context (drawing surface) of the whole screen
[3]  HDCSCREEN←W_Call 'GetDC' 0
[4]  A Get size of the TEST window
[5]  RECT←C2I W_Call 'GetClientRect' hWnd ''
[6]  A Get a Device Context for the TEST window
[7]  HDC←W_Call 'GetDC' hWnd
[8]  A Copy (and squeeze) the whole screen image into the TEST window
[9]  BIN←W_Call 'StretchBlt' HDC 0 0 (RECT[3])(RECT[4])
      HDCSCREEN 0 0 640 480 'SRCCOPY'
[10] A Release the DCs
[11] BIN←W_Call 'ReleaseDC' hWnd HDC
[12] BIN←W_Call 'ReleaseDC' 0 HDCSCREEN
V

```

It took me over 5 hours to work out what those lines actually were. Luckily I had the source code of a similar C program to help me.

Why did it take so long?

Well first I did not know what I was doing. I've written C Windows programs before, and I have written APL programs before, but I have not written APL*PLUS Windows programs before. Also, the Windows function `StretchBlt` is not one of those included in `APLDEMO.INI` or `APLWDEF.INI`, so that it had to be added.

Next, I kept getting UAEs (Unrecoverable Application Errors). On getting a UAE, the Agent is terminated leaving the APL window stranded. Clicking on the APL icon showed the Close option greyed out, indicating that it was not valid. The only way I found I could remove this window, was to select **Settings** from its system menu and run the Terminate option, to which Windows suggested that I must then close down the rest of Windows and reboot the machine.

Originally, I tried to get the Device Context (DC) for the whole screen via the Windows function `CreateDC` as did my sample C program, and as Petzold's *Programming Windows* recommends. However, I was unable to call this function successfully from APL. In C it would be called via the code:

```
hDC>CreateDC("DISPLAY",0,0,0);
```

`CreateDC` requires as its first argument a *Long pointer to a string*; the string being the text "DISPLAY". Remembering that, in C, text between double quotes

produces an array of type `Character` with a terminating `Null`. When an array is used as an argument to a function, only its address (a pointer) is actually passed.

Another quote from the Interface manual:

"If members of a data structure contain pointers to other data structures, you must build the pointers. To do this, you allocate Windows memory, copy data into it, and store the pointers into the appropriate location in the character representation of the data structure."

Well as stated above, for my needs, `CreateDC` requires a pointer to the structure, a character array, containing the text "DISPLAY". So we need to allocate some space, get its address, fill it with the text "DISPLAY" before we can pass its address to `CreateDC`. All of which the C programmer does simply by typing "DISPLAY" as the first argument to `CreateDC`.

The manual contains an example of code to do this, where it shows how to call `MAKELP`. I used this code as a basis for my function `DISPLAY`:

```

V R+DISPLAY;text;sel;lpData
[1]  A Returns a Long Pointer to the text DISPLAY in WINDOWS memory
[2]
[3]  A Define TEXT string, with trailing NUL.
[4]    text+'DISPLAY',0TCNUL
[5]
[6]  A Allocate enough WINDOWS memory to hold this text.
[7]  A      Use WINDOWS function to GlobalAlloc.
[8]    sel+W_Call 'GlobalAlloc' 'GPTR'((10)ppertext)
[9]  A      GPTR defines GMEM_FIXED and GMEM_ZEROINIT memory.
[10]
[11] A Convert the selector into a long pointer. Use with caution.
[12] lpData+MAKELP sel 0
[13]
[14] A Write the text to memory.
[15] text W_Mem lpData
[16]
[17] A Return the pointer to the text DISPLAY
[18] R+lpData
V

```

However, every time I try to run the example, or my function `DISPLAY`, I get a UAE when it tries to write the text "DISPLAY" via the function `W_Mem` into the space allocated by `GlobalAlloc`. With the help of Dave Phillips of Cocking & Drury, it was found that this bug only occurred under Windows 3.0, running the same code under 3.1 works OK.

Another small problem was that the example in the manual for *MAKELP* has a typographical error. The string had been terminated with `TCNL` rather than `TCNUL`. Strings in C have to be terminated with a `NULL`.

So because I cannot pass `CreatedC` a long pointer to the string "DISPLAY" I cannot create a DC to the entire screen via `CreatedC`. In *ECHO*, I finally got round this problem by using the function `GetDC` with a `NULL` parameter. This is an undocumented feature of `GetDC`, normally, you need to pass it a handle to the window for which you wish to obtain the device context. Passing it a `NULL` causes it to return the handle to the entire screen. This is also very much easier to code in *APL*PLUS*.

Linking *ECHO* up as a callback function, so Windows calls it when certain events occur, such as a request to repaint the window, can have some quite interesting effects.

What the Manual Says

If you feel that I am being a bit negative about this interface, here are a few quotes from the supplied manuals:

*"You can also run APL*PLUS II/386 as a windowed application, but performance may be too slow to be satisfactory in most applications."* Installation Guide page 3-23.

*"Sometimes APL*PLUS II/386 takes an unusually long time to perform certain operations when running under Windows 3."* Installation Guide page 3-31.

"Note: This function is slow already; tracing slows it down even more." Windows Interface page 4-8. This is referring to *DemoHello1*, the "Hello World" program!

"The primary reason that this program is slow is the interface used to communicate between APL and the Agent." Windows Interface 4-11.

"Caution: Do not run this application [DemoHello1] too many times. A Windows bug causes a UAE if a window class is registered.... more than about 7 times." Windows Interface 4-11.

"The Windows GetInstanceData function does not accept a far pointer. NEVER use this function. It can crash the Agent or corrupt data." Windows Interface 6-11.

"The fact that callbacks can occur asynchronously.... can cause normal APL name scoping to get tangled up." Windows Interface Appendix A-2.

Limitations of the Interface

The interface is a very clever way of getting a DOS application interacting with Windows. With enough APL/Windows knowledge, other software resources, and plenty of computer memory, it can be used to produce Windows applications.

However, just as APL programmers want large workspaces, Windows wants as much memory as possible. Because APL*PLUS is a DOS application, any memory it uses is not available to Windows. Effectively, the computer's memory is split between Windows and APL. On my 4Mb machine 2Mb is available to each. If APL was running as a Windows application, the full memory would be available to Windows (4Mb in my case) to use how it wants to, including giving it all, on demand, to a single application, for example APL. But with APL running as a DOS application, this is not the case.

Another problem with the interface is that Windows functions often require parameters which are pointers to data. These pointers must point to data within the Windows memory area. They cannot point to data in the APL workspace. Hence the need to tell the Agent to allocate memory and fill it from APL. Thus parameters are often built twice, once in the APL workspace, and then again under Windows by the Agent.

So with this interface, you have half the memory available and you must double up on the data. OK, not all APL's data needs to be duplicated. For example, if you wanted to create a large bitmap such as a screen image in APL, it would have to be duplicated in Windows. The shame of it is that APL would be the ideal language, and far better than C, to manipulate a bitmap.

What Can it be Used For?

I have been very negative about this interface up to now. However, given that it is a DOS product, running under Windows, with the ability to interactively "program" Windows in C, it is quite amazing that it works at all.

If you have a Manugistics APL II application, which you would like to run under Windows, with a Windows interface, and the interface is only to be used to pass control information, as against bulk data, then Version 4 might be just what you want. This interface can create windows, buttons, menus etc. and pass information around relating to them. When it comes to dealing with bitmaps however, things start to get very tricky. Even so, remember that Dyalog APL/W original version did not support any graphics of this sort, while APL*PLUS II 386 does.

Conclusions

As a professional APL programmer, I would not attempt to use APL*PLUS II/386 Version 4 under MS Windows for any task. If I have to write a Windows program and am given the choice between Borland C++ (I have not tried Microsoft C) and this version of APL, I will choose C++, because it is easier, better documented by both supplier and third parties, faster, cheaper, requires less Windows memory, and is far more robust.

As a tool for an APL programmer to explore the wonders (or pitfalls) of Windows programming, it has some potential, but only if you don't mind learning quite a bit of C on the way and have a fast machine with 8Mb or more of memory.

Not all is gloom however. I attended a demonstration of Version 5.0's Windows development interface by Chris Lee of Manugistics Inc. at Cocking and Drury in October 92. Version 5.0 should be available before the end of 1992.

The installation procedure has been improved. Chris Lee said that it took him half an hour instead of the 4 hours for Version 4.0. The Windows development interface requires Microsoft Windows Version 3.1, it will not run under Version 3.0. The recommended minimum memory is 8Mb of RAM. The interaction between the APL Agent and the DOS APL system is no longer via shared memory and clock interrupts. Now, using the Windows 3.1 virtual device driver, the Agent treats the DOS APL as a virtual device. This radically changes (for the better) the relationship between APL and Windows.

In Version 4.0 the default initialisation file APLWDEF.INI is a plain ASCII file which has to be interpreted every time it is used. In Version 5.0 it has been replaced by a "compiled" binary file, which has all the MS Windows 3.1 calls built in. Version 5.0 now includes, in Manugistics' words, *APLGUI – an Easy-to-Use Windows Development Toolkit*.

Final Thoughts

The Version 4.0 Windows interface is a working model, but impractical in practice. Version 5.0 looks like the real thing, for real programmers with real computers.

Do you remember the quote about Real Programmers and APL: "*Real Programmers don't use APL, anyone can be obscure in APL*"? I think the Microsoft programmers who wrote Windows were "Real programmers".

RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Meeting at Morgan Stanley 23 October 1992

notes by Jonathan Barman

I was quite surprised to find that the lecture room for our meeting was attached to, and part of, the main dealing room at Morgan Stanley. Dealing rooms are not normally exposed to strangers, and allowing us to be close to the action seemed a good omen for the content of the meeting. I was not to be disappointed.

Smalltalk

The meeting started with a demonstration of a dealing system written in Smalltalk. Paul Wood and Clive Jackson showed off a really slick Windows system. It was hooked up to receive prices from the Stock Exchange, so that as soon as a price changed the appropriate cell on the screen briefly flashed, and there was a running ticker tape of the deals that had taken place. In true Smalltalk/Windows style, the system was event driven, responding to input from dealers and the Stock Exchange. There were masses of features to help the dealers make the best decisions, charts of historical data, forecasting tools etc. etc. When a trade has been made the details are sent to the Stock Exchange. The SE computer records the deal and then broadcasts it to all the market makers. The round trip, Morgan Stanley to SE and back, takes two seconds, a little statistic I found quite impressive. Another impressive statistic was that the system had been created by two people working on it from February this year. Smalltalk must be taken as a serious contender for application development.

A

Although the Smalltalk demonstration was impressive, the star attraction of the afternoon was, for me, a demonstration of *A* by Michael McConigley and Jon Willis. *A* is a dialect of APL written by Arthur Witney, and it is specially tailored for speed, speed and more speed. Huge quantities of data can be handled and applications can be developed really quickly.

The similarities between *A* and J are quite striking. John Brock told us about the genesis of J, when Arthur Witney spent an afternoon with Ken Iverson and created the first interpreter fragment for J. In *An Implementation of J*, (Iverson Software 1992), Roger Hui gives details of that famous meeting, in Appendix A entitled *Incunabulum*. (Did Roger deliberately choose Appendix A for Arthur and

A?) The Appendix contains the C code of the interpreter fragment. The style of coding is very compact and concise, and looks like APL.

Like J, *A* has $\square 10$ firmly fixed at 0. Operators such as reduction work along the first dimension by default, so that $+/M$ in *A* is the same as $+/M$ in APL. Nested arrays are grounded, so that scalars can be boxed. The rank operator looks identical to J or Sharp APL.

All functions are defined and edited in direct definition format, and $\rightarrow$ is not available for branching. Multiple line functions are possible, but are not so easy to enter and edit. This seems a very sensible restriction which would force people away from writing 1000 line functions with hundreds of labels. Michael gave us a glimpse of the functions used in a windowing utility, and all seemed to be one line functions doing just one task each. Also, all the variables in a function are local by default, and you have to take explicit action to access global variables. Maintenance must be much easier in this environment.

Functions can be assigned to identifiers, as in J and Dyalog APL, so that you can say $SUM \leftarrow +/$.

There is a new data type of Name, which is indicated with a back quote in front of the name, for example $a \leftarrow 'red' 'white' 'blue'$. All APLs have to process names when parsing code and maintaining the Symbol Table, and a good APL will have efficient facilities as part of the interpreter. *A* lets you get at these facilities. An array of names behaves in the same way as an array of characters, for example $a = 'white'$ would return $0 \ 1 \ 0$. There is a super-fast execute function (%) for name-type variables, so arrays of function and variable names become a practical proposition.

A uses most of the APL symbols, but there are several exceptions, such as # being used instead of $\square$ for the index function, and \$ instead of) for system functions, so you \$off rather than)OFF. The major difference from normal APLs and J is that there are some reserved words in *A*, *if else* and *time*. The keyword *time* has caused some problems. It was added because everyone is hooked on speed, and a good timer is essential. However, everyone also wants to know the time of day! It just shows how keywords can make life difficult.

Nested arrays are specified using a semicolon as a separator, for example $1; 'five'; 'name'$. This means that passing multiple arguments into a function looks exactly like $\square FMT$ in APL*PLUS, Sharp APL etc., so you could have $FUN(a; b; c; d)$.

I got quite excited when I saw the Context switching facilities. This is like $\square NA$, in that you can run $\mathcal{A}$ functions and get at $\mathcal{A}$ data in other Contexts. Contexts can also contain C programs, so $\mathcal{A}$ can call C programs. As in APL2, C programs can also call $\mathcal{A}$. The syntax for getting at functions and data in other Contexts is rather neat, with a dot embedded in the name, and is very similar to the suggestions made by Robert Frey in *Object Oriented Extensions to APL and J*, in Vector 9.2. I wish this feature could be included in standard APLs. It would completely remove the problems almost everyone has with naming utilities so that they do not clash with names used in an application.

More excitements came up with the memory mapped files. There is no filing system as such, I-beam is used to link a file to a name and the entire file appears as an enormous vector in workspace. So, `'filename' I 1 data` links the filename to the 'variable' `data`, and expressions such as `(5678#data)+'A'` will change the data directly in the file. Note the selective specification, as in APL2.

$\mathcal{A}+$

John Brock told us that they had had lots of problems in programming Windows applications using Motif. As $\mathcal{A}$ can call Motif C functions directly there was no problem with the interface, the problem lay with Motif itself, which was slow and used lots of memory.

Incredibly, Morgan Stanley wrote a replacement for Motif in 3 months, which can be directly accessed from $\mathcal{A}$. $\mathcal{A}$ now becomes $\mathcal{A}+$ (or was it $\mathcal{R}$?). There is only one copy in memory of the data on screen, and nothing has to be swapped backwards and forwards between the Windows and $\mathcal{A}+$. We were given a staggering demonstration of a window displaying a huge matrix, I cannot remember now if it was 10 million or 1000 million integers. Scrolling around this monster and popping in changes was just as quick as looking at a tiny matrix. Inadvertently Michael changed every element of the array which stopped everything for about a minute, indicating the sheer size of the beast.

Window systems are developed with a simple-looking scheme using just 6 functions in the workspace, although there are many more hidden away in another Context. Straightforward instructions such as `show 'm is 'array` display `m` as an array in a window on screen. There is a similar simple syntax for buttons, radio buttons etc. etc. Callback functions were neatly handled using yet more special features of $\mathcal{A}+$.

Most of the features for handling windows are written in $\mathcal{A}+$. An impressive example was the ability to group a set of objects together in their relative positions on screen. You just move the objects around until they are vaguely in the right place and then *group* snaps them all together.

During the presentation some staggering statistics were thrown around. The Sparc II machines are equipped with 400 Mb of memory! Apparently some \$20 million has been spent in the development of $\mathcal{A}$ and $\mathcal{A}+$. It is quite definitely not for sale as it is seen as a tool giving Morgan Stanley the competitive edge. Development times are estimated to be 3 times better than C, and maintenance is 7 times better. It is also reckoned to be faster than C, as Arthur has spent so much time optimising every primitive; $\mathcal{A}$ is the fastest sort in the world!

It was fascinating to see that the developers had their desks actually in the dealing room, so that dealers could get problems solved immediately, and at any time. John Brock's desk, car and home telephone numbers are on every dealer's keyboard, so he is never off duty.

Normal APL

Having been shell-shocked by $\mathcal{A}$, Shumit Rehman's demonstration of the system for brokers dealing in cash bonds seemed a bit tame. It is written in APL*PLUS, and is in fact an advanced system full of features. The system was written some 3 years ago when Windows environments were not available, and it was interesting to compare the look and feel of the system with the Smalltalk system demonstrated earlier. Many of the facilities were the same, but the way one used them was quite different. I am not sure which is best. Windows are the current vogue, but the old ways can be just as fast and efficient, as Shumit demonstrated.

Conclusion

When starting to write up this meeting I chanced to open an old copy of Vector, 6.4 April 1990, where there was a note by Ed Shaw on a meeting of the NY/SIGAPL group at Morgan Stanley in New York. His reactions were just the same as mine — astonishment at the amounts of data, memory, workspace sizes and speed!

Optimising your APL

by David Eastwood

When I am attempting to teach APL I usually place program efficiency quite low in my choice of priorities, behind such considerations as legibility, portability and reliability. There does come a time, however, when your APL system simply takes too long to run and you need to do something about it.

Making APL run faster can be addressed in a number of ways, some of which are quite drastic. You might:

- buy a better computer
- speed up your APL code
- get a new programmer
- use something other than APL

... or some combination of the above. For owners of microcomputers, the option to replace hardware is not as flippant as it may first appear given the relative costs of hardware and programmer time, but it may not offer much to the mainframe user. The 'why did you choose APL in the first place?' debate is entertaining and indeed I would like to explore on some other occasion the topic of the sort of problem for which APL is not the right choice (yes, there are some!). What I will be doing here is to discuss some of the means to make your APL code run faster.

The options available include:

- better data design
- changes to APL code
- introduction of non-APL code via Auxiliary Processors or APL compilers

I shall confine myself here to considerations of APL code design, touching briefly on the use of non-APL routines. Often, of course, simple observation of a system will indicate that there are one or two routines that are the major bottlenecks. Attention should be focussed on these areas rather than protracted efforts to make everything run faster.

These notes are drawn from a number of sources (see References), almost all of which, unfortunately, are either out of print or not generally available, so this

presentation is offered as a means to reissue some of this earlier material, together with some of my own additions.

One of the great joys of APL is that the programmer does not have to worry greatly about the well-being of his computer. You can choose an algorithm and let the computer worry about the type of data being used, how much storage space to allocate and so on. Inevitably, when considering the speed at which things happen you do have to think about the internal workings of APL but I shall be remaining firmly in the realm of observations that are reasonably universal.

Bear in mind that an APL interpreter can be considered as a large collection of routines for data manipulation and so it should be obvious that there will be some that are more efficient than others. In addition interpreter writers add special algorithms for special cases, for example $\wedge . =$ for string search might be a candidate for a special routine. Under these circumstances you will always find performance quirks. It is not really a good idea to code to suit interpreter quirks since the interpreter writers are usually on the lookout for sections of code that can be improved in the next release (especially anything that might be seen as a benchmark).

Any timings shown will be shown for 4 interpreters — APL*PLUS/PC (v9), APL*PLUS II (v4), APL2/PC (v1.02) and APL.68000 Level II (v2). I am using the co-processor version in each case. Timings are in milliseconds and the PC versions run on the same 20Mhz 386-based machine. The APL.68000 ran on an Apple Mac II which is of roughly the same performance.

APL Internals

As far as I am aware, all APLs store data in roughly the same way. As a user you will be accustomed to distinguishing between character and numeric data (I hope) but an APL interpreter will further distinguish between a number of types of numeric data which usually include:

- Boolean
- Integer
- Real

Some APLs will introduce further data types such as complex numbers. Different amounts of storage space will be allocated to these different types of data, for example 1 bit per element for Boolean data or 64 bits per element for Real data. The APL interpreter will change the representation of the data as required. In

general, an interpreter will not check data for an optimal representation at every possible point — it would put an intolerable overhead on performance. Instead a number of rules are often followed, for example:

- `↑` or `↓` must generate integers
- `÷` must generate reals
- `=` must generate booleans
- numbers above a certain size must become real

There are a couple of implications here.

I have, for example, seen a number of ways to store the date as a single number:

199210 or 1992.10 for November 1992

You might have chosen one or other of these representations because of the way you wished to split the number back into Year and Month but you should bear in mind that an integer representation of data (199210) will usually take up half the space of a real number version (1992.10), and in some cases even less.

Nearly all APLs will store boolean data as 1 bit per element and character data as 8 bits per element and choices of data representation under these circumstances can be even more important. The exception here is APL*PLUS/PC which regards boolean data as 16-bit integer. Thus:

`LIVE_ACCOUNTS ← 'YNYNNNNNYY'`

will usually take up 8 times as much space as

`LIVE_ACCOUNTS ← 1 0 1 0 0 0 0 0 1 1`

without significantly improving the legibility of the code since most APLers are quite happy with the convention that 1 means 'yes'.

In terms of speed of execution, your choice of data representation can have an impact. Arithmetic with real numbers will generally be slower than integer arithmetic on two grounds:

- Increased complexity of operation
- Comparison Tolerance (the measure of 'exactness' used for comparison operations).

Consider these two operations:

```
A←1000?1000
B←A÷100
```

```
A←A,1001
B←B,10.1
```

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
R←1001←A	116	110	880	380
R←10.1←B	7667	1430	2310	2030

Note that I have been careful to put the 'target' at the end of my search object.

Moving on from consideration of the amount of space taken up by data to the way in which it is stored, you will find that APL interpreters generally store variables in the same way in the workspace. The interpreter maintains a Symbol Table which points directly or indirectly to an area of memory where the actual data that makes up a variable is held. If you could examine the makeup of a variable you would see something like this:

HEADER	TYPE	RANK .	SHAPE	DATA
--------	------	--------	-------	------

and the data is usually held in row by row order (in other words the order in which you would see the data if you used the ravel function).

Thus a matrix:

```
MAT←2 3p19
```

would look like

```
<HEADER BYTES> <CODE FOR NUMERIC> <2> <2 3> <1 2 3 4 5 6 7 8 9>
```

You would expect that APL operations that merely adjust the shape of data rather than the content (ravel, reshape etc.) should be very fast indeed. Whenever the content of a variable is changed APL will have to do a greater or lesser amount of work in terms of moving or rearranging this data around within the workspace, and this leads conveniently into the next topic.

Giving the Interpreter an Easy Time

It would seem reasonable to predict that the less you make the APL interpreter do the quicker the job will be done. Turning this prediction into reality can sometimes be difficult. Is it quicker to use the more complicated APL primitives? Is it quicker to use one-liners or loops?

I shall run through a number of examples.

(i) Joining data.

Given the way in which APL stores data internally you should be able to predict that it is significantly quicker to join data along the first dimension rather than the last. Thus when APL is asked to append a row to a matrix, the new data is just joined to the end of some existing data and copied within memory whereas appending a column to a matrix will require the merging of two sets of data as it is copied.

Thus for

```
A←100 100p100
B←100?100
```

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
R←A,[1]B	617	990	1710	490
R←A,[2]B	883	7580	2310	600

Bear this in mind when you are designing your data structures.

(ii) Numbers of operations.

A simple consideration of the number of operations carried out by APL should also enable you to find some extra performance. First consider an example which adds up a number of invoices and adds VAT:

```
VAT←1.175
INVOICES←0.01×1000?1000
```

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
R←VAT*+/INVOICES	500	490	270	930
R←+/VAT*INVOICES	1250	4670	660	1590
R←VAT+.×INVOICES	8600	9550	2800	1760

In the first example we have 1 multiplication and 1000 additions, in the second we have 1000 additions and 1000 multiplications. Using a fancy operation doesn't reduce the amount of work the interpreter has to carry out, in fact the extra power of inner product is 'wasted' on this example.

Often the theory that 'parentheses are bad' will lead APLers into producing code that is slower than it need be. Here are a couple of ways of computing an average, the obvious way and the 'look, no parentheses' way:

$A \leftarrow 1000 ? 1000$

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
$R \leftarrow (+/A) \div \rho A$	316	660	270	550
$R \leftarrow +/A \div \rho A$	1680	5770	1480	2850

The next example is less obvious but illustrates the penalty that can be attached to type conversion (as well as the performance implications of real versus integer arithmetic):

$A \leftarrow 1000 ? 1000$
 $B \leftarrow 2.5$

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
$R \leftarrow B++/A$	300	610	220	490
$R \leftarrow +/A, B$	1366	5050	610	1260

A final example on this particular topic shows two alternative ways to generate a vector of numbers from A to B:

$A \leftarrow 1000$
 $B \leftarrow 10000$

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
$R \leftarrow A + \iota B$	1300	2420	3960	1920
$R \leftarrow A + \iota B - A$	1250	11090	60	2630

The first operation requires a simple drop after the ι whereas the second will require a whole series of additions. You would expect the ι operation to be quite quick and the drop does not require a vast amount of internal data manipulation. With the numbers chosen above, the second expression requires one subtraction, one ι operation and then 900 additions. The first expression requires one addition, one ι operation and then one $\downarrow$ operation.

For the size of data chosen here, there are no immediate conclusions except that there may be something a bit odd going on with APL*PLUS/PC. The results for APL2/PC illustrate nicely the importance of not reading this sort of note too uncritically. APL2/PC has a special data type called an 'Arithmetic Progression Vector' which is held as just a few bytes. The sort of operation shown above is ideal for this type of data structure.

(iii) Ways to select data.

There are a number of ways to select data, and all APLs will offer:

- Reshape
- Take/Drop
- Compression
- Indexing ([] or [])

The functions are listed in increasing order of sophistication. Reshape will only offer data starting from the beginning. Take and drop will select data from the beginning or end of a variable but can only offer a 'chunk' of the data in an unsorted form. Compress will select data from points throughout the variable but cannot vary their relative positions. Indexing can select data from anywhere in a variable and present it in any order.

A←100 100p10000710000				
B←1 1,98p0 C←12 E←1100				
(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
R←2 100+A	84	110	60	160
R←~98 0+A	84	110	50	160
R←B/[1]A	84	7530	160	220
R←A[C;E]	450	330	110	250

Again a couple of anomalies, but on the whole the more complex operation is slower. A sensible conclusion to draw from the results above is to use the simplest function that satisfies your requirements.

Further Tricks and Tips

None of the examples discussed so far has been other than fairly legitimate APL code but many optimisation techniques do begin to stray into the realms of somewhat dubious style. I shall offer a few borderline examples here.

(i) Predeclare data

Although one of the strengths of APL is the way in which you can continuously change the shape, size and type of data you can pay a performance penalty for this power.

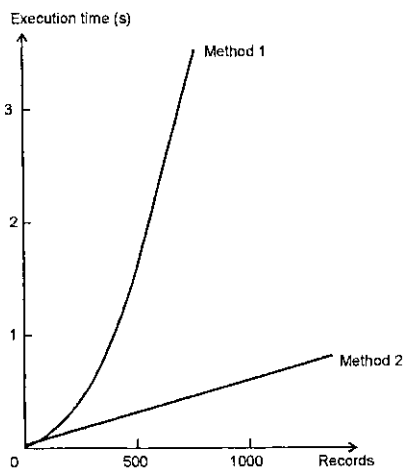
Very often you know the shape and size of a result before you start. You might, for example, be reading a known number of records from a file. Consider the difference between setting up an empty data variable before you start and performing continuous appends.

This example simulates reading 1000 records into a variable:

<pre> ▽ APPEND; I; L [1] R←0 20ρ' ' [2] I←1 [3] L←(1000ρLN), LX [4] LN:R←R, [1] 20ρ' DUMMY DATA' [5] →L[I←I+1] [6] LX: ▽ </pre>	<pre> ▽ INDEXΔIN; I; L [1] R←1000 20ρ' ' [2] I←1 [3] L←(1000ρLN), LX [4] LN:R[I;]←20ρ' DUMMY DATA' [5] →L[I←I+1] [6] LX: ▽ </pre>
---	---

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
APPEND	5816	13730	5940	8570
INDEXΔIN	3400	4230	3840	2080

You should note that in the example above the differential between the two methods does not remain constant — the time taken for progressively larger catenation operations is not linear as this graph from Smith [1] shows:



(ii) Style of loops

Variations of looping style can affect the speed of execution. Consider these four functions that do the same job with marginal differences in looping style:

<pre> ∇ <i>LOOPS1</i>;Z;I;A [1] <i>R</i> <i>METHODS OF LOOPING</i> [2] <i>A</i>+?100 4p1000 [3] <i>I</i>+1 [4] <i>LNEXT</i>:Z++/A[I;] [5] →((1+pA)≥I+I+1)/<i>LNEXT</i> ∇ </pre>	<pre> ∇ <i>LOOPS1A</i>;Z;I;A;<i>ROWS</i> [1] <i>R</i> <i>METHODS OF LOOPING</i> [2] <i>ROWS</i>+1+pA+?100 4p1000 [3] <i>I</i>+1 [4] <i>LNEXT</i>:Z++/A[I;] [5] →((<i>ROWS</i>)≥I+I+1)/<i>LNEXT</i> ∇ </pre>
<pre> ∇ <i>LOOPS2</i>;Z;I;A;<i>L</i> [1] <i>R</i> <i>METHODS OF LOOPING</i> [2] <i>A</i>+?100 4p1000 [3] <i>I</i>+1 [4] <i>L</i>+((1+pA)p<i>LNEXT</i>),<i>LX</i> [5] <i>LNEXT</i>:Z++/A[I;] [6] →L[I+I+1] [7] <i>LX</i>: ∇ </pre>	<pre> ∇ <i>LOOPS3</i>;Z;I;A;<i>L</i> [1] <i>R</i> <i>METHODS OF LOOPING</i> [2] <i>A</i>+?100 4p1000 [3] <i>I</i>+1 [4] <i>L</i>+((1+pA)p<i>LNEXT</i>),<i>LX</i> [5] <i>LNEXT</i>:Z++/A[I;] [6] <i>I</i>+I+1 [7] →L+1+L [8] <i>LX</i>: ∇ </pre>

(1 Execution)	APL.68000	APL*PLUS/PC	APL2/PC
<i>LOOPS1</i>	467	550	610
<i>LOOPS1A</i>	383	440	500
<i>LOOPS2</i>	316	390	380
<i>LOOPS3</i>	370	440	490

LOOPS1 is probably the way most people would write the code, but note the speed improvement that comes from carrying out the test for the number of rows in our matrix just the once rather than every time round the loop.

LOOPS2 and *LOOPS3* show an alternative scheme in which a vector of line labels is computed before the loop, quite a common technique when the number of iterations is known in advance. *LOOPS2* is indeed a bit quicker than *LOOPS1* but only marginally faster than *LOOPS1A*. *LOOPS3* introduces the penalty of repeated + operations.

(iii) Format and execute

It is not a good idea to have to change the format of data from character to numeric — the process can be quite slow. A common problem with systems that have 'evolved' is that data undergoes repeated type conversions because the initial phase of data analysis has been skimmed. See what sort of overhead is introduced by having to convert from character to numeric before doing some arithmetic.

```
A←,(⍋1000 1ρ1000?1000), ' '
B←1000?1000
```

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
R←±A	15050	16310	11150	17640
R←±B	400	1100	280	550

You should also remember that execute is doing just that, executing the APL statement. The following example arose recently when we did have to execute a text string. We were given some 'comma delimited' data (a common sort of interchange format) and we executed it without considering the penalties:

```
A←~1↓,(⍋100 1ρ100), ' '
B←,(⍋100 1 ρ100), ' '

```

(50 Executions)	APL.68000	APL*PLUS/PC	APL2/PC	APL*PLUSII
R←±A	4117	1150	930	1540
R←±B	1550	280	220	330

See what all those hidden catenations did.

(iv) Use the utilities or system functions

It is worth investigating well-written utilities as these can often usefully replace 'raw' APL. Some APL interpreters will supply special system functions or assembly language routines to help speed up operations that are not particularly quick in APL.

Prime examples would be String Search ($\square SS$), Matrix to Vector conversion ($\square BOX$ or $VTOM$) and Delimited Blank Removal ($\square DBR$). String Search, for example, in 'raw' APL often uses the $\wedge . =$ inner product. This is undoubtedly elegant in terms of code but horrendously wasteful of time and space. Each character of a string is examined for a match rather than carrying out only enough checks to see if a match is possible. Large intermediate and final results are generated.

If you do seek to replace a bit of your code with an imported utility or system function, do look out for minor differences in behaviour, especially under boundary conditions.

(v) Looping or non-looping

I will not offer any specific examples here. APL's array-handling capabilities and powerful range of functions and operators allow many operations to be carried out in a single line of code. In principle this should be the best way to do things — you are avoiding the overhead of APL having to interpret lines of a function over and over again. On the other hand the non-looping solution may be achieved at the expense of workspace overhead (using $\wedge . =$ for string search, for example).

Not Using APL

APL is not going to do very well with problems involving many repetitive (simple) operations on small amounts of data. A prime example here might be where a computation depends on the result of the previous computation.

Under these circumstances, the APL programmer can do a certain amount to speed up his APL code but in the end he may have to consider calling non-APL routines. These might either be via an Auxiliary Processor, *MA* or one of the 'APL compilers'. This route needn't be as arduous as you might imagine.

We were recently asked to supply an Auxiliary Processor to compute a Cyclic Redundancy Code (the famous CRC code that always crops up when something has broken). The algorithm involves Modulo-2 long division which in turn requires a long, looping APL solution. We found that, once we had understood the Maths involved, it took roughly the same amount of time to write the answer in both APL and Assembly Language (using two programmers who were of equivalent expertise in APL and Assembly Language respectively). The Auxiliary Processor was 480 times faster than the APL solution.

I quote this example because in one of the sources I have used (Wiedmann) he quotes an identical job done in the early 80s at STSC. Their Assembly Language solution was 860 times faster!

Those who like to compare the 'size' of APL solutions versus other languages might also like to note that this is also an example where APL doesn't give you a vast advantage in terms of code size. The APL solution was 12 fairly complicated lines of code and the Assembler source that actually did the work was 32 lines (albeit with another 30 or so lines for the APL-AP interface).

References

- | | |
|-----------------|---|
| Smith[1] | <i>A Design Handbook for Commercial Systems</i> , Adrian Smith, J Wiley, 1982 |
| Smith[2] | <i>Stone-wallling in APL</i> , Adrian Smith, Vector Vol 9 No 1, July 1992 |
| Sykes | <i>Use and Misuse of APL</i> , Roy Sykes, STSC, 1973 |
| Wiedmann | <i>Efficiency in the APL environment</i> , Clark Wiedmann, Proceedings APL85 |
| Eastwood, Poate | <i>APL Programming Techniques</i> , David Eastwood & Robert Poate, MicroAPL training material |

Genetic Algorithms: Their APLications

by J.M. Sandles

This is a write-up of the talk I gave to the British APL Association on December 2nd 1992. I have tried to include as much as possible from the presentation, but at times have had to shorten some of the explanations which were a lot easier using the visuals.

The underlying idea of genetic algorithms (GAs) originates from nature. Primarily they are search algorithms to handle combinatorial-type problems. In this paper I will give a rough idea of the sort of problems that genetic algorithms will tackle, how they work, and how they are coded in APL. Genetic algorithms provide a useful alternative solution to problems that are hard to *formulate* as LPs (Linear Programs are used to solve a variety of minimisation/maximisation problems) or that are hard to *solve* using heuristics.

What Sort of Problem do they Solve

Problems are generally solved using some kind of search or optimisation technique. There are three types of search — calculus-based, enumerative and random. Calculus-based methods are local in scope and can encounter difficulties with multiple-peak problem spaces.

GAs, APL and Speed ...

GAs are inherently parallel, and fit the parallel world of APL very well. Unfortunately, APL has not yet been ported onto a parallel architecture (well mine hasn't anyhow — any offers?), and because of this the algorithms tend to run slower rather than faster than existing methods. But, they may solve your harder problems better than what you already have, and they may also allow you to solve new problems faster, given that you require a less intricate knowledge of the problem using this method.

Back to Basics — a Short Biology Lesson

One of the most frightening things when coming to terms with GAs is that the literature is plagued with Biological terminology. Because it draws heavily on Genetics, you get confronted with terms like genotype, phenotype, meiosis,

dominance and recessiveness, and so on [1]. These all have a role to play, but for the purpose of this introduction I will ignore them. A simple genetic algorithm can get by without any of these concepts. This is because a GA actually draws upon another biological process much more than it does on genetics. This process is that of natural selection and evolution. Darwin believed that lifeforms became better adapted to their environments through natural selection — and hence GAs produce better solutions by mimicking this process.

An example of how this occurs in real life is found in the peppered moth. The peppered moth exists in both a white and a black form. It lives on trees and it relies on the trees to camouflage it from potential predators. In a normal environment, most trees are covered in a light lichen which means that the white variant is better adapted to the environment. In these conditions the light variant can be observed in much greater numbers than the dark variant, but some of the darker variant always survive from generation to generation.

In industrial areas, where the lichens turn black due to pollution, the darker variants are much better adapted to the environment and since the darker variant had never been wiped out it soon becomes dominant in this region. This shows that the process of selection is optimising the lifeform to any given environment — but since the process is random it maintains some older solutions in case of change. But, even more importantly, can the species change to a completely new environment?

For this the process of selection needs to do more than choose between existing black and white moths — it needs to create new variations to meet new needs. Nature has two methods of bringing about variation; the first can be viewed as a kind of interpolation — called crossover — where it takes parts of the genes from each parent and recombines them to form a new individual. For the moths the gene which specifies black colour crossed with the gene which specifies white colour could well produce a grey species (well adapted for an environment of grey trees!).

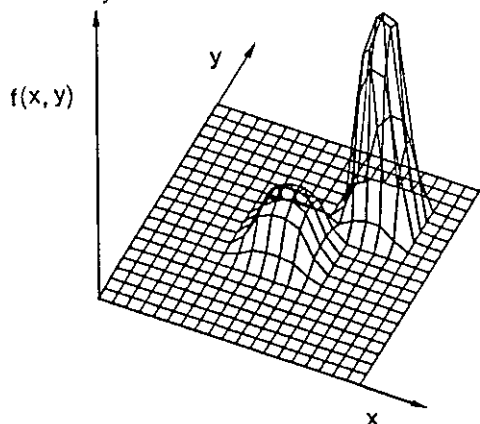
The second type of variation is much more drastic and is required when some totally new environment comes along and threatens the species (for example pink trees). This is called a mutation and is the random changing of a gene (for example pink moths!). Most mutations will be destructive, but every now and again a mutation will provide a species with just what it is looking for, and the feature will evolve (giraffes' long necks, enable them to eat leaves from higher in trees — hence less competition and hence they are better adapted).

GAs also use these same two methods of obtaining variation. To summarise how GAs draw on their genetic counterpart:

Components of a simple Genetic Algorithm

	GA	Genetic analogy
1	method of representing possible solutions	chromosomes & genes
2	method of judging fitness	survival of the fittest
3	method of random selection of the parents of the next generation	natural selection
4	Variation: 1. Crossover 2. Mutation	Crossover Mutation
5	A POPULATION OF POTENTIAL SOLUTIONS	A POPULATION OF INDIVIDUALS

The major difference between a GA and a conventional algorithm is that the GA carries a population of potential solutions at each iteration; so that rather than just moving from one point in the sample space to another it moves from many points to many points, often combining information it has gained from various parts of the sample space (crossover) and sometimes making random jumps (mutation) in order to find new parts of the space. Goldberg[2] illustrates this nicely:



A traditional algorithm starts at a random point in the space and moves from one point to another; it stands a good chance of getting stuck in local maxima. A GA takes a population of initial solutions and moves through the space by using combinations of each solution (crossover), and will jump off local maxima using random leaps (mutations) so that it finds a better population of solutions.

A Simple GA Workspace

The simplest form of GA is that which works on a species (solution) which is encoded as a binary vector. It will soon become clear why this is the case.

There are three global parameters involved in the process:

P_c — the probability that two parents are crossed using crossover.

P_m — the probability of a gene being mutated.

P — the population size.

The way these are set is also problem-dependent. High crossover rates and mutation rates lead to faster climbs up the solution space, but may also cause the problem to not converge due to too much good information being lost at an early stage. A larger population size is required as the problem gets more complex, but this increases both processing time and memory usage (*WS FULL!*).

Crossover will be of the simplest form — cross two parents at a randomly chosen point in the string, e.g.

$$\begin{array}{l} A = 1\ 0\ | 0\ 1 \rightarrow A' = 1\ 0\ | 0\ 0 \\ B = 1\ 1\ | 0\ 0 \rightarrow B' = 1\ 1\ | 0\ 1 \end{array}$$

Mutation is also simple — just toggle each bit with probability P_m ; usually P_m is very low to prevent too many bits being changed.

So how do we begin? The initial population is created at random. For this problem, if we have a population size of 10 we could do:

$POP = 1 + ?10\ 402$

The procedure then is a simple loop (sorry folks)...

```

      V RUN;gen;POP;fitness
[1]   a Simple genetic run
[2]   gen←0
[3]   POP←INITIALISE
[4]   REPORT(gen)
[5]   Top:gen←gen+1
[6]   →End IF gen>maxgen
[7]   fitness←FITNESS POP
[8]   POP←fitness SELECT POP
[9]   POP←CROSS POP
[10]  POP←MUTATE POP
[11]  →Top
[12] End:
      V

```

The good news is that no more looping is required. *FITNESS* returns a vector of values for each member of the population, which should be a reflection of how good a solution is. Sometimes this function can be quite tricky to write. Care should be taken to ensure that the fitness function returns values which are representative of how good a solution is. If you do not distinguish enough between a really good solution and an average solution, the GA will never converge to a solution. If you distinguish too much then the chances are the GA will get stuck in local maxima.

Adrian Smith [3] helped with the APL for *SELECT* and *CROSS*. *SELECT* builds up a weighted roulette wheel of relative fitnesses, and then picks as many random points on it as we require parents. This is modelling the random nature of natural selection. Members of a generation with higher fitnesses have a higher probability of being selected as parents of the next generation.

```

      V R←FITNESS SELECT POP;wheel
[1]   a Breed POP according to fitness
[2]   wheel←+\\100×FITNESS÷+/FITNESS
[3]   a Now spin and pick...
[4]   R←+wheel÷.<0.01×?(1÷POP)ρ10000
[5]   R←POP[1+R;]
      V

```

CROSS assumes that the population size is always even. This would be checked for somewhere in the initialisation code. *CROSS* swaps genes very nicely using ϕ , the extra dimension required for this to work comes from the indexing into

POP. It calls a function called *FLIP* which takes a vector of cross-probabilities *Pc* and returns a boolean. Each element will be 1 with the associated probability that it is passed. (*RANDOM* takes a number *N* and generates a vector of *N* random numbers between 0 and 1).

```

      V R←FLIP VEC
[1]  A Flips a coin
[2]  R←((RANDOMpVEC)≤VEC)
      V

      V R←CROSS POP;n;choice;site;crossmat;lchrom;pq
[1]  lchrom←1+pPOP ∘ n+0.5×1+pPOP
[2]  choice←(FLIP npPc)/n ∘ site+?np-1+lchrom
[3]  crossmat←(n,lchrom)p0
[4]  crossmat[choice;]←(site+.z;lchrom)[choice;]
[5]  pq←(2,n,lchrom)pPOP
[6]  pq←crossmatφ[2]pq
[7]  R←(n,lchrom)pq
      V

```

All we need now is *MUTATE*, which again uses *FLIP* to generate the indices at which to mutate:

```

      V R←MUTATE POP;s;i
[1]  A Mutate sites with probability Pm
[2]  s←pPOP ∘ POP←,POP
[3]  i←(FLIP(pPOP)pPm)/pPOP ∘ POP[i]←~POP[i]
[4]  R←spPOP
      V

```

And that is all you really need. The APL is considerably more elegant than its Pascal equivalent [2] and nested APLs would be even better. What you may also require is some kind of reporting mechanism to let you analyse generations and their progress. I built a small database table for each generation and dumped each generation off to file. It was then easy to set up an analytical workspace with graphs of fitness etc. to allow me to monitor progress.

In the following example you will see that although the underlying principles are the same, the methods of crossover and mutation are usually application-specific, and you often need a bit of ingenuity to define operators which remain appropriate to the problem.

A GA Problem — Magic Squares

To illustrate how this method works I will use the classic permutation problem of the magic square. An n by n magic square contains the numbers 1 to n^2 and each row and column of the square must sum to the same value, as do the two diagonals. This number can be shown to be $\frac{1}{2}n(n^2+1)$. An example of a 5 by 5 magic square is:

	5	23	15	16	6	=	65
	11	13	22	12	7	=	65
	4	1	17	25	18	=	65
	24	9	8	10	14	=	65
	21	19	3	2	20	=	65
==	==	==	==	==	==	==	
65	65	65	65	65	65	65	

Using GAs, this square was found using 1,158 generations using a very simple technique. Convergence to a near optimal solution is very quick (about 30 generations) but it has to run for considerably longer to find an exact solution. It has not yet found a solution to a square with even n ; I would like to see some kind of a proof as to why you can not solve even- n magic squares.

It is quite simple to set up the GA. The representation is easy. I used an n by n matrix of integers (i.e. the magic square!). The fitness function is also quite simple; I used the sum of squared differences of all the column and diagonal sums from the required value. I experimented with weighting the diagonal differences — hence forcing the algorithm to solve the diagonals first and the columns later. This was quite effective.

Crossover is unnecessary (set the probability of crossover to 0), because any crossing of squares will violate the permutation. There are some advanced forms of crossover which maintain the permutation, but I have not yet tried these. All the variation then must come from mutation — which is also redefined for the problem at hand.

Mutation needs to be redefined because of the nature of the squares. They are a permutation and hence the mutation needs to preserve this. It will swap one value in the square with any other value close to it. How close this is, is defined by a global variable — the mutation granularity — this was set to 10 in the above run (this is similar to the mutation probability — in that it controls how drastic a mutation is). It should be set proportionally to the size of the square.

Using this simple technique you will soon find yourself solving odd numbered magic squares. But it will be very slow.....

This problem illustrates more than anything that GAs can solve very strictly defined problems, only needing a small amount of time to get close to the solution, but if the algorithm is not refined as searching progresses, the search generally reduces to being random. I refined the above problem by reducing the mutation granularity as the search progresses. This seemed to prevent too much wandering about the problem space at the end of the run.

Here is the 13 by 13 magic square which it solved in 1155 generations....

8	65	167	11	2	168	157	83	134	23	111	71	105
33	14	63	40	4	160	143	137	48	53	130	119	161
145	55	141	133	22	108	116	26	28	10	117	97	107
81	152	19	27	164	86	1	76	127	113	100	79	80
159	62	43	89	16	75	135	50	115	165	15	147	34
5	47	99	163	140	146	3	64	35	138	112	102	51
46	49	91	151	154	18	69	41	57	139	30	132	128
123	162	144	39	149	7	31	122	166	52	73	13	24
95	106	169	59	70	88	155	90	45	12	104	25	87
148	78	21	103	101	82	9	129	131	96	56	66	85
158	72	67	124	42	114	20	94	54	118	142	68	32
98	150	37	92	120	17	156	84	29	61	77	126	58
6	93	44	74	121	36	110	109	136	125	38	60	153

Review

I have no doubt that in the future (and to some extent even now) GAs will become a much touted methodology. We need to understand the basic principles involved in order to appreciate that they are in fact very simple. I personally feel that they are best suited to solving ad hoc problems of which we have very little prior understanding. Unfortunately they are too inefficient (unless implemented on a parallel architecture/supercomputer) to match more traditional methods applied to well-understood problems.

References

- [1] Holland, 'Adaptation in Natural & Artificial Systems', University of Michigan Press, 1975.
- [2] Goldberg, David, 'Genetic Algorithms', Addison Wesley, 1989.
- [3] Smith, Adrian, 'Genetic Algorithms' — personal communication, Nestlé Rowntree, 1992.

Herdy, M, 'Applications of Evolution Strategies', 'Parallel problem solving from nature', Lecture Notes in Computer Science no. 496, p188, 1991.

Goldberg, David, 'The theory of virtual alphabets', 'Parallel problem solving from nature', Lecture Notes in Computer Science no. 496 p13, 1991.

APL-Club Germany, Schwetzingen, November 6 1992

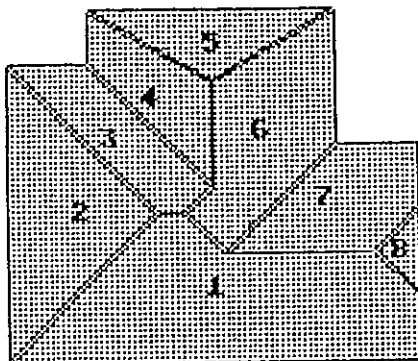
notes by David Eastwood

We haven't often been able to cover meetings of other European APL user groups in Vector but I was delighted to be invited to attend a meeting of the APL-Club of Germany recently and I thought that the readership of Vector might be interested in some details of the meeting and also of the APL-Club of Germany. The APL-Club of Germany has about 120 members and conducts a broadly similar range of activities to the BAA. In contrast to the BAA, the APL-Club only has two meetings per annum but these are all day events. On this occasion the meeting was held in a hotel in Schwetzingen (just outside Heidelberg) and a number of delegates met the night before at the hotel for a social evening.

The meeting was opened by Dieter Lattermann who welcomed the 30 or so delegates. Dieter used the opportunity to cover APL-Club business — in particular he distributed copies of Gary Helzer's APL Encyclopaedia which the APL-Club had translated into German. He gave a report on APL92 (see Vector 9.2 for our report) and gave news of next year's event in Toronto. Seven 45 minute presentations were given covering a diverse range of topics.

An Application of APL in the Roofing Trade

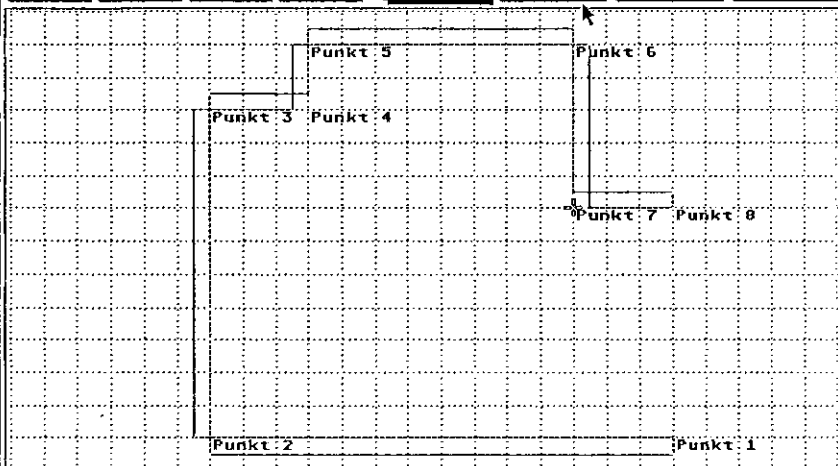
We began with a talk by Michael Zippel on the application of APL in the roofing trade (using Atari STs). Michael's Zimmerer package designs roofs, or more correctly generates specifications of the correct positions and connections of all necessary beams in the roof. The software runs either on an Atari ST or on a fairly fast 386 PC, and is designed to help the builder translate an architect's layout into a detailed set of working plans. The target design is shown opposite, and the diagram overleaf shows the first step — a layout for the supporting walls is entered via a 'CAD' screen.



HAUERPUNKT	9	Profil:	NEIGUNG: 45.000	KNIESTÖCK	TRAUFE
x=	3.000		Hofweg	h	Überstand 0.500
y=	0.000		Giebel	Obhz.r 0.150	Höhe 2.500
NULLPUNKT:	7			Fu.PF. 0.050	h 0.120



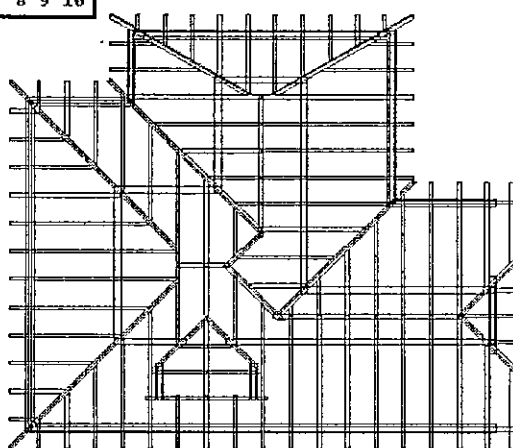
RECHNER GRUND PROFIL DATEN WEITER ABSCHLUSS ABRUCH HILFE



The system will then produce details of the required woodwork taking into account slopes and separations between beams — you will note a window has been added as well:

ZIMMERER Eingabe Daten Graphik Bild Darstellung ENDE

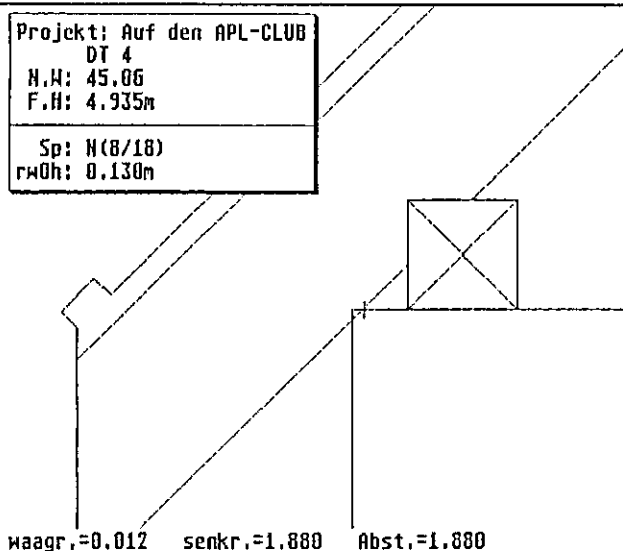
Projekt: Ausmittlung
DT 1 2 3 4 5 6 7 8 9 10



+

The final structure can be inspected and rotated and the user can 'zoom in' to see details of all the required cuts that have to be made at the various joints — some of these can be extremely complicated when a number of beams meet.

ZIMMERER Bild ENDE



The builder can produce an ordering list together with detailed drawings of individual beams. Michael alternated between the sort of screen picture shown here and photos from a construction site showing the same roof actually being built. This is the second successful APL package I have come across for use in the construction industry and it highlights a somewhat unexpected application area for APL.

Compiled APL for Supercomputers

We then jumped right to the other end of the computing spectrum with a talk by Walter Schwarz. Walter's talk was about 'Compiled APL for supercomputers' and covered the Acorn (APL to C On Real Numbers) APL compiler. I had first seen references to the Acorn compiler in the APL90 Proceedings (in a paper by Bernecky, Brenner and others). That paper discussed the compiler being used on a Sun 386 and also on a Cray XMP. In outline the Acorn compiler can compile a restricted set of APL expressions into C (the restrictions include use of real

numbers only — hence the name). Walter discussed more recent work (presented at a workshop on array functional languages in Montreal in 1991) where the compiler code was destined to be run on a parallel machine — the Connections Machines CM-2 system with a VAX or Sun front end. The project referred to was an exercise to do preliminary work to gain some understanding of working in a parallel environment, and also to study the results of executing Acorn code on this type of system. I won't attempt to cover the technicalities but those of you who have just bought a new PC might like to compare specifications. The CM-2 used had 32K processors and 1K floating point co-processors. Each processor had 8K of memory. Walter discussed the system architecture, the means of communication between the front end and the parallel machine, and also the sorts of APL functions that could be compiled and run on the CM-2.

The hypothesis that was being tested was that there should be a good match between APL and a parallel programming system, and it has to be said that this was not proven.

Performance was tested by running a number of routines that had been run on other systems such as the Cray X-MP. On the first test the CM-2 (parallel) system ran some 200 times slower than the Cray and with the second the ratio of CPU times was more than 400. This difference of 2 orders of magnitude was not expected (!) and investigations were carried out to see if the difference could be explained in terms of:

- wrong expectations about the relative machine performance
- problems in implementing the Acorn system for this architecture

To try and track these down, the performance of an inner product operation compiled by the Acorn system and also in CM-FORTRAN was compared. In these tests the Acorn code was one order of magnitude slower than the CM-FORTRAN. The absolute figures are interesting — the FORTRAN code reached speeds of 25 Mflops (millions of FP operations per second). In a second test, the interprocessor communication scheme was altered and one of the benchmarks re-run. This time round it was shown that by using an alternative inter-processor scheme the execution time of the algorithm could be reduced by almost an order of magnitude.

The conclusions drawn were firstly that no evidence could be provided to show that APL is a suitable programming language for the CM system, but also that many of the performance problems could be directly attributed to design choices in the implementation of the Acorn run-time system. More work is needed!

Summary of Other Sessions

I contributed a talk (in English I'm afraid) on APL68000 and also gave an overview of what the British APL Association gets up to. Otto Rusy talked about some work he had been doing in the product marketing field on product positioning, and then moved on to share his discovery of dialog boxes with the audience. Otto also explained some of the Dyalog window control system functions.

Bernd Gerber of the Karl-Friedrich-Gymnasium Mannheim talked about his efforts to introduce APL to his students. He has been using APL since 1985 in the form of various 'low cost APLs' starting with Pocket APL; then after a brief experiment with I-APL he has moved on to a combination of STSC APL and TryAPL2. Bernd painted a familiar picture of small or non-existent budgets constraining his efforts. APL is taught in some of the informatic courses, but is up against alternative options such as Pascal, BASIC, Comal and increasingly the use of packages such as Works. On the brighter side, Bernd reported that there are always some students who become fervent APL enthusiasts, although his comment that they never join the normal classes seemed to paint a familiar profile of the budding APLer.

Frank Juschus from the DKFZ — a medical research centre — gave an interesting talk about the Heidelberg APL Ray Tracing Model. Working on mainframes or very fast PCs (50Mhz 486 machines) they are using APL to analyse complex 3-dimensional structures for medical diagnosis and treatment. 2-dimensional data resulting from CAT scans are converted into pseudo 3-dimensional arrays to allow a visualisation of the scanned object from arbitrary angles.

Steven Halasz of Lingo Allegro from the USA gave a closing talk which reviewed attempts over the years to 'sell' APL. I must confess that since the presentation relied on an intimate knowledge of prime-time US comedy shows, my notes degenerated markedly. Indeed, I find that I can no longer distinguish between statements that I was meant to read as true and those that were meant to be taken as false. However, collected 'bon mots' included: 'if you weren't so stupid you'd like APL'. I think we were told that APL is successful (or was it unsuccessful) because 'IBM likes APL' and 'children like APL' (is either of these statements true? ... are they connected?!).



RENAISSANCE DATA SYSTEMS

Enlightenment Thru Information Processing

Specializing in Books and Software on APL, J
and other Curiosities of merit.

HAVE YOU GIVEN A COPY OF IAPL OR J TO A TEACHER OR STUDENT?

Name: _____ Date: _____

Street: _____

City: _____ State: _____ Postal Code: _____

Country: _____ Telephone: _____

For a copy of our most recent catalog,
please send a self-addressed, legal-
sized envelope (stamped if from U.S.)
or mail this form to:

Renaissance Data Systems
Park West Finance Station
P. O. Box 20023
New York, New York 10025-1510

Learn APL

APL Shareware/Demos and Related Publications

APL and Mathematics

APL AS A TOOL OF THOUGHT Proceedings

Other Important Sources of APL Information.

APL History

APL References and Techniques

Special Subjects in APL

APL Interpreters and Software

J=: A Powerful Dialect of APL

Other Software for Use with APL

To Russia with Love and for APL92

Other Curiosities of Merit

**I-APL: An International voluntary project of individuals and
companies seeking to share their love of APL.**

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

Seize the Time

by Dick Bowman

People have accused me of being very quiet recently; or busy at other things. Well, maybe, but I've come to some sort of conclusion about APL, and it's one that I feel bears the seeds of being potentially a very positive conclusion. Of course, we don't have a foregone conclusion. Let's think about revolutions, and the climate which causes revolution to happen...

Basically we have two ingredients in the revolution which might interest us most within our technical domain; an external situation where there is (or may soon be) a need for vastly increased productivity in building computer applications, and an internal situation where the tools at our disposal have improved significantly in the recent past. Of course, I'm writing here from a very narrow perspective, not one that sees APL as a notational advance for human thinking, just one that builds software to do useful, interesting, or paid-for things.

The first thing that made me realise we have to do something was seeing a flyer for a 'Rapid Application Development' conference, which conspicuously (to me) omitted any reference to APL. Now, don't I remember 'Rapid Application Development' being one of the keystones of our existence? Are we being frozen out, or are we freezing ourselves out?

Productivity

At APL92 one of the regular APLxx attendees made a rather surprising remark; I paraphrase him as saying that his company (an APL consulting house) was doing very well out of the recession. So he said it again. Why? Well, where he operates companies are very concerned about getting the most for their money, recognise the productiveness of APL, and as a consequence the demand for APL is at a premium over demand for less productive application building tools.

Here's one avenue to exploit.

Equally, to stay with this newfound optimism, let's believe that one day this recession will come to an end. It may happen sooner than we (and our employers, actual or potential customers) expect; the pace of recovery may be much faster once it begins. At this point there are going to be a lot of projects to plan, process models to update and so forth; all things where computing application development is a crucial precursor.

But where's the mainstream computing world nowadays? Look around yourself and you'll see the vast majority of 'for sale' development happening in C; which is fine for a final product to be sold by the truckful, or to be run over and over again. But have you costed out development in C against use of a high level language? Watch out though, the gap is narrowing.

APL Advance

The new generation of APL interpreters is more interesting than many of us realise; what's important is that in their various ways they recognise the true significance of both human and software interfaces. Dyalog and Manugistics bring us integration into Windows and other GUIs; APL2 comes as a family of more-or-less compatible products; Sharp APL for the PC provides a realistic APL at a realistic price (take a look at this one, it really is quite something); and MicroAPL gives us access to the Mac. What it's up to us to do is to begin capitalising on the tools available to us.

Here's a quote from the latest (November 1992) Byte magazine. '... what you mostly need to develop Windows applications at the moment is *patience...*' (their italics, page 253). Well, I don't have a lot of patience, but I do have some tools that let me build Windows applications using APL pretty well. They could be better, but so could everything. We have an advantage right now, we may not have it this time next year.

Choose Your Applications

One of my never-ending problems with APL is affordability, and it's a situation that probably can't change overnight. Some of us find scenarios where a single run of an application can justify not only the high-price APL interpreters but also the Sun workstation to use it on; not all of us are so lucky. But generally speaking I think there are two major application scenarios for APL to capitalise on.

- High value, non-repetitive modelling applications; the sort of place where a program never runs the same way unchanged. These are the people who can afford Dyalog as it's priced now; they have an excellent starting point.
- Personal programming, filling in the holes left by ready-made packages, where the programming team is y-o-u; Sharp APL/PC (I forget the precise name) looks good here.

Of course there are other application profiles, and you may very well find that APL is an excellent fit for your purposes in these; but I think that the two above are the most likely candidates for significant growth.

Tools

This is really my point; I think we have to take it upon ourselves to do a lot better here than we have in the past. Look at what the 'average' professional C programmer draws on: in addition to the base compiler there are debuggers, profilers, forms designers, utility libraries and so on. It's all very well for a prominent APL vendor to say that we get all of these inside their package, but my credibility is strained.

My impression of the typical APL programmer's toolkit is that it takes one of three forms:

- The stuff that came with the interpreter
- A ramshackle collection accumulated over the years, or homegrown
- Nothing at all

This has to change, and pretty dramatically. I think we have to begin a collective programme to build a more comprehensive and universally accepted toolkit. What I want to see develop is a situation where instead of APLers being able to agree on 20% of their known universe they can agree on 80%.

Here's what I suggest as some significant categories for toolkits:

- Meta-APL, which expands the common language core (all those APL2 emulations of $\square VR$ and so forth)
- Interfaces, both in the form of software and tutorial documents, so that hooking up APL to popular packages becomes a no-brainer activity
- Code management systems, for the mega-applications

Of course, this is not going to be a trivial pursuit. It may be a lot less 'fun' than exploring J. But it just may be part of the route to earning a better living with APL. It'll take time and it'll take money. These tools will have to be paid for, and I'd hope that we move into a mode where APL software is both inexpensive and readily bought and sold. We have to build it for ourselves and sell it among ourselves (just like the antique shops) so that when newcomers arrive they'll walk into a richer environment.

At the end of the day one of the prime indicators of health in an industry is the amount of third-party activity that surrounds the centre; at present I perceive APL as having very little. I think this will change. I think it has to change.

A Little J Horner

by Howard A. Peelle

Horner's method for evaluating a polynomial (see Spunde's Technical Correspondence, Vector 8.3 (Jan 1992) pp 138-9) can be expressed in J as follows, for (ascending) coefficients CS and a single value V:

```
+'*//).,V,.CS NB. From Iverson's AN INTRODUCTION TO J p.22
```

A defined verb

```
Horner =. +'*/ @ ). @ , @ , .
```

can be used (albeit inefficiently) with a list of values VS:

```
VS Horner"0 1 CS
```

Alternatively, a little verb which adds its left argument to VS times its right argument can be inserted in list CS all at once:

```
(+ VS&*)/CS
```

By way of explanation, a taste of J follows:

In the first expression, ,. is Append (an extension of APL's Catenate-along-the-last-dimension) which joins V to each item in CS. The resulting 2-column matrix is Ravelled (.) then Beheaded (.) does 1 Drop) producing a vector of coefficients interspersed with copies of V. (Note that J notation employs some two-character symbols, here, for example, ,. and). with period suffixes.) Then insert / (same as APL's Reduction operator with a vector) applies two verbs (functions) to that vector. Plus + and Times * (x in APL) are Tied (') together as a gerund (akin to a function array) — forming boxed noun equivalents of the verbs (see Bernecky's "Gerunds and Representations", APL91 ACM).

The J definition of Horner is similar to direct definition in that it uses only one line but uses the copula =. (like APL's ←) instead of a colon and is "tacit", that is, uses verbs without showing their arguments explicitly. Here the Tied verb +'* is the argument for Insert (an adverb); the resulting derived verb is the left argument of the Atop conjunction (@) and Behead (.) is its right argument. That verb composition is the argument of another Atop @ with Ravel (.) on its right. That, in turn, is finally composed Atop the result of Append (.,) applied to the

two arguments given to Horner. It might be noted that this definition is equivalent to — but not procedurally identical to — the algorithm above, which executes verb composition in the reverse order. This can be seen by typing the name of the verb (here, Horner) by itself on a J system, which produces a boxed representation showing the nesting of verbs (and adverbs and conjunctions).

Once defined, Horner can, of course, be executed: `V Horner CS`. It can also be executed, as is, with a list of values `VS` by applying the Rank conjunction (") shown here with a right argument of `0 1` indicating each scalar in the left argument to be used with the vector right argument (and the result assembled accordingly). Note that if a noun were given as the right argument (instead of pronoun `CS` here), either it or the right argument to " must be parenthesized.

Lastly, consider the alternative of composing a verb which adds its left argument to `VS` times its right argument and inserting that verb between each item of `CS`. `(+ VS&*)` is a "hook" composed of two verbs: Plus (+) and a monadic verb composed of `VS` With (&) Times (*). In general, a dyadic hook `x (f g) y` is `x f g y` and the conjunction & bonds its noun argument with its verb argument to derive a monadic verb.

The plum — defining a verb for the last expression of Horner's method — is left for you, Jack, to pull out.

University of Massachusetts
Amhurst, MA 01003 USA

From the Vital Execute to Fractals and 5-Fold Symmetry

by Gérard A. Langlet

Abstract

The vital execute is an infinite loop which will execute a program or an expression as many times as possible. Iterations stop as soon as any error is encountered or when a break occurs. We ARE driven by such a force until death. So is Nature. So is the Universe which is, at the same time, fractal, symmetric and asymmetric.

APL Simulation

Although it is possible to program such a concept in various APL-level-1 implementations, using error trapping which is not covered by the ISO standard, and then not fully compatible, we shall prefer the `⌈EA` (EXECUTE ALTERNATE) feature of APL2, which is simple, ignores error types and has been adopted by various implementers. *A* and *B* being APL statements, `A ⌈EA B` tries to execute *B*. If no error occurs, everything is all right; in the case of ANY error, the execution of *B* is interrupted, then *A* is executed. The best thing to do is to put into *A* an expression which never produces any error.

As an example, let us consider the following "vital-do" function which is short and self-explanatory:

```

      ∇ vdo ΔE
[1]  '→' ⌈EA ΔE ∘ →1
      ∇

```

`ΔE` is any APL expression. Character delta is used so as to allow the use of "normal names" within the expression. If no error occurs during execution of `ΔE`, this function generates an infinite loop with it. If any error or break happens, then the absolute branch is executed so that `vdo` as well as the possible calling functions are stopped in a clean way, i.e. with a clear state indicator.

Applications

For pedagogy as well as for everyday-programming, the vital execute simplifies the burden of controlling loops, unless one has disabled the break or interrupt key(s)! There is no more need to control the end of the loop. "vdo" is a synonym of "do it as much as you can", if not of LIFE itself.

Example: $I+5 \diamond vdo \text{ ' } 1I+I-1 \text{ '}$ *produces (in 1-origin)*
 1 2 3 4
 1 2 3
 1 2
 1
 + (empty vector)

and stops automatically because 1^{-1} has produced an error.

Do not try replacing "-" with "+", or buy paper and be patient!

The best usage of the vital execute we have found concerns automata, sorting (models for parallel processing), or data transforms (Fourier analogues in image-processing) especially when they involve recursive or fractal processes. Also, when programming in APL2, the "disclose" at various depths of sub-cells is not so easy, perhaps because of the absence of Iverson's function-ranks concept, and "vdo" has brought some help. Of course, the left argument of $\square EA$ can contain many other statements than the absolute stack-clearing branch, even an empty vector, so that the control returns to the calling program. ' $\rightarrow 0$ ' as the left argument of $\square EA$ may not be adequate in all cases, e.g. if ΔE calls another function, because the branch concerns the called function and not the calling one; however this is what we shall do from now on, in this paper.

We now give a short example which is bound to symmetries in Nature and results from the theory of binary integration [1].

$N \leftarrow pB \diamond vdo \text{ ' } R \leftarrow 1 \uparrow pB \diamond K \leftarrow 1 \uparrow L \leftarrow Rp1 \ 0 \diamond A \leftarrow L \uparrow B \diamond B \leftarrow A \neq K \uparrow B \diamond B \leftarrow B, A \text{ ' } \diamond B \leftarrow NpB$

Let B initially be a square bitmap with a dimension such as $2p8$ and try to see what happens after successive iterations of the above-given expression, which has (as usual in RISC-programming style) no parentheses, no goto (vdo taking care of the loop), and moreover, does not make use of arithmetical primitives at all, the only logical scalar primitive is $\neq$ (XOR), $1 \neq$ being formally equivalent to NOT.

In order to display bitmaps nicely, we shall use the following function:

```

      ▽ R←MAT R
[1] R←2/' ▀ '[R+[]IO]
      ▽

```

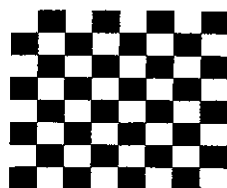
The first candidate will be the *COMB* matrix; let us see what happens, iterating three times the expression with the following *B*.

$B \leftarrow COMB \leftarrow \begin{bmatrix} 8 & 8+1, & []IO-.1 \end{bmatrix} \begin{bmatrix} 8 & 0 \end{bmatrix}$ and displaying iterates:



↑ initial *B*
↑ second iterate

first iterate ↑
third iterate ↑



Isn't it a good way of generating a chess-board?

Then we discover that the expression has the capabilities of:

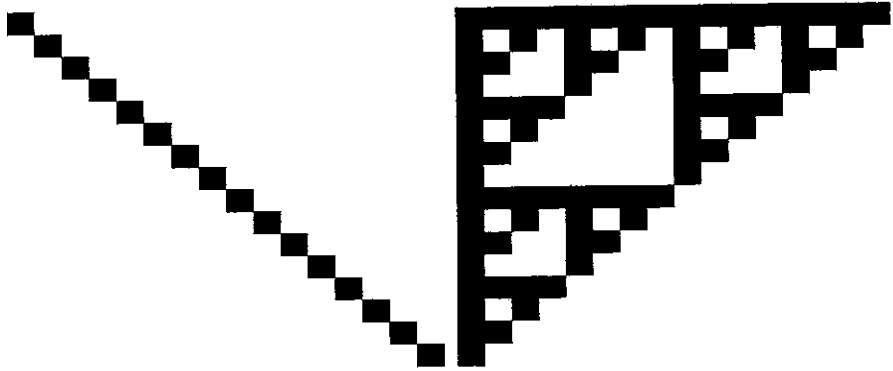
1. reproducing original *B* at the third generation;
2. producing a symmetric of the comb (equivalent of $\phi \ominus COMB$) at the first generation.

So, the expression is equivalent to a matrix product (try to find which one!) which indeed corresponds to a very powerful symmetry operator.

It is indeed connected to the way crystals grow in plane layers, especially quasi-crystals in physics. More than that, such an expression is able to build a nice fractal.

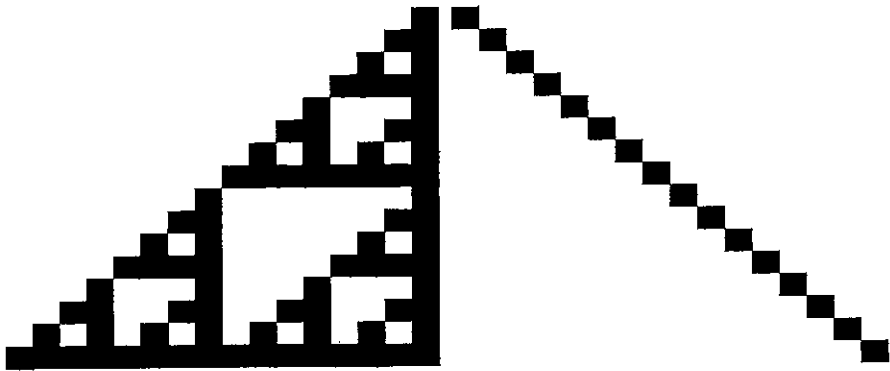
Let us try with a unit Boolean matrix as initial B .

$B \leftarrow 16 \rho 17 \uparrow 1$



$\uparrow$ initial B
 $\downarrow$ second iterate

first iterate $\uparrow$
 third iterate $\downarrow$



The first iterate is a Sierpinski triangle (also called the 16-geniton after the terminology of [1]), the second one is its $\phi\theta$ or $\theta\phi$ symmetric and the third one is the original unit matrix. This would be true starting from a unit matrix with any size.

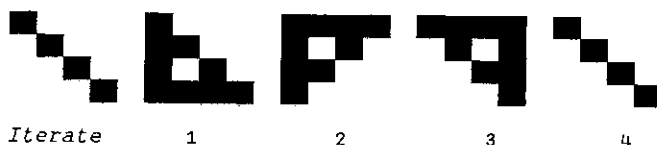
One can indeed relate the initial APL expression to matrix algebra, knowing the properties of genitons: the square of a geniton is its own binary matrix inverse, so that all genitons are cubic roots of the unit matrix. (The binary matrix inverse

is, when it exists, given by $\square CT > | \square B$ and the binary matrix product is $A \wedge . \neq B$ or $1 \wedge 2 | A + . \times B$ which is faster in most implementations, $1 \wedge$ being useless in APL*PLUS/PC, which has no Boolean coding, and in Dyalog-APL which converts to binary automatically.)

Now, let us try another version of the initial expression, after commuting the dimensions on which selections and catenations take place:

```
N ← ρB ⋄ vdo 'R ← 1 + ρB ⋄ K ← 1 ≠ L + R ρ 1 0 ⋄ A ← L / B ⋄ B ← A ≠ K / B ⋄ B ← A ⌊ B' ⋄ B ← N ρ B
```

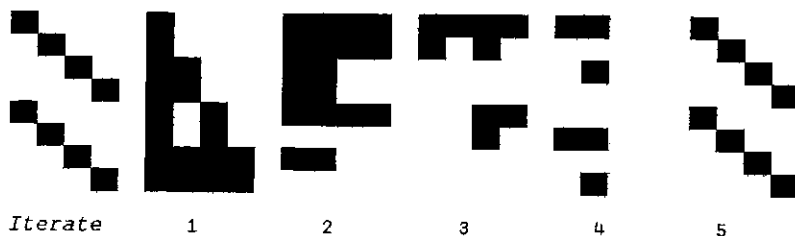
and try to display what happens to $B \leftarrow 4 \ 4 \rho 5 \uparrow 1$ a smaller unit matrix:



This new expression has a 4-fold symmetry capability:

The first iterate	is the Θ symmetric of the 4-geniton,
the second one	is the 4-geniton itself,
the third one	is the ϕ symmetric of the 4-geniton,
the fourth one	is the original unit matrix B .

Let us now experiment doubling B along its rows: $B \leftarrow B \uparrow B$:



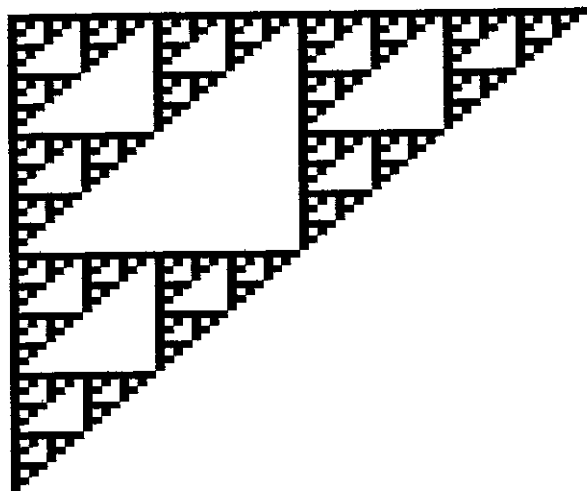
What has happened?

The original B becomes $2/\Theta$ of the 4-geniton as the first iterate, which produces some bizarre patterns as the second, third and fourth iterates... and is born again at the fifth generation. This would be true for any initial pattern with the same size: the 5-fold symmetry has suddenly emerged from a very simple APL expression with no complex numbers, no square root of 5, no trigonometry, no exponential, no golden section and no apparent Fibonacci series, even no arithmetic.

The 5-fold symmetry operators are bound to Penrose tilings and concern many processes involved in Life (not only quasi-crystals): from the structure of many viruses to the ones of starfishes, of silk fibers, of many plants and of our hands. All symmetries of the Universe can result from applying just $\neq$ with some elementary rules, even the ones of helices (and double helices) as shown in [1] and some other previous papers. Holograms, Fourier-like transforms as well as impressive genetic automata also result from the unique properties and of the *exclusive* use of the same logical primitive (*Exclusive OR*).

B+ 64 64 p65+1

First iterate +



These new facts could not have been brought to light without APL, the only programming language in which the use of $\neq$ i.e. binary integration or asymmetric parity propagation (the elementary process of Nature, which also leads to rebuilding chaos theory and gives at last the explanation of Mendel's laws as well as the one of Pauli's principle), is really intuitive.

References

- [1] G.A. Langlet, *Towards the Ultimate APL-T.O.E.*, APL92, St Petersburg, Russia & APL Quote-Quad, ACM Press, USA (July 1992).

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

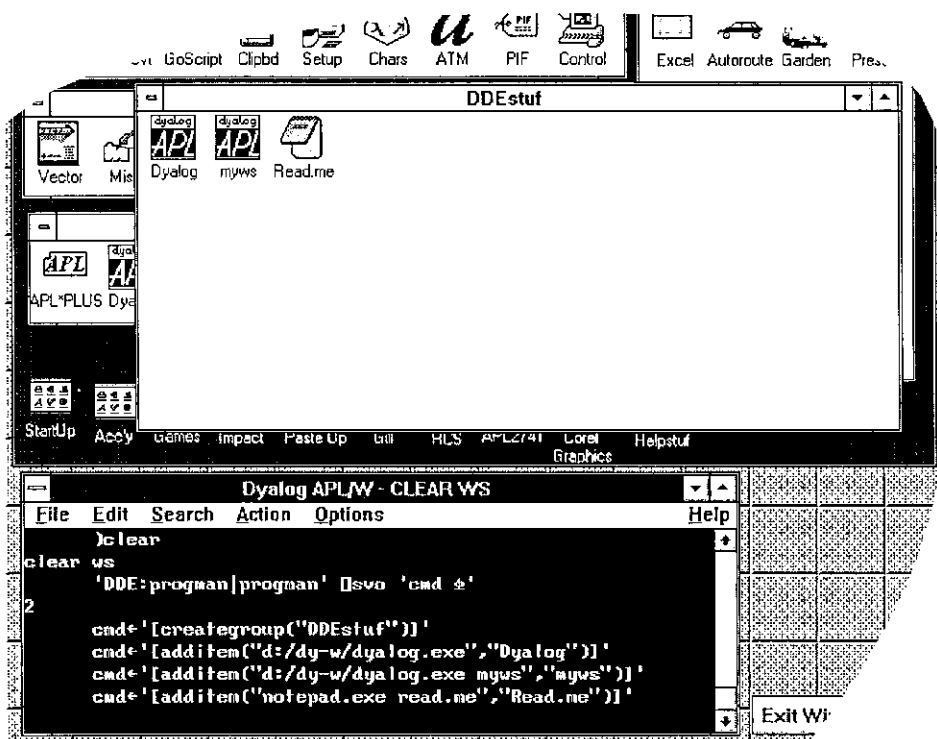
Contents

Hackers Corner: Talking to PROGMAN	Adrian Smith	98
Technical Correspondence		100
ASWI Programming Standards; some Why's.	F.H.D. van Batenburg	101
APL*PLUS II Keyboard Configuration	Ajay Askoolum	107
Integrating with Insight	Norman Thomson	113
Some J Verbs for Orthogonal Factorial Experiments	Keith Smillie	117
Jacobi's Method for Eigenvalues an Illustration of J	Donald McIntyre	125
AMENDMENT: "A Change for the Better"	Donald McIntyre	134
An Example of Intransitivity in Probability	Graham Parkhouse	141

Hackers' Corner: Talking to PROGMAN

by Adrian Smith

Did you ever wonder how such a wide variety of Windows installation routines manage to get their target software added into Program Manager? The final act of a well-behaved 'Setup' is to add a new group, and then populate it with a string of icons, usually including an instance on 'Notepad' preset to open a READ.ME file. Something a bit like this, in fact:



... all of 5 lines of APL achieve the same effect! As is so often the case with Windows, there is a simple and well-documented interface — once you find the appropriate documentation [1]. In this case you need to look towards the back of the chapter on DDE, but you also need to understand a little of how Windows applications treat 'DDE-Execute' commands.

In the examples from Vector 9.2, I showed a few ways of exchanging data with a couple of standard Windows applications — MS Word Vn 2.0 and Excel Vn 4.0 [2]. In both these examples, I worked with a DDE 'Topic' which in some way marked out the data to be shared — in Excel it is a named range, in Word it is a block of text marked with a Bookmark. All well-behaved Windows programs also offer the additional option of communicating with a 'system' topic, and one of the useful things this does is to *execute* commands passed over the DDE channel — essentially you can drive the application by remote control!

Windows Program Manager illustrates the technique rather well — in the example shown opposite I first set up a connection to its 'system' topic (called 'progman' just to add a little confusion):

```
'DDE:progman\progman' □svo 'cmd 1'
2
```

From now on, anything assigned to *cmd* is executed for us by Program Manager ... the example adds a new group and inserts a couple of APL icons (and a Notepad icon) into it (yes, all those [] and "." do matter!):

```
cmd+ '[creategroup("DDEstuf")]'
cmd+ '[additem("d:\dy-w\dyalog.exe","Dyalog")]'      a ... etc.
```

The Program Manager macro commands you need to know about are:

```
cmd+ '[creategroup("Groupname")]'
cmd+ '[additem("cmdline","descr","Iconfile",IconIndex)]'
cmd+ '[deletigroup("Groupname")]'
cmd+ '[showgroup("Groupname",action)]'      a See below ...
```

'Creategroup' will simply display "Groupname" if it already exists; the entries for "Iconfile" and "Iconindex" are optional ... by default you get the first icon in the .EXE specified as the *cmdline*. Valid (and useful) *showgroup* actions are:

1. Activate and display group window
2. Activate, but iconize group
3. Activate and maximize group
6. Minimize group window

I hope that gives you enough information to get you going.

References

- [1] Microsoft. *Windows 3.1 SDK Guide to Programming*, Chapter 22, pages 20-24.
- [2] Smith, Adrian. *Shared Variables and Windows DDE*. Vector Vol.9 No.2 p128.

TECHNICAL CORRESPONDENCE

APL2 Packages: a Cautionary Tale

From: Nicholas Small

12 Dec 1992

The following cautionary tale, relating to the use of packaged workspaces in APL2, may cause readers some amusement.

On 3rd November I made some modifications to a workspace and saved it. A few days later one of my users came to me to complain that the date appearing on the printout produced by the program was incorrect, although the data were up to date. The date being printed was 3rd November.

A quick examination revealed that the date was put in the output from an object named *TODAY* — but this object was not a variable set by the program, rather, it was a function, and a packaged function at that. Examination of the listing for this function revealed all: function *TODAY* looks to see if a variable *Today* exists and if it does then the value of *Today* is returned; otherwise *Today* is set to the current date and that is the value returned. (I guess that it was written like that to facilitate the production of a report with a given date.) When I saved the modified workspace, the value of *Today* was saved with it. What I should have done next (see my article in Vector 7.2) was to *)CLEAR*, *)COPY* the workspace, reset the latent expression, name the workspace, then *)SAVE* again. Simple enough, I suppose, but easy to forget when one's attention is distracted.

Nicholas Small
8 Cardigan Road,
LONDON E3 5HU

ASWI Programming Standards: Some Whys

From: F.H.D. van Batenburg

15 June 1992

Vector 8.2 published our ASWI standards for programming and documentation. This publication listed our proposal as a number of prescriptions without elaborating upon the reasons behind the various ideas.

It is not always easy to determine what is obvious in a proposal and what is not. In the discussions we had in reaching the ASWI standard some points were discussed at length, while others were taken for granted immediately among the participants. But some reactions from Dutch users among whom we circulated a preliminary version showed still different attitudes. This shows — not surprisingly, of course — how different experiences make up for the initial acceptance or rejection of different points.

Recently, Ray Cannon reacted in Vector 8.4 against various parts of the ASWI proposal. I am pleased to see his comments as they help us to pinpoint the most “offensive” parts and hence enable us to select those points which need more explanation, or which need to be changed.

In this sequel I will explain some of the considerations regarding:

- *SID* in user-functions
- *SID* in sub-functions
- *SID* in standard functions
- lowercase characters
- length of names of local variables
- miscellaneous

SID in User-functions

Let us start with the *SID* in the name of functions. ASWI proposes to start the names of all user-functions with a 3-letter code, the *SID* (System IDentification). First of all, this proposal did not descend from heaven as a purely hypothetical idea. On the contrary, it is the formalisation of a long-standing practice at Philips and a big bank in Holland during more than a decade. Subsequently that proposal did not generate any discussion in the ASWI committee (or, as yet, elsewhere), because those users were already very satisfied with this practice. This alone, however, doesn't make the benefits obvious, and Ray's objection indicates that some elaboration of the reasons is indeed required.

Yes, in extreme cases, it is harder to find a particular function if all the functions in the workspace start with the same *SID*. But in general, a WS is **not** only filled with functions with the same *SID*. In my simulation workspace for example, one finds *DLG*-functions to manage the user-dialogues (copied from workspace *DLG*), one finds *MDF*-functions (from workspace *MDF* with Machine-Dependent-Functions), one finds *GRF*-functions (from a graphics workspace), and so on. Now a *)FNS* or a *□NL 3* beautifully lists each of those groups together. So this is much more of a help than a hindrance.

Moreover, I see this recipe applied in real-world applications often enough (for example in the file-workspaces of APL68000 where the Atari-functions are called *STREAD*, *STTIE*, *STWRITE* and the Macintosh functions are called *MTIE*, *MREAD* and *MWRITE*). Nor were there any objections in the past to the names of the functions *TSTEXPONENTIAL*, *TSTNORMAL* and *TSTPOISSON* in my workspace *TSTDISTR* (this workspace tested against various distributions). Nor with the functions *RNDEXPONENTIAL*, *RNDNORMAL* and *RNDPOISSON* in my workspace *RNDVARTS* (this workspace generated random numbers according to different distributions).

So, again, I would say that those facts indicate that application of a *SID* was considered useful long before our proposal.

Furthermore, I wouldn't object if functions in the APL Statistical Library had for example the prefix *DES* for all descriptive statistical functions and had the prefix *TST* for all testing functions, or even *TNR* for a collection of functions with statistical tests which assume normal-distributed variates and *TOT* for other tests. I must admit that I can also imagine that one prefers *MEAN* instead of *DESMEAN* and *SPEARMAN* instead of *TSTSPEARMAN* because *MEAN* and *SPEARMAN* are so well established names that one doesn't want to burden the user with those prefixes. So this example is an in-between, for which there is probably as much in favour of the prefix as there is against it.

Ray is quite right that sometimes a *SID*- prefix in the name is undesirable, but I don't think that APL programmers are much disturbed by such a *SID*-prefix. However I can imagine that a *SID* prefix might be undesirable for the end-user sometimes ...

- not if the end-user doesn't see the main-program because the workspace starts in *□LX* there is no problem.
- not if the end-user has to activate just a few functions like *LITRETRIEVE* and *LITSEARCH* in a literature system, although I must admit that here our recipe might be debatable.

- but yes, I definitely agree that ...

LITRETRIEVE LITALL LITBOOKS LITOF 'BROWN'

is ludicrous and should be...

RETRIEVE ALL BOOKS OF 'BROWN'

So in general I would say that a *SID* prefix is often very useful and that such a proposal in ASWI is merely a formalisation of a long-standing practice. I must admit however, that our proposal was probably stated somewhat too categorically and that it should have left some room for exceptions.

SID in Sub-functions

For imported subfunctions there is nothing wrong with using a *SID*-prefix either. Regarding Ray's statement "a utility should be independent of its origin" I would like to comment that a utility should not be independent of its task; and its task determines its prefix. I found it very useful that the graphics of STSC started with $\square G$ and the file functions with $\square F$ and the window functions with $\square W$. Again, nothing purely theoretical, but a real-life implementation before our proposal came to life.

Now what about a prefix for *IOTA*? If I knew why Ray programmed a function *IOTA*, I wouldn't have problem finding a proper *SID*. Was the intention to simulate standard APL functions among which was *IOTA*? Then why not *APLIOTA*, *APLESTI*, *APLRHO*, and *APLTAK*? Then the *SIM*-workspace would either contain an *APLIOTA* or a *SIM_IOTA*. The name *APLIOTA* indicates that this is the original utility from the *APL*-workspace. And the name *SIM_IOTA* would indicate that the function is not any longer working as one might expect, because it is more or less subtly adapted for use in the *SIM*-workspace.

The advice from ASWI is to change the name of an imported subfunction as soon as one tampers with it to adapt it for the particular application in which it is imported. Ray comments to that "a true utility's name should be independent of its origin, if the origin of a function is important, then comment it, don't clutter up the space name". To this I would say: if you adapt a utility for a particular application it is not a "true utility" any more. It is exactly to warn against unsuspected deviant behaviour that this proposal will protect you. Comments alone are certainly not powerful enough (do you consult the comments every time you invoke a utility? For many I don't).

SID in standard functions

The only case in which I have some real misgivings about the ASWI standard is about the *SID*-proposal regarding the standard names like *SIDΔBEGIN*, *SIDΔABSTRACT*, *SIDΔDESCRIBE*, *SIDΔHOW* etcetera. There was much debate in the ASWI committee about this.

1. One group argued that they often merged several workspaces together. If no prefix was used, with each new workspace copy the previous *BEGIN*, *ΔDESCRIBE* and *ΔHOW* would be replaced. As they wanted to consult all the descriptions they preferred a prefix so one could still recognise all descriptions.
2. The other group argued that most of those *ΔFUNCTIONS* are used by novices who don't know the workspace. So they don't know the *SID* either and they just want to load the workspace and type *ΔABSTRACT* or *ΔDESCRIBE* to see what they have in their hands without bothering about any *SID*.

We finally opted for the first case, but our committee was not unanimous.

Lower-case Characters

For global variables ASWI requires that a name consists of lower-case characters preceded by the *SID*, and for sub-global variables a name of lower-case characters preceded by *SID_*. Ray is right that it is useful to mention the alternative of underscored characters for implementations without lower-case characters. We didn't do this because basically our proposal was drawn for a project of shareware program development using APL2/PC and APL68000. However, to take other APL implementations into account would give our proposal a broader impact. So I propose to mention in the ASWI committee the alternative of underscored characters in the next version.

Length of Names of Local Variables

ASWI requires that names of local variables do not exceed 4 characters. Although the particular number 4 is somewhat arbitrary of course (why not 5, or 3?) my experience is that APL functions tend to lose their surveyability rapidly with lengthy names. As most of the APL functions and operators occupy 1 character, a variable of, say, 4 characters already dominates a code-line much more than warranted. So my personal preference is to use 1 or 2 character names in small functions, and highly incidental 3 or 4 characters in bigger functions. It is my experience that "natural names" can be abbreviated to 2 or 3 characters and still be recognisable. The ASWI committee doesn't share Ray's fear that shortening a "natural name" will diminish readability. We believe the contrary.

Miscellaneous

The ASWI standard ends with a Miscellaneous paragraph with "a few remarks...on APL programming practice". Here we suggest that "good" APL-functions...

5. work for missing left argument (default) and don't crash for empty arrays.
6. receive data in the right argument and control in the left.

Both suggestions are gross over-simplifications. They were intended to steer APL programmers towards some preferred standard practice. These remarks were not intended to mean literally that either...

- a function which desperately needs data in its left argument is a bad function.
- a function which has a default for a missing left argument is always a good function.
- a function which has control and data in the left and right arguments respectively is always a good function.
- a function which has data in both the left and the right argument is always a bad function.

What we did want to accomplish was to force a programmer to reflect about his design and to wonder...

- "is there a compelling reason why I want to deviate from this suggestion and have no default for the left argument?" (if so, fine). By the way, an empty vector (' ') is a suitable substitute for a missing left argument in implementations without ambivalent functions. I will propose in the ASWI committee to mention the alternative of ' ' in the next version.
- "if I have two types of information — one processed data and the other control information which steers the function — is there a compelling reason not to distinguish those types and not to present the controlling information in the left and the processed data in the right argument?" If there is a compelling reason to deviate, then fine again.

I hope to have clarified most of the points in the ASWI proposal to which Ray objects. We do indeed think that the ASWI proposals aid readability and maintainability. The recipes are not as ephemeral and hypothetical as Ray probably thought. Most of them were derived from our own everyday experience. As to Ray's suggestion of applying them to see if they help or hinder, we can assure him that we will do so, and have done so.

Ray's comments have been worthwhile contributions to improve our standards with regard to:

- giving a little bit more leeway to omit a prefix *SID* in user-functions.
- reconsidering the *SID* in standard functions.
- mentioning underscored characters as the alternative to lower-case characters.
- mentioning the ' ' -alternative to missing left arguments.

F.H.D. van Batenburg,
Leyden University Institute for Theoretical Biology,
Kaiserstraat 63, Leiden,
Holland

APL*PLUS II Keyboard Configuration

by Ajay Askoolum

Background

The APL keyboard is at once the most visible strength and weakness of the language in so far as its acceptance is concerned. It is generally true that an APL programmer finds the relevant APL symbol without difficulty even when the keycaps are not annotated or engraved with the APL symbols; this is the case on any country mapping supported by DOS. This consistency is a major strength.

The weakness of this regime comes to the fore with APL beginners. The special symbols are alien enough, but the real difficulty is in locating them on a keyboard which does not have the symbols engraved on the keycaps.

The APL-Keycap Challenge

APL*PLUS II allows the keyboard to be fully configured and customised. IBM supply replacement APL keycaps for their 102-key keyboard. Is it possible to use IBM APL keycaps and a reconfigured APL*PLUS II keyboard without conflict with other applications on the PC? Why would you want to find out anyway? The answers depend on personal choice but must surely herald a major potential benefit, a fully-functional UK keyboard.

For the purposes of APL the advantages are obvious: a visible APL keyboard for use in the UK, that is one with engraved APL symbols and with a £ sign in text mode. The novice no longer guesses the location of the special APL symbols as they are visible. And the configuration does not conflict with other packages on the PC.

The reality of the APL*PLUS text keyboard, that is the one invoked with ALT/CAPS LOCK in a session started with K=APL in the CONFIG.APL file, is that it is a **US 101-key keyboard** irrespective of the keyboard configuration in the AUTOEXEC.BAT or CONFIG.SYS start-up files in the PC. Thus SHIFT 3 produces # and not £ and the key immediately to the left of the Z is dead. It does not exist on the US keyboard, hence the difference in the key count of UK and US keyboards.

The pre-requisites for the option of customising the keyboard fully are therefore an IBM 102-key UK keyboard and DOS version 4 or later.

Replacing the Keycaps

It is easy to replace the existing keycaps with the IBM APL keycaps, particularly as there is a tool supplied to do the job. Having done that, you find that some of the keycap characters no longer correspond to the characters actually obtained on depressing the key. For instance, SHIFT 8 does not produce (but * etc. This is resolved by adding the following line to the CONFIG.SYS file and rebooting the PC:

```
INSTALL=C:\DOS\KEYB.COM UK,437,C:\DOS\KEYBOARD.SYS /ID:168
```

This will have the effect of replacing the default keyboard (ID 166) with the alternative layout (ID 168).

In order to make it easy to refer to specific keys on the keyboard, I have used the notation [row;column], where 'row' is the keyboard row starting from the top, and 'column' is the key number in that row counting from the left. The Esc key is therefore [1;1], the BACKSPACE key is [2;14] and so on.

At the DOS prompt and with the APL keycaps on, the omissions are:

1. the key [3;13] does not have the [symbol engraved
2. the SHIFT key [2;1] does not produce | as engraved but █
3. the [5;2] key does not have the correct engravings but has | and _

These would not be problems if the keyboard had ID-168 keycaps to start with. As it is, the above list is not daunting to remember in comparison with the task of remembering the location of all the APL symbols.

Another disparity exists between IBM APL and APL*PLUS II. The → symbol is on shift [3;12] in IBM APL and is UNSHIFTED [3;13] in APL*PLUS, and ␣ is on alt [3;13] in IBM APL and APL*PLUS has the symbol ⍳ on that key. These can be remapped, and other keys such as ⍒ and ≠ can be put on the keyboard where other symbols are not assigned.

Remapping the APL*PLUS Keyboard

The actual method of the keyboard remapping technique is discussed in chapter 11 of the User Manual for Version 4. The essential concepts to grasp are:

- a) key button number

Appendix 20 of the Reference Manual of Version 4 provides the relevant reference. This is essentially the scan code associated with each key on the keyboard which is recognised by the BIOS routines in the PC. Thus it is

hardware dependent, and cannot be changed. However, for the UK keyboard there is one addition, key button number 42. The table for the typewriter keys is as follows:

KEYBUTTON NUMBERS FOR UK 102 KEY KEYBOARD (ID 168 LAYOUT)

\	1	2	3	4	5	6	7	8	9	0	-	_
1	2	3	4	5	6	7	8	9	10	11	12	13
Q	W	E	R	T	Y	U	I	O	P	@	[	
17	18	19	20	21	22	23	24	25	26	27	28	
A	S	D	F	G	H	J	K	L	;	:	]	
31	32	33	34	35	36	37	38	39	40	41	42	
Z	X	C	V	B	N	M	,	.	/			
46	47	48	49	50	51	52	53	54	55			

b) raw shift state numbers

Chapter 11 of the User Manual of Version 4 provides the relevant reference. In brief, the table is:

RAW SHIFT STATE TABLE	
RAW SHIFT STATE	DEFAULT SHIFT KEY
0	No Shift
1	Shift
2	Ctrl
4	Alt

Note that the raw shift state numbers are 0 and powers of 2 and that combinations of raw shift state such as ALT+CTRL i.e. $2 + 4$ produces a raw shift state of 6. However, SHIFT+SHIFT produces 1, that is only unique raw shift state numbers are recognised in any combined keystroke.

c) effective shift states

Again, Chapter 11 of the User Manual of Version 4 provides the relevant reference. The table of valid combinations is:

EFFECTIVE SHIFT STATE TABLE		
RAW SHIFT STATE	APL Keyboard	Text Keyboard
0 no shift	0	1
1 shift	2	3
2 ctrl	4	4
3 ctrl+shift	9	9
4 alt	5	6
5 alt+shift	7	8
6 alt+ctrl	10	10
7 alt+ctrl+shift	4	4

d) the ASCII table or $\square AV$ atomic vector

Each character is associated with a unique number from 0 to 255, in index origin zero. The associated number with any character can be given by $(-\sim\square IO) + \square AV_1 \text{ character}$ e.g. $(-\sim\square IO) + \square AV_1 '\epsilon'$ returns 3.

Thus, in order to assign £ to SHIFT 3 on the text keyboard:

- establish the keybutton number (KN) of key {2;4} i.e. 4
- establish the raw shift state i.e. 1
- look up the effective shift state (ESS) i.e. 3
- find the ZERO ORIGIN index of £ in $\square AV$ i.e. 156

The next step is to combine this information to produce a keystroke number as follows $1000 \perp KN, ESS$. Thus for the example $1000 \perp 3 \ 4$ produces 3004. The final step is to add the line `KEY 3004 = 156` to the CONFIG.APL file. On the next invocation of APL*PLUS II, £ will be on the text keyboard: try ALT+CAPSLOCK, then SHIFT + 3. Some examples of reassignments are:

```
KEY 2007 = 241 ◊ a put Δ on alt key [3;16]
KEY 2027 = 26 ◊ a put + on shift key [3;12] as in IBM APL
KEY 42   = 3 ◊ a put ε ON KEY [4;13]
```

In the re-mapped keyboard, it would be useful (for screen design) to have the line-drawing symbols on the ALT key combination in APL*PLUS in text mode.

One scheme for these could be:

TEXT MODE ALT/KEYSTROKE CHARACTERS FOR UK 102 KEY KEYBOARD (ID 168 LAYOUT)

\	1	2	3	4	5	6	7	8	9	0	-	~
Q	W	E	R	T	Y	U	I	O	P	@	[	
A	S	D	F	G	H	J	K	L	;	:	]	
Z	X	C	V	B	N	M	,	.	/			

The keybutton numbers do not change. The zero origin indices of the characters | | etc. are 176 177 178 179 180 etc. respectively. The effective shift state is 6 and the keybutton numbers are as on the table above. Thus, the lines to be added to CONFIG.APL are:

```
KEY 6001= 176
KEY 6002= 177
KEY 6003= 178
KEY 6004= 179
KEY 6005= 180
etc.
```

Function Keys

A further aspect of customisation deals with function keys. By default, the function keys are dead, that is they produce no action. Moreover, the event numbers associated with F1... are 601 ... etc. and these events do not cause an exit from `WIN`, and `^1 WKEY 601` produces *DOMAIN ERROR* and therefore cannot be defined as an exit key.

It would be useful to select events to associate with the function keys, such as F1 to mean help, F7 to mean previous page, F8 to mean next page etc. These would be useful for instance when navigating the ring editor. If the event numbers associated are within the range of `WKEY`, all the events could be set to cause an exit from `WIN`.

For example, event number 499 (edit object at cursor) could be associated with F2 with the line `PFKEY 2=499` in the CONFIG.APL file, and then `^1 WKEY 499` will set up F2 as an exit key. Conceivably, the event numbers associated with the

function keys could vary from workspace to workspace thereby creating a problem for any `WIN` handling functions. This can be resolved as follows:

```
RES+WIN FieldWindows o a Trap WIN results
DEF+cPFKEY "0 100 200..+(-IO)+:12 o a Get defs of fn keys
PFK+(DEFcRES[IO+1])/,600 700 800..+(-IO)+:12 o a Fk event no
```

Thus no matter what the current event number of the function keys, the `WIN` routines would always use the event numbers associated with the function keys.

Ironically, this facility for redesigning the keyboard mapping for APL*PLUS II sessions can be abused to the extent that any symbol can be placed on any keybutton number, thereby adding to rather than removing the chaos.

Perhaps STSC should supply UK keyboard configurations for both ID 166 and ID 168 together with a consistent set of predefined function keys with sets of engraved APL key caps for established makes of keyboards as an option. This must be more convenient than the previous practice of supplying character ROMS and expecting users to manage the hardware modification themselves. This implies two `CONFIG.APL` files on disks destined for the UK.

On the issue of `CONFIG.APL`, STSC should supply versions where every single parameter is initialised to a default. Ideally, a robust installation routine should change the defaults automatically. This would avoid some bugs slipping through (`KEYLOG` and `KEYSRC` cannot be initialised in `CONFIG.APL` with version 4.0.03) and would prevent enigmatic errors which take a lot of valuable time to debug. In my case, I spent a whole day establishing why a `)COPY WS OBJECT` was generating a `WS NAME ERROR` message. The answer? My `CONFIG.APL` had specified a `tempdir=` pointing to a subdirectory which did not exist.

References

- [1] APL*PLUS II Version 4 Reference Manual
- [2] APL*PLUS II Version 4 User Manual
- [3] *Keyboard Layouts and Code Pages* (IBM DOS Version 5.0)
- [4] APL*PLUS/PC Version 6.0 Reference Manual (re internal tables defining keyboard interpretation)

Integrating with Insight

by Norman Thomson

This article was triggered by the function for Romberg integration given by C.E. Williams in Education Vector 9.1 with a note expressing hope for an improvement — well let's try!

First Williams' comparison of Romberg integration with the Simpson Rule is not one of like with like — Simpson's Rule and the Trapezium rule on which Williams' algorithm is based, are base integration techniques. Romberg's method by contrast is a refinement technique which can be exercised on one or other of the base methods. In fact Romberg with one iteration is Simpson's Rule, so the former is necessarily slower provided there is more than one iteration.

The refinement technique commences every time a new base integration is performed and consists of taking a scalar s which will in fact be the result of the new base integral evaluation and a vector v which will be the sequence of refinement steps since the last base integration and constructing a result vector r where

```
r[1] = s
r[2] = r[1] + k[1].(r[1]-v[1])
r[3] = r[2] + k[2].(r[2]-v[2])
...
```

and so on, a process somewhat like (but not directly translatable into) that for evaluating $s \downarrow v$. The quantities $k[i]$ are a sequence of constants — $1/3, 1/15, 1/63, \dots$ if the base technique is the Trapezium rule, $1/15, 1/127, 1/4096, \dots$ for the Simpson Rule. We are thus now in a position to define the refinement procedure by a function *REFINE*:

```
[0]   r←v REFINE s
[1]   a global variable Q = 1(Trap),2(Simpson)
[2]   →L1 IF 0=ρv←,v
[3]   r←, (1+v)REFINE s
[4]   →0 r←r, (1+r)+(1+r-v)÷1+2*2×Q×p r
[5]   L1:r←s
```

The function *REFINE* uses only ISO-APL functions. From this point however the power of APL2 to define operators is invaluable for the purpose of identifying the algorithmic structure.

The next step is to define the base methods themselves. The algorithms are so similar that it is sensible to combine them in a single operator *METHOD* whose second operand is 1 for Trapezium, 2 for Simpson:

```
[0] Z←L(P METHOD Q)R;T
[1]  a L=no. intvs, P=integrand, Q=1(Trap),2(Simp), R=range of integr'n
[2]  Z←T×/(+Q+1)×(1,((L-1)P(-Q)+4 2),1)×P"(+R)+(T←R-÷L)×0,÷L
```

The minimum number of intervals is 1 for Trapezium, 2 for Simpson.

Now the Romberg process can be described in terms of the base method *METHOD* and the refinement technique *REFINE*:

```
[0] Z←N(P ROMB Q)R
[1]  a N=no of iter'ns, P=integrand, Q=1(trap),2(Simp), R=range of int'n
[2]  →L1 IF N=0
[3]  →0 Z←((N-1)P ROMB Q R)REFINE(2×N)(P METHOD Q)R
[4]  L1:Z←1 P METHOD Q R
```

I prefer to identify a new iteration as starting with each invocation of the base technique which is the sense in which *N* is used in my *ROMB*. Williams considers each item in a new "*REFINE*" vector (what I call a refinement step) as an iteration so that 10 of his iterations are equivalent to 4 of mine or more generally $\frac{1}{2}n(n+1)$ of his are equivalent to *n* of mine. The operator *ROMB* returns a base integration together with the complete set of refinement steps which follow it, so taking the function which Williams uses as an example:

$$\int_1^2 \sqrt{x^2 + 3x - 4} \, dx$$

define the APL equivalent function:

```
[0] Z←FN X
[1] Z←(X÷1 3 -4)×0.5
```

and the numerical equivalence of his *ROMB* can be confirmed by:

```
4 FN ROMB 1 1 2
1.570382324 1.574352706 1.574697433 1.574769562 1.574786867
```

The relative execution performance of my *ROMB* and Williams' function depend on the integrand. Over a range of functions those for my *ROMB* are a small fraction of his — in most cases under 10%.

Rather than compute a given number of iterations, it is in practice more usual to seek an integration procedure which terminates when two iterates agree to a specified degree of precision *EPS*. An appropriate operator for this purpose is:

```
[0]   Z←EPS(P ROMBERG Q)R;T;N
[1]   ⍝ P is integrand, Q = 1 (trap), 2 (Simpson), R = range of integration
[2]   Z←T+(1+N←0)P METHOD Q R
[3]   L1:Z←(T←1+Z)REFINE(2*N+N+1)(P METHOD Q)R
[4]   →L1 IF EPSL<T←1+Z
[5]   Z←1+Z
```

To count iterations insert a counter-incrementing statement between lines 3 and 4 of *ROMB*.

ROMBERG is not the only possible refinement technique. In essence it doubles the number of intervals at each integration whereas it may be the case that adequate precision has already been achieved in one half of the interval. An alternative refinement technique is so-called *adaptive* integration which divides the tolerance *EPS* equally between the two halves of the range, and tests each separately, proceeding only in that half or halves where further precision is necessary. This refinement technique is described by:

```
[0]   Z←EPS(P ADAPT Q)R
[1]   ⍝ P is integrand, Q = 1(trap),2(Simp), R = range of integrn
[2]   Z←(2*Q)(P METHOD Q)R
[3]   →0 IF EPSN<Z-Q(P METHOD Q)R
[4]   Z←+/(0.5*EPS)(P ADAPT Q)"0 1φ"R,"0.5*+R
```

The value of having several alternative techniques at one's disposal is illustrated by integrating the following "difficult" function:

$$\int_{-1}^1 (.46(e^x + e^{-x}) - \cos x) dx$$

whose APL rendition is:

```
[0]   Z←GN X
[1]   Z←(0.46*+/*X,-X)-2○X
```

The Simpson rule (method 2) with 2 and 4 intervals gives results which agree to 4 decimal places:

```
2 GN METHOD 2 -1 10.4795546
4 GN METHOD 2 -1 10.4795551
```

which is confirmed by using *ADAPT* with Simpson which converges immediately:

```
.000001 GN ADAPT 2 -1 1
0.4795551
```

However integrating to a precision of 5 decimal places using *ROMBERG* shows that this answer is wrong:

```
.000001 GN ROMBERG 2 -1 1
0.4794283
```

The above integration required 6 iterations. Using the Trapezium rule as base method to obtain the same precision

```
.000001 GN ROMBERG 1 -1 1
0.4794283
```

requires 14 iterations. *ADAPT* with the Trapezium rule converges to the correct answer following 1023 interval subdivisions, that is 2047 separate calls to *ADAPT*.

```
.000001 GN ADAPT 1 -1 1
0.4794285
```

To summarise, compared with Williams' function the number of APL lines has been reduced from 30 to 12, four techniques are available instead of one, execution times have been drastically cut, and, most important of all in my view, the APL code provides an insightful description of the true nature of the methods by separating them into their essential stages of approximation and refinement. For the practically minded, five minutes typing these 12 lines provides a remarkably comprehensive integration package — worth a year's subscription to *Vector* in itself! I think I can modestly claim an improvement.

Some J Verbs for Orthogonal Factorial Experiments

by Keith Smillie

Abstract

Verbs are developed in J for analysis-of-variance calculations for factorial experiments with an arbitrary number of factors and with either a complete model with all main effects and interactions or a model with an arbitrary selection of main effects and interactions.

Analysis-of-Variance Computations

The main computational problem in the analysis of variance is the partition of the total variation of a dependent variable as measured by the sum of squares of deviations from the arithmetic mean into a number of orthogonal components for each of several main effects and their interactions. If the data are considered to be arranged in a rectangular array with an axis for each factor and one for replications, then a first step in the partitioning process is the calculation of some, or possibly all, of the marginal totals of various subarrays. For example, for a one-factor design the data may be arranged in a table, and the four marginal totals consist of the row totals, the column totals, the grand total, and the items of the array. In general, for an n -dimensional rectangular array there are 2^n marginal totals. For each set of marginal totals a weighted sum of squares is found by squaring each item, summing and dividing by the number of items occurring in each total. From these weighted sums of squares the required analysis-of-variance table may be found relatively simply.

To illustrate the computational procedure to be used in this paper we shall consider a three-factor design with the factors designated by A, B and C so that the data may be considered arranged in a three-dimensional array with any one axis representing replications. Then let T_a represent the marginal totals obtained by summing over the second and third axes, T_{ab} the margins obtained by summing over the third axis, and so on. To each set of these marginal totals — there are eight in a three-dimensional array — there corresponds a weighted sum of squares, and, for example, S_{ab} represents that one obtained by squaring each item of T_{ab} , summing and dividing by the number of items along the third axis.

Then the sum of squares associated with the main effect for factor A may be shown to be equal to

$$SS_a = S_a - S,$$

where S is the weighted sum of squares obtained by summing all of the observations and dividing by their total number. Similar expressions hold for the sums of squares for the B and C main effects, SS_b and SS_c , respectively. The sum of squares for the AB interaction is equal to

$$SS_{ab} = S_{ab} - (S_a + S_b) + S,$$

with similar expressions for SS_{ac} and SS_{bc} . The sum of squares for the ABC interaction is

$$SS_{abc} = S_{abc} - (S_{ab} + S_{ac} + S_{bc}) + (S_a + S_b + S_c) - S.$$

Expressions for the sum of squares for any main effect or interaction term for an arbitrary design may be written down in a similar manner.

The relationship between the subscripts for any sum of squares and the corresponding weighted sums of squares required for its calculation should be noted. For example, for the ABC interaction of the last paragraph the subscripts for the weighted sums of squares may be represented by {abc}, {ab}, {ac}, {bc}, {a}, {b}, {c} and {} which is the power set of the set {abc} of subscripts designating the interaction. Also the sign associated with each weighted sum of squares depends on the number of factors appearing in its subscript, in this example a positive sign if the number of subscripts is odd and a negative sign if the number is zero or even. The required sum of squares is the sum of these signed weighted sums of squares.

The above discussion suggests the following sequence of calculations for finding the sum of squares for an arbitrary main effect or interaction:

- Compute the weighted sums of squares for each member of the power set of the specified subscripts for the main effect or interaction with unspecified positions in the subscripts indicating axes to be summed over.
- Group the weighted sums of squares according to the number of factors appearing in the subscripts and in ascending or descending order of number of factors, and find the sum of the terms in each group.
- Compute the alternating sum of the sums found in the previous step. This gives the required sum of squares apart from the sign which may be accounted for by taking the absolute value.

The development of the verbs in J will be presented in three parts: definition of some preliminary verbs required in the main calculations, verbs for the main calculations discussed in the last paragraph, and a final verb which uses these verbs to assemble the analysis-of-variance table. All of the verbs and the sample data used are given in a listing of the script file in Figure 2.

Some Preliminary Verbs

First we shall define the truth table verb

```
tt=. ,@({]@#2(<0 1))
```

which gives all boolean arguments of the truth table whose size is specified by its argument, so that, for example, `tt 2` is:

```
+---+---+---+---+
|0 0|0 1|1 0|1 1|
+---+---+---+---+
```

and `tt 3` is

```
+---+---+---+---+---+---+---+---+---+
|0 0 0|0 0 1|0 1 0|0 1 1|1 0 0|1 0 1|1 1 0|1 1 1|
+---+---+---+---+---+---+---+---+
```

We shall also define the sorted truth table verb

```
stt=. <"1@|. "1@(/: +/"1)@(>@tt)
```

which sorts the items in ascending order of the number of 1s each contains, and `stt 3` is:

```
+---+---+---+---+---+---+---+---+---+
|0 0 0|1 0 0|0 1 0|0 0 1|1 1 0|1 0 1|0 1 1|1 1 1|
+---+---+---+---+---+---+---+---+
```

The main effects and interactions will be labelled in the analysis-of-variance table by appropriate upper case letters but represented internally in binary form, so that, for example, in the three-factor design discussed in the last section the AB interaction will be represented externally as AB and internally as 1 1 0. To simplify the conversion between representations we shall define the verbs

```
tag=. ]#({.&alphabet)@#@]
```

where

```
alphabet=. 'ABCDEF'
```

and

```
numtag=. (((.&alphabet)@(#$@])) e. [
```

and, for example, tag 1 1 0 is AB and 'AB' numtag TestData, where TestData is a three-dimensional array to be used later in the paper, is 1 1 0. We shall also define the verb

```
alltags=. (tag 2. >.)@(12.).)@stt
```

which gives all the labels for a factorial design of any size, and, for example, alltags 3 is

```
+-----+
|A|B|C|AB|AC|BC|ABC|
+-----+
```

Finally we shall define the verb

```
ps=. <"1@expand"1 >@stt@(+/) ,
```

where

```
expand=. /:@\:@[#{#@[{.}]
```

has been suggested by Kenneth Iverson and is given in McIntyre (1991), to find the power set of the subscripts corresponding to a main effect or interaction. As an example, ps 1 1 0 has the value

```
+-----+
|0 0 0|1 0 0|0 1 0|1 1 0|
+-----+
```

which indicates that the calculation of the sum of squares SS_{ab} requires the weighted sums of squares S , S_a , S_b and SS_{ab} .

Basic Calculations

The calculation of any set of marginal totals in an arbitrary array may be accomplished by first transposing the array so that the axes to be summed over occur first and then summing over these axes. We shall specify the axes by a boolean list of length equal to the rank of the array with 0s indicating the axes being summed over. For example, consider summing over the first, third and fourth axes of an array a , where $\$a$ is 2 3 4 5, say, so that the boolean list specifying the summation is 0 1 0 0. Since $/:0 1 0 0$ is equal to 0 2 3 1, the transposition may be accomplished by the expression $0 2 3 1|:a$ which gives an array of shape 2 4 5 3.

Application of the verb $+/$ three times, the number of 0s in the boolean list specifying the marginal totals, to this array will give the desired marginal totals. These calculations may be accomplished simply by the explicit verb

```
T=. ' ' : '+/^(+/-x.)(/:x.)|:y.'
```

where the right argument gives the array and the left argument specifies the marginal totals required. Then the above example is given by `0 1 0 0 T a`. The weighted sum of squares is given by

```
S=. (+/@((*:@,.)@T)) % (*/@(-.@[ # $@]))
```

where the arguments are the same as for `T`. As an example of the use of the above verbs consider the three-by-four table `x` given by

```
4 7 5 6
9 4 3 8
2 5 7 3
```

Then `0 0 T x` is the sum 63 of the items of `x`, `1 0 T x` is the row sums 22 24 17, `0 1 T x` is the column sums 15 16 15 17, and `1 1 T x` is the array `x`. The corresponding values of the weighted sums of squares given by `S` are 330.75, 337.25, 331.667 and 383, respectively.

A verb for calculating the sum of squares for a main effect or interaction may now be defined with a little thought as

```
ss=. | @ (-/) @ (>@ps@[numtag]) (+/"1@[ +/. S"1 _ ])
```

and, for example, `'A' ss x` is 6.5 and `'AB' ss x` is 44.8333. If we define the verb

```
df=. */@<:@(numtag # $@)
```

for the degrees of freedom for any term, then the verb

```
term=. ss [ ] , [ , %) df
```

will give a three-item list of degrees of freedom, sum of squares and mean square for any one term. For example, `'A' term x` is 2 6.5 3.25 and `'AB' term x` is 6 44.8333 7.47222.

An arbitrary number of terms may be calculated by the verb

```
allterms=. (>@[ ] term"1 _ ]
```

and `('A';, 'AB') allterms x` is the table

```
2      6.5      3.25
6  44.8333  7.47222
```

The Analysis-of-variance Verb AOV

It is a relatively simple matter to use these verbs to define an explicit verb AOV that will assemble and display an analysis-of-variance table for an arbitrary model and rectangular array of data.

Figure 1 gives a listing of AOV and two examples of its use. We note that the right argument gives the array and the left argument specifies the model. A monadic use of the function with the data as the single argument gives an analysis of variance for a full model with all main effects and interactions of all orders. The result of the verb is the analysis-of-variance table giving an appropriately labelled row with the degrees of freedom, sum of squares and mean square for each term, an error term if there is one, and a total. The numeric part of the table is also given as the global pronoun AOVtable.

Keith Smillie
Department of Computing Science
University of Alberta
Edmonton, Alberta T6G 2H1

References

- [1] Bennett, C. A. and N. L. Franklin, 1954. *Statistical Analysis in Chemistry and the Chemical Industry*. John Wiley and Sons, Inc., New York.
- [2] McIntyre, D. B., 1991. "Mastering J". APL91 Conference Proceedings, Palo Alto, California, pp. 264 - 273.

AOV

```

{alltags # $y.) AOV y, | : AOVtable=. x. allterms y.
| Labels=. 8{. "1 >x.
| dfTotal=. <:*/$y.
| ssTotal=. (+/*:,y.) - '' ss y.
| dfError=. dfTotal - +/0{"1 AOVtable
| $.=. > (dfError=0) { (<6 7 8 9 10 11),<9 10 11
| ssError=. ssTotal - +/1{"1 AOVtable
| AOVtable=. AOVtable,ssError[1,%.%dfError
| Labels=. Labels,'Error'
| r=. Labels,"1 (5 12.5 12.5)":AOVtable
| AOVtable=. AOVtable,TotalRow=. dfTotal,ssTotal
| r, 'Total ' ,5 12.5":TotalRow

```

```

TestData
25 7 21 4 10 16
5 21 4 25 7 6
3 6 16 18 20 18
19 17 16 2 15 6
13 23 8 25 19 19
20 14 23 9 10 18
27 26 19 24 13 9
19 13 20 18 16 12

```

```

$TestData
2 4 6

```

```

AOV TestData
A      1 252.08333 252.08333
B      3 69.16667 23.05556
C      5 74.66667 14.93333
AB     3 10.41667 3.47222
AC     5 51.91667 10.38333
BC    15 309.83333 20.65556
ABC    15 1495.58333 99.70556
Total  47 2263.66667

```

```

('A';'B';'AB';'C') AOV TestData
A      1 252.08333 252.08333
B      3 69.16667 23.05556
AB     3 10.41667 3.47222
C      5 74.66667 14.93333
Error  35 1857.33333 53.06667
Total  47 2263.66667

```

```

AOVtable
1 252.083 252.083
3 69.1667 23.0556
3 10.4167 3.47222
5 74.6667 14.9333
35 1857.33 53.0667
47 2263.67 0

```

Figure 1. The verb AOV.

```

tt=. ,@{@[#&(<0 1))
stt=. <"1@|."1@(/: +/"1)@(>@tt)

alphabet=. 'ABCDEF'
tag=. ]#({.&alphabet)@#@[
numtag=. ({.&alphabet)@({#&@[}) e. [
alltags=. (tag &. >)@({1&}.)@stt

expand=. /: @\:@[#{@[{. ]
ps=. <"1@expand"1 >@stt@(/)

T=. ' ' : '+/^(+/-X.)(/:X.)}:y.'
S=. (+/@((*:@.)@T)) % (*/@(-.@[ # $@])
ss=. | @ (-/) @ (>@ps@([numtag]) (+/"1@[ +/|. S"1 _ ]])
df=. */@<:([numtag # $@])

term=. ss [ ] , [ . %] df
allterms=. (>@[ ] term"1 _ ]

s0=. '(alltags # $y.) AOV y.'
t0=. 'AOVtable=. x. allterms y.'
t1=. 'Labels=. 8{."1 >x.'
t2=. 'dfTotal=. <:*/$y.'
t3=. 'ssTotal=. (+/*:y.) - '''' ss y.'
t4=. 'dfError=. dfTotal - +/0{"1 AOVtable'
t5=. '$=. > (dfError=0) { (<6 7 8 9 10 11),<9 10 11'
t6=. 'ssError=. ssTotal - +/1{"1 AOVtable'
t7=. 'AOVtable=. AOVtable,ssError([,%.%)dfError'
t8=. 'Labels=. Labels,'Error''
t9=. 'r=. Labels,"1 (5 12.5 12.5)":AOVtable'
t10=. 'AOVtable=: AOVtable,TotalRow=. dfTotal,ssTotal'
t11=. 'r, 'Total ' ',5 12.5":TotalRow'
t=. t0;t1;t2;t3;t4;t5;t6;t7;t8;t9;t10;t11

AOV=. s0 : t
x=. 3 4 $ 4 7 5 6 9 4 3 8 2 5 7 3

TestData=. 25 7 21 4 10 16 5 21 4 25 7 6
TestData=. TestData, 3 6 16 18 20 18 19 17 16 2 15 6
TestData=. TestData, 13 23 8 25 19 19 20 14 23 9 10 18
TestData=. TestData, 27 26 19 24 13 9 19 13 20 18 16 12
TestData=. 2 4 6 $ TestData

```

Figure 2. Script file.

Jacobi's Method for Eigenvalues: an Illustration of J

by Donald McIntyre

Introduction

J is Iverson's powerful new dialect of APL available for a wide range of computers [1, 2]. I have already given examples of J, explaining some of its interesting syntax with reference to applications ranging from elementary arithmetic to system programming and the construction of a database [3-5]. The conciseness of the notation enables many quite complicated operations to be performed by entering a few short lines, in the manner of Iverson's Direct Definition in older APL [6, 7]. In this paper I take an example of a problem that must be solved iteratively. It requires a program of several lines with conditional branching.

Eigenvalues and Eigenvectors

The inner product of a matrix and a vector is another vector, which in general will differ in length and orientation. If there is no change in orientation, the vector is called an *eigenvector* of the matrix. In this case the ratio of the length of the new vector to the length of the original one is an *eigenvalue* (originally called a *latent* or *characteristic root*) of the matrix. Eigenvalues can be complex numbers, but the eigenvalues of a symmetric matrix of real numbers are always real. Jacobi's method is used to compute the eigenvectors and eigenvalues of such matrices.

The number of linearly independent rows or columns of a matrix (for these are equal) is called the *rank* of the matrix. This is not to be confused with the use of *rank* in other contexts, such as denoting the shape of the shape of an array. The rank of the inner product of two matrices cannot exceed the rank of either matrix from which it is produced. In particular, when a matrix is multiplied by its own transpose the product has the same rank as the original matrix. Because the product of a matrix with its own transpose is necessarily symmetric, Jacobi's method can be used. The rank of a matrix equals the number of non-zero eigenvalues, consequently the Jacobi solution measures the amount of independent information in the data.

When you hold up your open hand your fingers and thumb approximate to 5 radiating vectors in 3-dimensional space. Their orientation could be described by a matrix of 5 rows (items) and 3 columns. Multiply the matrix by its transpose to get a 3 by 3 product. It has 3 eigenvalues, and one of them is close to zero because the vectors lie nearly in one plane. If the matrix represents five students each taking 3 tests, then we know that the tests do not provide independent information.

Jacobi's method can be used with geometrical data (as in structural geology) or data represented in an imaginary multi-dimensional space (as in educational tests or chemical analyses).

Jacobi's Method

Vectors lying in a plane are rotated through the angle γ by multiplying them by the 2 by 2 rotation matrix:

$\cos \gamma$	$-\sin \gamma$
$\sin \gamma$	$\cos \gamma$

Rotations in multi-dimensional space are performed by a modified identity matrix in which the elements in the cells (i,i) , (i,j) , (j,i) , and (j,j) correspond to those in the 2 by 2 rotation matrix.

The method is based on the fact that eigenvalues remain unchanged on rotation. A sequence of rotations is performed, each time chosen so as to reduce to zero the largest absolute off-diagonal value (the *pivot*). The result is a matrix with eigenvalues on the diagonal and zeros in all other positions. When the same sequence of rotations is performed on an identity matrix, the columns of the final result are the eigenvectors of the original matrix.

Procedure

To write the Jacobi algorithm in J we must first define some basic operations. We must be able to *amend* a matrix, based on the position and value of the pivot. Using a matrix of random numbers as a partial example (even although it is not symmetric), place the numbers 996 997 998 999 in the required positions:

```

      setrl=. 9! :1
      setrl 7^5
      ]y=. 50~ 76 6$100
      _37 25 _5 3 _29 _46
      _17 17 43 _12 1 33
      _47 _45 2 17 _50 _12
      _44 _9 18 8 43 34
      2 _41 15 _9 20 41
      26 _24 _46 23 _18 13

```

NB. Set Random Link

Define the following verbs:

```

      ut=. ,@(</~@i.)@#
      pt=. (, i. >./@ (ut # ,))@|
      pm=. <.@ (pt % #) , # | pt
      pv=. {~ <@pm
      pa=. 0 0&{ ; ] ; |. ; 1 1&{
      ia=. pa@pm { i.@$

```

NB. Upper Triangle

NB. Pivot in Triangle

NB. Pivot in Matrix

NB. Pivot

NB. Permutations for Amend

NB. Indices for Amend

The value of the pivot (in the upper triangle) is:

```

      pv y
      _50

```

With respect to the ravel of *y*, the linear indexes of cells to be amended are:

```

      ia y
      14 16 26 28

```

Identification of the cells that are to be amended is a function of *y*

```

      amend=. ia@]

```

Note that a noun (produced by applying *ia* to *y*) and an adverb (}) give a noun.

```

      996 997 998 999 amend y
      _37 25 _5 3 _29 _46
      _17 17 43 _12 1 33
      _47 _45 996 17 997 _12
      _44 _9 18 8 43 34
      2 _41 998 _9 999 41
      26 _24 _46 23 _18 13

```

Define the following verbs:

ip=. +/ .*	NB. inner product
readkb=. ".@(1:1@1:)	NB. read from keyboard
write=. 11:2&2	NB. write to screen
diag=. (<0 1)& :	NB. matrix diagonal
uf=. -:@-/@(pm(diag))	NB. u function
rss=. %:@+/@*:	NB. sq rt of sum of sqs
vf=. rss@(pv,uf)	NB. v function
sign=. >:&0 - <&0	NB. sign of the sine
cosf=. %:@((vf + @uf) % +:@vf)	NB. cosine function
sinf=. sign@uf * -@pv % +:@(vf * cosf)	NB. sine function

The Jacobi Program

Now define and display the dyadic function `jacobi` whose left argument is the tolerance allowed for convergence. The statements are boxed. Names ending with a right parenthesis are labels (a feature not implemented in February 1991 when the program was originally written); and `$.` is the suite, i.e. the vector controlling the sequence of execution [2].

The maximum number of iterations is set as a global variable. Should this limit be reached, an opportunity is given to permit further iterations.

```
maxit=. 20
s0=. <'it=. 0 [ I=. Q=. =/~ i.#R=. y.'
s1=. <'loop) $.=. >(end;$.){~ x.<|p=. -pv R'
s2=. <'v=. %: +/~p,u=. -: -/ (pm R){ diag R'
s3=. <'sin=. (sign u)*p%+:v* cos=. %: (v+|u)%+:v'
s4=. <'$.=. >(end;$.){~ */0~:-.-|sin,cos'
s5=. <'r=. ((cos,-sin),sin,cos) (ia R)} I'
s6=. <'Q=. Q ip |:r [ R=. r ip R ip |:r'
s7=. <'$.=. >(loop;$.){~ maxit<it=. it+1'
s8=. <'write 'Number of further iterations or 0?''
s9=. <'$.=. >(loop;end){~ maxit<it=. it-readkb 1'
s10=. <'end) R,:Q'
jacobi=. ' ' : (s0,s1,s2,s3,s4,s5,s6,s7,s8,s9,s10)
```

`jacobi`

<pre>: it=. 0 [I=. Q=. =/~ i.#R=. y. loop) \$.=. >(end;\$.){~ x.< p=. -pv R v=. %: +/~p,u=. -: -/ (pm R){ diag R sin=. (sign u)*p%+:v* cos=. %: (v+ u)%+:v \$.=. >(end;\$.){~ */0~:-.- sin,cos r=. ((cos,-sin),sin,cos) (ia R)} I Q=. Q ip :r [R=. r ip R ip :r \$.=. >(loop;\$.){~ maxit<it=. it+1 write 'Number of further iterations or 0?' \$.=. >(loop;end){~ maxit<it=. it-readkb 1 end) R,:Q</pre>
--

Notice how the line which defines *r* amends by placing the four rotation values in the matrix. The line defining *Q* shows how 3 inner products give new values for *Q* and *R*. *R* begins as the original matrix and ends as the matrix with the eigenvalues on its diagonal; *Q* begins as the identity matrix and ends as the matrix whose columns are the eigenvectors.

Using Jacobi

It will be convenient to have the following utilities:

<code>clean=.] * (<:)</code>	NB. Sets small values to 0
<code>rfd=. %&180@o.</code>	NB. Radians From Degrees
<code>dfr=. rfd^:_1</code>	NB. Degrees From Radians
<code>arcos=. dfr@(_2&o.)</code>	NB. Arcosine in degrees
<code>round=. [* <.(0.5&+@(%~))</code>	NB. Tolerance is on left
<code>smatrix=. : ip]</code>	NB. symmetric mat: as a Fork
<code>smatrix=. ip~ :</code>	NB. symmetric mat: as a Hook

Test data from a seismological study gave the following Direction Cosines for five coplanar vectors [8, Table 1]. Direction Cosines are the cosines of the angles between a vector and each of three Cartesian axes; they are identical to the coordinates of a unit vector with the same direction.

```

n=. -p=. 0.7071
]table1=. (p,n,0),(n,p,0),(0,p,n),(0,n,p),:n,0,p
0.7071 _0.7071 0
_0.7071 0.7071 0
0 0.7071 _0.7071
0 _0.7071 0.7071
_0.7071 0 0.7071

]m=.0.0001 round smatrix table1
1.5 _1 _0.5
_1 2 _1
_0.5 _1 1.5

]z=. 1e_5 clean 1e_6 jacobi m
0 0 0
0 3 0
0 0 2
0.57735 _0.408248 _0.707107
0.57735 0.816497 0
0.57735 _0.408248 0.707107

```

The eigenvalues are the diagonal elements of the first item; the eigenvectors are the corresponding column vectors of the second item.

We now verify that the results are indeed eigenvectors and eigenvalues. The product of the matrix and an eigenvector equals the product of this vector and the corresponding eigenvalue. Because the computations were carried out with limited tolerance, we cannot use `match (-:)`.

```
m ip 0{"1 (1{z)
1.66533e_15 _2.9976e_15 1.44329e_15
(0{0{0{z) * 0{"1 (1{z)
0 0 0
```

```
m ip 1{"1 (1{z)
_1.22474 2.44949 _1.22474
(1{1{0{z) * 1{"1 (1{z)
_1.22474 2.44949 _1.22474
```

```
m ip 2{"1 (1{z)
_1.41421 _9.8829e_10 1.41421
(2{2{0{z) * 2{"1 (1{z)
_1.41421 0 1.41421
```

These comparisons can be made with a single expression:

```
(1e_9 clean |:m ip 1{z):(diag 0{z) * "0 1 |: 1{z
```

0	0	0	0	0	0
_1.22474	2.44949	_1.22474	_1.22474	2.44949	_1.22474
_1.41421	0	1.41421	_1.41421	0	1.41421

The eigenvectors of a symmetric matrix should be orthogonal. We show that they are:

```
1e_9 clean smatrix 1{z
1 0 0
0 1 0
0 0 1
```

It is convenient to format a sorted summary of eigenvalues and eigenvectors. Being familiar with APL's Direct Definition, but as a beginner in J, I found it easiest to write the verb in explicit definition (the place-holder for the argument is `y.`):

```
evf=. '|: (\:d){"1 (d=. diag 0{y.),1{y.' : ''
```

In this context recognition of the parallelism of `0{y.` and `1{y.` has an advantage.

```

]ez=. evf z
3 _0.408248 0.816497 _0.408248
2 _0.707107 0 0.707107
0 0.57735 0.57735 0.57735

```

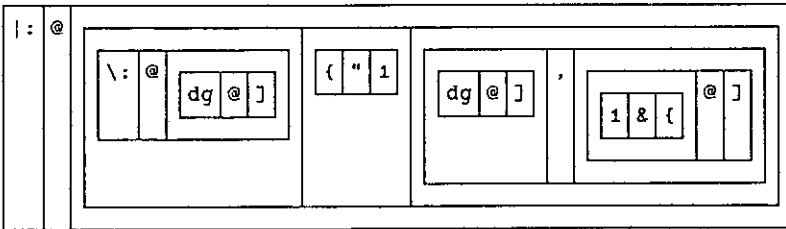
Each item (row) is an eigenvalue followed by the corresponding eigenvector, and the table is sorted by decreasing size of eigenvalue. One eigenvalue is zero, which confirms that the vectors are indeed coplanar. The corresponding eigenvector is the normal to this plane.

We can use this explicit definition to illustrate a way of converting from explicit to tacit definition. First abbreviate by introducing the verb `dg` for the eigenvalues (i.e. the diagonal of the first item). Then apply the adverb `:11` as described in [9].

```

dg=. diag@{.
'|: (\:d){"1 (d=. dg y.),1{y.':11

```



Using this display as a guide, and remembering that conjunctions grab whatever is to their right, we can write:

```

evf=. |: @(\: @dg {"1 dg,1&{)

```

Note that the rank of `1{z` is 2 whereas the rank of `}.z` is 3.

```

ez-: evf z
1

```

If we run again with `maxit=. 6` convergence will not be completed to tolerance `1e_5`. Enter 0 to see the answer at this stage, or 2 to do one more iteration — which will be successful.

The null axes of earthquakes in the Marianas Islands provide a real example [8, Table 2]. The direction cosines are measured from North, East, and Down respectively:

```

t=. _0.75 0.27 0.6, _0.85 0.53 0, _0.63 0.46 0.63,: _0.85 0 0.53
t=.t, _0.09 1 0, _0.17 0.98 0, _0.86 0 0.52,: _0.82 _0.28 0.52
t=.t,0.59 0.79 0.12, _0.59 _0.81 .07, 0.87 0.4 0.28,: .02 .87 0.5
t=.t,0.67 0.7 0.26, _0.66 0.64 0.39, 0.38 .85 .36,:_0.02 .99 0.12
t=.t,0.35 0.44 0.83, _0.88 _0.47 .09, .82 .44 .36,:0.67 0.64 0.37

y=. t, _0.26 0.65 0.73

m=. smatrix y
z=. 1e_5 clean 1e_6 jacobi m

```

The eigenvectors, in descending order of eigenvalues, are:

```

evf z
10.8801 0.501099 0.827536 0.253148
8.32209 0.815533 _0.353717 _0.458028
1.84991 0.289492 _0.435968 0.85213

```

That the results are indeed eigenvectors and eigenvalues is shown by:

```
(1e_9 clean |:m ip 1(z):(diag 0(z) *"0 1 |: 1{z
```

6.78694	_2.94367	_3.81175	6.78694	_2.94367	_3.81175
5.45201	9.00367	2.75428	5.45201	9.00367	2.75428
0.535535	_0.806502	1.57637	0.535535	_0.806503	1.57637

That the eigenvectors are mutually perpendicular is shown by:

```

1e_5 clean arcos smatrix 1{z
0 90 90
90 0 90
90 90 0

```

Fara and Scheidegger said that "the average square of the variance of the sine of the scattering" is 0.088, and hence the "rms-sine scattering angle" is 18. We compute these values as follows:

```

] s2=. (<./ diag 0(z) % #y
0.088091

```

The corresponding angle:

```

dfr _1 0. %: s2
17.2656

```

Acknowledgements

After I lectured on J at St. Andrews University (1 February 1991), Simon Kenyon, a geology student, asked me for help with his study of the relationship between the orientation and shape of pebbles in a sedimentary rock [10]. His need to compute eigenvalues and eigenvectors suggested the problem.

References

- [1] J is available from Iverson Software Inc., 33 Major Street, Toronto, Ontario, Canada M5S 2K9. Phone (416) 925-6096; Fax (416) 488-7559. Also from I-APL Ltd. in the UK (see Product Guide). This paper was submitted in November 1991. The examples, originally executed with J Version 3.3 have been revised for compatibility with Version 5 (27 June 1992).
- [2] Kenneth E. Iverson, *ISI Dictionary of J*, Version 5, Iverson Software Inc., Toronto (1992)
- [3] Donald B. McIntyre, *Hooks and Forks and the Teaching of Elementary Arithmetic*, Vector, Vol.8, No.3 (January 1992) 101-123.
- [4] Donald B. McIntyre, *Using J with External Data: two examples*, Vector, Vol.8, No.4 (April 1992) 97-110.
- [5] Donald B. McIntyre, *Using J's Boxed Arrays*, Vector, Vol.9, No.1 (1992) 92-105.
- [6] Kenneth E. Iverson, *Elementary Analysis*, APL Press, Swarthmore, Pennsylvania (1976) 219pp.
- [7] Donald B. McIntyre, *Experience with Direct Definition One-liners in Writing APL Applications*, Conference Proceedings, APL Users Meeting, Toronto, September 1978. Sponsored by I.P. Sharp Associates, Toronto (1978) p.281-297.
- [8] H.D. Fara & A.E. Scheidegger, *An eigenvalue method for the statistical evaluation of fault plane solutions of earthquakes*, Seismological Society of America, Bull. Vol. 53 (1963) p.811-816.
- [9] Roger K.W. Hui, Kenneth E. Iverson, & Eugene E. McDonnell, *Tacit Definition*, APL91 Conference Proceedings, Stanford, California, August 1991. APL Quote Quad Vol.21 Number 4 (August 1991), p.202-211.
- [10] L.D. Drake, *Till Fabric Control by Clast Shape*, Geological Society of America, Bull. Vol. 85 (1974) p.247-250

AMENDMENT: "A Change for the Better"

(The American Heritage Dictionary of the English Language)

by Donald B. McIntyre

It used to be that whenever the computer centre manager decided to install a new version of the operating system, the FORTRAN compiler, or the APL interpreter, we were forced to interrupt what we were doing in order to adapt to the changed environment, even though we may have been content and comfortable with the old one. With a personal computer we change only when we choose to do so. Moreover, should we want to, we can have several versions of the software immediately available on the hard disk; I can, for example, call up any one of ten different versions of Iverson and Hui's *J* interpreter. All perform as originally specified and each has with it several executable script files. It is helpful that *J* comes with two files (*status.doc* and *xenos.doc*) documenting the changes from all earlier versions.

Amend a Table

Because my paper on *J*'s Boxed Arrays [1] was based on *J* Version 3.3 [Reference 2 in the paper], I wish to explain where changes are required in order to execute under PC Version 5 (27 June 1992). For this purpose we can simulate the 5 by 6 array of nested cells previously used as a simple database:

`]d=.<"0 1.5 6`

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	17
18	19	20	21	22	23
24	25	26	27	28	29

The numbers identifying the cells can be called *linear indexes*, a term suggested to me by Eugene McDonnell. Thus, defining the verb *li* (a fork):

`li=. [ { 1.@$@]`

the linear indices of rows 1 and 4 are given by:

```
1 4 11 d
6 7 8 9 10 11
24 25 26 27 28 29
```

And the linear index of the cell at row 2 and column 5:

```
(<2 5) li d
17
```

The syntax now used for amend is illustrated by the following examples:

To place the array x in this cell:

```
x= < 9 8 7 6, :5 4 3 2
x 17) d
```

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	9 8 7 6 5 4 3 2
18	19	20	21	22	23
24	25	26	27	28	29

Note that a noun (17) and an adverb (:) result in a verb.

Scattered indexing presents no difficulty:

```
(x; 'London') ((2 5; 4 2) li d)) d
```

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	9 8 7 6 5 4 3 2
18	19	20	21	22	23
24	25	London	27	28	29

To expand the table and amend new rows 1 and 4:

```
exp=. /:@\:@[ { #@[{.]
e=. 1 0 1 1 0 1 1 exp d
t=. e (1 li d))~ 'Bob';'Bernecky';'Toronto';35;22000; 4 5
t (4 li d)~ 'Graham';'Woyka';'U.K.';62;35000;14 31 5 7
```

0	1	2	3	4	5
Bob	Bernecky	Toronto	35	22000	4 5
6	7	8	9	10	11
12	13	14	15	16	17
Graham	Woyka	U.K.	62	35000	14 31 5 7
18	19	20	21	22	23
24	25	26	27	28	29

```
y=. ('Graham';'Woyka';'U.K.';1;2;3),: 'Vin';'Grannell';'Los
Angeles';4;5;6 7 8
]z=. e (1 4 li e))~ 1 0{ y
```

0	1	2	3	4	5
Vin	Grannell	Los Angeles	4	5	6 7 8
6	7	8	9	10	11
12	13	14	15	16	17
Graham	Woyka	U.K.	1	2	3
18	19	20	21	22	23
24	25	26	27	28	29

Alternatively:

```
v=. 1 4&li
v e
6 7 8 9 10 11
24 25 26 27 28 29
u=. v@]
z=: e u~ 1 0{y
1
```

Although in these examples the indexes for amend are arbitrarily specified, they will often be computed as some function of the table; as, for instance, in the computation of eigenvalues by Jacobi's method [2].

Agreement

Given the following definitions:

```
mean=. +/%#
f=. ,mean
a=. 1.2 3 4 5
```

Then `a` consists of 2 items, each of shape 3 4 5. The mean of `a` is the sum of the corresponding cells of these two items divided by 2. Its shape is therefore 3 4 5:

```
$mean a
3 4 5
```

The hook `f` appends the means, consequently the shape is increased from 2 3 4 5 to 3 3 4 5:

```
$f a
3 3 4 5
```

Because the rank of `a` is 4, the same result is produced by:

```
$f"4 a
3 3 4 5
```

The rank conjunction ("`^`") lets us apply `f` to cells of rank 3, 2, or 1:

```
$f"3 a
2 4 4 5
$f"2 a
2 3 5 5
$f"1 a
2 3 4 6
```

This is clearer when the results are displayed, but the space required is more than is justified here. In each case the shape of the result is the same as the shape of `a` except that the length of one axis is increased by 1. The example illustrates the impressive power of the rank conjunction.

Given `p` and `q`:

```
p=. 1.2 3
q=. 2 3
```

then we can multiply the first row of `p` by 2 and the second row by 3

```

      p*"1 0 q
0  2  4
9 12 15

```

where each cell of rank-1 in the left argument is multiplied by the corresponding rank-0 cell of the right argument. The sentence $p*q$ formerly gave a length error, however the rule for agreement has been changed in J Version 5 so that now:

```

      p*q
0  2  4
9 12 15
      q*p
0  2  4
9 12 15

```

The rule is that the "frames of the arguments must match, or one must be a prefix of the other" [3, p.6]. In this case the arguments agree because the frames of the rank-1 cells of p and of the rank-0 cells of q are both 2.

Given:

```

y=. i.5
dev=. -mean

```

Then the deviations from the mean are:

```

dev y
_2 _1 0 1 2

```

The same result is returned when dev is written as:

```

dev=. -"1 0 mean

```

Because the right argument for the subtraction is rank-0 and the "ranks of a verb merely place upper limits on the ranks of the cells to which it is applied" [3, p.6], we can set the right rank to infinity ($_$):

```

dev=. -"1 _ mean

```

If y is rank-2:

The earlier rule for agreement required the shorter frame to match the suffix of the longer frame. Consequently with

```

dev=. -mean
y=. i.4 5

```

then,

```
dev y
```

which was formerly permitted [2, 4], now gives a length error. We must write, for example:

```
dev=. -"1 _ mean      NB. right rank is at least 1
$dev y
4 5
```

The 4 rank-1 cells (each of shape 5) are combined with the rank-1 cell (shape 5) resulting from mean y.

If y is rank-3:

```
y=. 1.3 4 5
```

then mean y is rank-2 and the deviations are computed by:

```
dev=. -"2 _ mean      NB. right rank is at least 2
```

The 3 rank-2 cells (each of shape 4 5) are combined with the rank-2 cell (shape 4 5) resulting from mean y.

If y is rank 4:

```
]y=. 1.2 3 4 5
```

then mean y is rank-3 and the deviations are computed by:

```
dev=. -"3 _ mean      NB. right rank is at least 3
$mean y
3 4 5
$dev y
2 3 4 5
```

The 2 rank-3 cells (each of shape 3 4 5) are combined with the rank-3 cell (shape 3 4 5) resulting from mean y.

It is unsatisfactory that the definition of dev had to change as the rank of the argument changed. Because the computation of the mean involves summation, the rank of the result of mean y is always one less than the rank of y. In each example the rank of the result is the rank of the original argument, but the rank of the cells used in the subtraction is one less than this. Although the rank of the

cells changes, the frame of the left argument is always of length 1; consequently we can generalise by referring to the frame of the cells instead of to their rank.

In J, "Negative numbers are used to specify cells whose frames are of length indicated by the magnitude of the number" [3, p.4]. In J Version 5 it is therefore possible to define a verb that applies to tables of any rank to give the deviations from the means of items:

```
dev=. -. _1 _ mean
```

i.e. the following all execute correctly:

```
dev ''
dev 5
dev 1.5
dev 1.4 5
dev 1.3 4 5
dev 1.2 3 4 5
```

Acknowledgements

It is a pleasure once again to thank Ken Iverson and Roger Hui for continuing to provide interesting and useful amendments to their system, and for guidance on many matters relating to this paper.

References

- [1] Donald B. McIntyre, *Using J's Boxed Arrays*. Vector, Vol.9, No.1 (July 1992) 92-105.
- [2] Donald B. McIntyre, *Jacobi's Method for Eigenvalues: an illustration of J*. Vector, Vol. 9 (1992) In Press.
- [3] Kenneth E. Iverson, *Dictionary of J*, Version 5 (1992) p.1-35, with Appendices p.1-18.
- [4] Donald B. McIntyre, *Mastering J*, APL91 Conference Proceedings, Stanford, California, August 1991. APL Quote Quad Vol. 21 Number 4 (August 1991), p.264-273.
- [5] Donald B. McIntyre, *Language as an intellectual tool: From hieroglyphics to APL*, IBM Systems Journal, Vol.30, No. 4 (1991) p.554-581.

An Example of Intransitivity in Probability

by Graham Parkhouse

This is something I wrote over a year ago. It is a wonderful illustration of how well APL can be used as a tool of thought: commenting, defining, calculating and displaying the way through a problem like this seems a very effective monologue. Being a dialogue with a computer might be a reason why it is better than a normal monologue!

The APL session contains almost the same amount of English commentary as the display of the results, with the APL definitions being a minor part. I approve of these proportions: output as manageable as the input, and about six parts of commentary to one part of code. It is an example which suggests an alternative to the commentary convention: that the interpreter should assume everything is commentary unless it meets an execute symbol, when it is to assume that everything to the right of it should be interpreted as APL.

```

⠇ This is a demonstration in APL of "dice odds intransitivity".
⠇ I have 4 dice, A,B,C, and D. On their 6 faces they are numbered
A←4 4 4 4 0 0
B←3 3 3 3 3 3
C←2 2 2 2 6 6
D←1 1 1 5 5 5

```

```

⠇ If I play A against C, there are these 36 possible outcomes
⠇ which I can show by comparing 6 sets of the 6 outcomes of A
⠇ with the 6 sets of the 6 outcomes of C

```

```

A*.>C
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
0 0 0 0 0 0
0 0 0 0 0 0

```

```

⠇ Which gives us a 1 for a win for A, and 0 for a loss, showing
⠇ that the probability of A winning over C is 16/36, or 4/9.
+/,A*.>C

```

16

Now let us find the odds of A against B, B against C etc.

+,[]+A.>B

```
1 1 1 1 1 1
1 1 1 1 1 1
1 1 1 1 1 1
1 1 1 1 1 1
0 0 0 0 0 0
0 0 0 0 0 0
```

24

A wins over B by $2^4/36$, or $2/3$

+,[]+B.>C

```
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
1 1 1 1 0 0
```

24

B wins over C, again with a probability of $2/3$

+,[]+C.>D

```
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 1 1 1
1 1 1 1 1 1
```

24

C wins over D with another $2/3$

Now, surely D will LOSE to A. But no!

+,[]+D.>A

```
0 0 0 0 1 1
0 0 0 0 1 1
0 0 0 0 1 1
1 1 1 1 1 1
1 1 1 1 1 1
1 1 1 1 1 1
```

24

D wins over A with a $2/3$ probability yet again!

This problem was introduced to me by John Allen Paulos in his book 'Innumeracy', subtitled 'Mathematical Illiteracy and its Consequences', Penguin, 1988. The problem, described on p. 100, is credited to Bradley Efron.

Index to Advertisers

Dyadic Systems Ltd	6
Manugistics	2
MicroAPL	8
APL Booklist (Renaissance Data Systems)	84

All queries regarding advertising in VECTOR should be made to Gill Smith, at 04393-385.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Authors wishing to use Windows Write should contact Vector Production for a copy of the Vector APL TrueType font. This is available encoded to the Dyalog APL, APL2/PC or (modified) STSC standards: STSC is currently the preferred layout, but either of the alternative layouts are acceptable.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 92/93

(these rates also apply to renewals)

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting UK member (student/OAP/unemployed only) £6		☐

PAYMENT — in Sterling only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____ | ____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Data Protection Act
The information supplied may be
stored on computer and processed
in accordance with the registration
of the British Computer Society.

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1992/93 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Anthony Camacho 0272-730036	11 Auburn Road, Redland BRISTOL, BS6 6LS.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 0488-648575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Duncan Pearson 0904-603510	c/o Operational Research Nestlé-Rowntree, YORK YO1 1XY
Projects:	George MacLeod 0442-878065	Greymantle Associates Ltd, Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Dr Peter Branson 081-848 9899	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Jonathan Barman	0488-648575
Secretary:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	Ray Cannon (0252-874697), Richard Weber, Sylvia Camacho, Duncan Pearson, David Ziemann (071-267 8032)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Compass R&D Ltd
15 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 0483-302249
Fax: 0483-302279

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants. RG24 0AL
Tel: 0256-811126
Fax: 0256-811130

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 071-353 4212
Fax: 071-353 3325

Cocking & Drury Ltd
180 Tottenham Court Road
LONDON W1P 9LE
Tel: 071-436 9481
Fax: 071-436 0524

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071-922 8866
Fax: 071-928 1006

Reuters Ltd
Reuters Nederland BV
PO Box 8230, 1005 AE Amsterdam
Netherlands
Tel: +31 20 570 8733
Fax: +31 20 570 8758

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel: 0225-462602
Fax: 0225-444552

Peter Cynax Systems
22 Hereford Road
LONDON W2 4AA
Tel: 071-229 5344

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: 03474-2337

Manugistics
2115 East Jefferson St
Rockville
MARYLAND 20852 USA
Tel: (301) 984-5412
Fax: (301) 984-5094